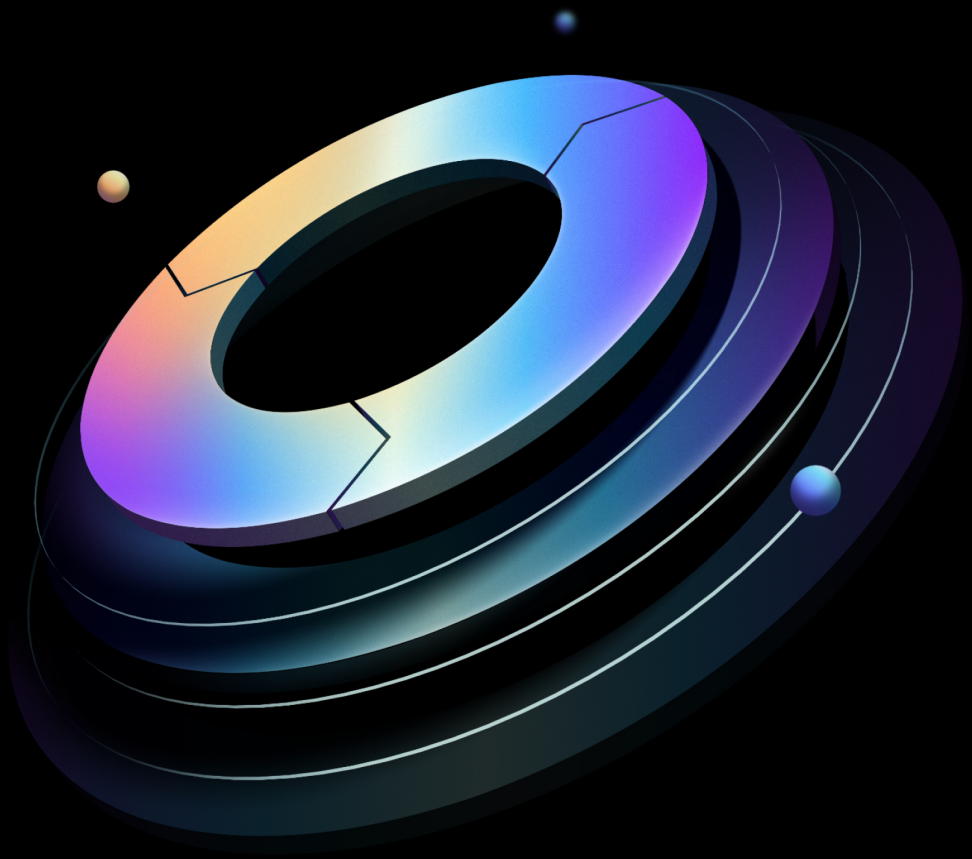
HashiCorp

Boundary: Operating Guide for Adoption

Built from commit 45cb599

HashiCorp | Validated Designs

10 July 2025



HashiCorp

Contents

1 Introduction

1.1 Why use HashiCorp Validated Designs?

HashiCorp introduced the Validated Designs program to provide customers with guidance based on extensive experience working with various organizations to deploy our solutions. The HashiCorp Validated Design (HVD) for Boundary offers a structured approach to implementing secure and automated user access management. By aligning with the recommendations in this document, you can achieve a production-ready service faster, establish a standard pattern for our support engineers to assist you efficiently, and enhance your ability to serve your customers, including application team developers, with optimized workflows.

Hashicorp Validated Designs provides prescriptive guidance curated from our experience supporting numerous customer journeys with Boundary.

1.2 Prerequisites

HashiCorp recommends the following prerequisites before implementing the Boundary Operating Guide for Adoption:

- Review Cloud Operating Model
- Review and implement the Boundary: Solution Design Guide
- Attend a Boundary workshop

HashiCorp recommends using Terraform to provision and configure Boundary. For guidance on using Terraform, please refer to the Terraform HVD.

1.3 Checklist

After completing the production readiness assessment, you are ready to implement core portions of the "Adopt" phase covered in this document.

1.4 Language and definitions

While this guide intentionally uses technology-agnostic language, there are some terms that do not translate seamlessly between providers. This document uses the following terms:

Term	Definition
Scope	A permission boundary modeled as a container for resources.
Global scope	The top-level scope that encompasses all child scopes within the boundary system. It serves as the root of the hierarchical structure to organize resources. The resources such as storage buckets, storage policies, aliases, workers, users, groups, roles and auth methods can be configured at global scope level.
Organ	The intermediate scope level, also referred to as organizations, are a child scope of global. Identity and access management related resources such as users, groups, roles and auth methods can be configured at the organization and global scope levels.
Project	The lowest scope level and a child scope of organization. It allows logical grouping of resources within an organization, such as targets, host catalogs, credentials stores and sessions.
Host catalog	A collection of hosts that Boundary can connect to, organized into host sets.
Host set	A subset of hosts within a host catalog which are considered equivalent for the purposes of access control.
Host	A resource with a network address reachable from Boundary, such as a server or database.
Target	A resource that ties network address information (via a direct address or by referencing host sets), credential libraries for injection or brokering (if desired) and port information to represent a networked service available for connection through Boundary. Targets also contain parameters, such as lifetime and connection count, to configure on the sessions created via authorization against the target.. A target can also be configured with ingress/egress worker filters that determine which workers are used to access targets.
Worke	A secure network proxy, enabling users to access private targets by establishing a direct network tunnel between the Boundary client on the user's machine and the target systems.
User	A resource that represents an individual person or entity for the purposes of access control. A user can be associated with zero or more accounts. A user authenticates to Boundary through an associated account and must be associated with at least one account before they can access Boundary.

Term	Definition
Group	A resource that represents a collection of principals, allowing them to be treated equally for access control purposes. As a principal itself, a group can be assigned to roles, and any role assigned to a group is indirectly assigned to all users within the group. A group can be defined at the global, organization, or project scope levels.
Managed group	A resource that represents a collection of accounts. The collection is formed by evaluating account information (e.g. LDAP groups, OIDC claims) defined by the auth method's identity provider against the managed group's configuration. An account can be associated with zero or more managed groups within the same auth method. It can be used as a principal in roles.
Role	A role is a collection of permissions granted to any principal assigned to it. Users, groups, and managed groups can be configured as principals in a role.
Authentication method	The mechanism by which users authenticate to Boundary. Can also be integrated with external identity providers like LDAP, Active Directory, or OIDC providers.
Account	A representation of a user's identity within a specific authentication method. Accounts can be created manually or automatically generated and are associated with a user in the same scope as the account's auth method..
Credential	Authentication details such as passwords, tokens, or keys used to access resources.
Credential store	Secure storage for managing and accessing credentials. This may be built-in to Boundary or contain information for accessing an external store like HashiCorp Vault.
Credential library	A collection of credentials of the same type from a single credential store that can be brokered or injected into the network session when users are accessing the networked services via sessions.
Session	A session is a set of related connections between a user and a host. A session may include a set of credentials which define the permissions granted to the user on the host for the duration of the session. Limits can be placed on the session, such as a maximum lifetime and/or a maximum connection count.
Session recordings	Recordings of user sessions for auditing and monitoring purposes.

Term	Definition
Storage Bucket	A container for storing session recordings and other data within Boundary.
Storage Policy	Rules and configurations governing the management and retention of data within storage buckets.
Availability zone (AZ)	A distinct data center within a region that provides redundant and isolated infrastructure to ensure high availability and failover protection. Each region consists of multiple AZs to offer resilience against system failures.
Region	A geographically distinct area that hosts multiple data centers, providing redundancy and fault tolerance for cloud services. Each region consists of multiple, isolated locations known as Availability Zones.
Instance	A physical or virtual server or hardware unit used for computing purposes.
Load Balancer	A hardware or software device used to distribute incoming network traffic across multiple servers.

1.5 Use cases covered

This document covers the “Adopting” phase of operating Boundary on the maturity model and includes the following:

Use Case	Summary
IDP Integration	Integrating identity providers with HashiCorp Boundary centralizes and secures user authentication, streamlining access management across your organization. By leveraging existing identity providers such as Okta, Auth0, Microsoft Entra ID (any OIDC based IdP), LDAP servers such as OpenLDAP, and Microsoft Active Directory, Boundary ensures that user access is consistent, secure, and easy to manage.

Use Case	Summary
Managed Groups	By integrating with identity providers like LDAP, Azure AD, and OIDC, Managed Groups enable dynamic user membership based on predefined criteria such as roles, departments, or project teams. This reduces administrative overhead and enhances security by ensuring that access policies are consistently enforced as users join, move within, or leave an organization.
Admin Governance	Boundary provides visibility into which identities access specific systems and allows you to control and terminate sessions automatically or manually. It creates a system of record for user access and actions during remote sessions, helping you maintain security compliance and enforce strong access controls.
Secure Proxy	Secure Proxy is a Boundary Worker capability that is responsible for proxying sessions between clients and targets, it has various operating modes, topology, and network connectivity requirements.
Credential Management	A set of credentials are required when a user tries to access a remote machine. There are two types of credentials - static and dynamic. Credential management refers to the management of these static and dynamic credentials using the concepts of credential brokering or credential injection.

2 People and process

Boundary requires thoughtful considerations around people and processes in order to maximize its value. It is also necessary to rethink how an important platform technology such as Boundary is positioned and offered. Changing culture and processes can be difficult, as can organizational and political challenges. Our observations across the industry give us a glimpse into what "good" looks like, which we share below. Please keep in mind that no two organizations are the same, and your specific circumstances may require you to choose how you organize your teams.

2.1 Internal developer platform

The Internal developer platform (IDP) has emerged to address the challenge of reducing development cycles and organizational complexity, especially as cloud computing and IaC have drastically cut down infrastructure provisioning times.

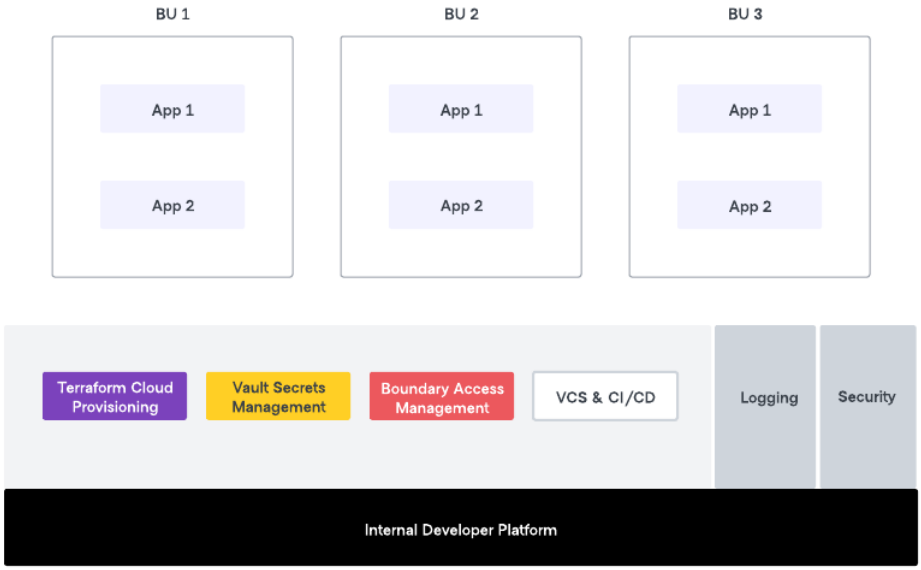


Figure 1: The internal developer platform

The IDP builds on the time-tested shared service model, improved by adding a product management approach. In this model, a Platform team works with developers to build golden paths that development teams can consume in a self-service model.

Access to infrastructure for developers and other stakeholders is a critical component and hence an Access Management service is an integral part of any IDP. We recommend that the team with ownership of the IDP (aka the Platform team) also take ownership of operating Boundary, with the Security team and other related teams providing governance.

With this approach, two key roles are identified: producers and consumers. The terms "Producer" and "Consumer" are used to define the roles and responsibilities within a shared service or IDP provided by a Platform team.

Note

While we talk of the "Platform team" as a singular team, it is possible that the "Platform team" is composed of multiple sub-teams with separate areas of responsibility. For example, Access Management components of the IDP might be owned by a separate team than Infrastructure provisioning components. These sub teams will however collaborate closely to ensure that developers in the various business unit app teams can easily access these shared services.

2.1.1 Producers

In a shared service model, the Producers have a role in configuring the system to meet the needs of the different consumers. Their responsibilities include:

- Operating Boundary and ensuring it is available as a tier-0, mission-critical service.
- Ensuring workflows are clearly defined for application teams.
- Set up integration between Boundary and Vault for dynamic credentials.
- Work closely with cross functional teams to identify and targets/hosts to Boundary.
- Offering necessary enablement to the consumers.

The Producers ensure that the shared services are readily accessible and efficiently utilized by the consumers, empowering them to leverage the benefits of the Boundary platform effectively.

2.1.2 Consumers

In a shared service model, consumers refer to any team within the organization that utilizes the shared services provided by the Platform team. These teams have responsibilities that include:

- Initiating requests for access to targets in Boundary.
- Perform tasks on target systems accessed via Boundary.

Consumers play a crucial role in leveraging the shared services to meet their specific needs and requirements while adhering to the guidelines and standards set by the Platform team.

2.2 Platform team

The Platform team is responsible for overseeing the overall implementation and management of the shared tools and services to enable golden workflows, and driving its adoption within the organization. When a Platform team matures, they will tend towards providing their consumers an integrated user experience (that implements golden workflows) using the IDP.

In the context of Boundary adoption, the Platform team ensures efficiency and standardization by establishing and enforcing standards and best practices, promoting consistency across project teams and minimizing the risk of errors and inconsistencies that can disrupt operations. They enable scaling and reusability by enabling automated "golden workflows" for common user workflows that streamline operations and reduce redundant efforts. These golden workflows include and are not limited to the setting up of Boundary organizations, projects, hosts and host sets (including dynamic host catalogs), credential libraries, credential stores and integrating them with Vault for dynamic credentials

By providing valuable knowledge and expertise in Boundary to the consumer community, they serve as a centralized resource for expert guidance. This reduces the learning curve for project teams and ensures consistent application of knowledge throughout the organization.

Automation and integration are facilitated by the Platform team, automating common tasks using IaC and integrating it with other DevOps tools. This streamlines processes, increases speed, and reduces manual errors. They also address the security aspect by ensuring adherence to security best practices in infrastructure setup and maintenance, reducing vulnerabilities to cyber threats.

Moreover, the Platform team significantly reduces the time to market for new features and applications. With a mature IaC/Terraform service, they expedite the deployment of new infrastructure, management of the various components of Boundary enabling faster time to market.

2.2.1 Automation/Tools team

The automation/tools team is a tactical component of the Platform team. They manage and maintain the automation and tools required for systems and services to operate effectively. They enable/implement "golden workflows" for consumers to leverage the shared services and tools that are part of the platform. As this team matures, they become the product owners and implementers of the IDP.

There could be multiple teams within the Automation/Tools team, each responsible for a different tool or group of tools. The CCoE provides overall strategic guidance for this team, and they collaborate with the consumer community to ensure that the services or workflows they enable meet their needs.

In simpler terms, the Automation/Tools team is responsible for making sure that the tools and automation that are used by the Platform team are working properly. They also work with the consumer community to make sure that the services and workflows that are enabled by these tools meet their needs.

2.2.2 Golden workflows

A golden workflow is a standardized, repeatable process for completing a specific task or achieving a specific outcome. It is typically well-documented and tested, and it is designed to be efficient and effective. Golden workflows are often used in software development, IT operations, and other business processes.

There are many benefits to using golden workflows. They can help to:

- Improve efficiency and productivity
- Reduce errors and mistakes
- Ensure consistency and quality
- Improve communication and collaboration
- Accelerate onboarding and training
- Facilitate automation

With Boundary, the Platform team enables golden workflows for federating identity and enabling secure access to remote target systems. The Platform team empowers AppDev and other teams to independently access target systems and perform their tasks bound with a session time-to-live (TTL) and dynamic ephemeral credentials while maintaining centralized management and control. The Platform team oversees and governs these workflows, ensuring compliance with policies, security requirements, and best practices.

2.2.3 Consumer workflows

These workflows are enabled by the Platform team but used by various consumers including but not limited to various application/development teams.

- **Developer Workflow**: In this workflow the developer is enabled to access remote target systems and perform their operations.
- **Target onboarding workflow**: This foundational workflow involves onboarding a target to Boundary. This involves attaching credential libraries and credential stores to a target, setting up automated target discovery workflows if applicable and setting up RBAC policies. This workflow will be leveraged by teams looking to onboard newer hosts for secure remote access.
- **Credentials onboarding workflow**: For dynamic credentials, this involves determining the location within HashiCorp Vault where secrets for a particular credential will be stored and ensuring the credential library can consume those secrets. This workflow will be leveraged by security teams responsible for the governance of HashiCorp Vault to setup dynamic credentials.

3 Initial configuration

After setting up a Boundary cluster, it's essential to perform initial configuration steps to ensure the environment is secure, functional, and ready for use. Below are the prerequisites and high-level steps you should follow.

3.1 Prerequisites

Boundary Enterprise

- You have reviewed the Boundary Enterprise deployment guide and have a running Boundary cluster and the database has been initialized.
- You have a valid recovery KMS as defined in the controller configuration file.

HCP Boundary

- You have reviewed the HCP Boundary setup instructions and have a running HCP Boundary cluster.
- You have authenticated to the HCP Boundary cluster using your admin credential.

3.2 High-level steps for initial configuration

> **Note**
>
> Steps 1-6 listed here apply only to the Boundary Enterprise version. For HCP Boundary, the system pre-creates the password-based authentication method, roles for global admin and anonymous users, and the global admin credentials during the HCP Boundary cluster setup.Steps 7-9 apply to both HCP Boundary and Boundary Enterprise.The commands below are run against the Boundary cluster. You can specify the Boundary cluster address using the `-addr` flag with the command or by exporting the `BOUNDARY_ADDR` environment variable before executing the commands.For example, `export BOUNDARY_ADDR=<HCP-BOUNDARY-CLUSTER-URL|BOUNDARY-ENT-CLUSTER-URL>`

1. Authenticate with recovery KMS
    - Start by authenticating to Boundary using the recovery KMS via the CLI. This grants you superuser privileges to configure initial resources. Please refer to log in with recovery KMS for detailed instructions.
2. Create an initial password auth method
    - Set up a password-based authentication method at the Global scope level. This method should be used only for the initial setup, testing, or as a fallback mechanism if the primary auth method (e.g., LDAP or OIDC) fails or is unavailable.
    - For example, run the below command to create a password auth method at the global scope level.

```
boundary auth-methods create password \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global' \
  -name 'password' \
  -description 'Password auth method'
```

3. Create a login admin account
    - Create a login account within the password authentication method. This account will be used to manage and configure Boundary. Please refer to create login account for the command to create the login account.

- For example, run the below command to create a login account.

```
boundary accounts create password \
  -recovery-config /tmp/recovery.hcl \
  -login-name "admin" \
  -auth-method-id <auth_method_id_from_last_step>
```

4. Create an admin user and associate with the login account
   - Create an admin user and link it to the previously created login account. You will configure this user with administrative privileges to manage Boundary resources in the next step.
   - Run the below commands to create and associate the user to the login account.

```
boundary users create -scope-id 'global' \
  -recovery-config /tmp/recovery.hcl \
  -name "admin_user" \
  -description "Global admin user"

boundary users add-accounts \
  -recovery-config /tmp/recovery.hcl \
  -id <admin_user_id> \
  -account <admin_account_id>
```

5. Create an admin role and assign to admin user
   - Define an admin role with permissions to manage all resources in Boundary. Assign this role to the admin user.
   - Run the below commands to create and assign the admin role to the admin user.

```
boundary roles create -name 'global_admin' \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global'

boundary roles add-grants -id <global_admin_role_id> \
  -recovery-config /tmp/recovery.hcl \
  -grant 'ids=*;type=*;actions=*'

boundary roles add-principals -id <global_admin_role_id> \
  -recovery-config /tmp/recovery.hcl \
  -principal '<admin_user_id>'
```

6. Create a role for anonymous (unauthenticated) users

   - Run below command to allow anonymous users to list scopes and auth methods in the global and organization scopes.

```
boundary roles create -name 'global_anon_listing' \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global

boundary roles add-grants -id <global_anon_listing_id> \
  -recovery-config /tmp/recovery.hcl \
  -grant 'ids=*;type=auth-method;actions=list,authenticate' \
  -grant 'ids=*;type=scope;actions=list,no-op' \
  -grant 'ids={{.Account.Id}};actions=read,change-password'

boundary roles add-grant-scopes -id <global_anon_listing_id> -grant-scope-id "
    children"

boundary roles add-principals -id <global_anon_listing_id> \
  -recovery-config /tmp/recovery.hcl \
  -principal 'u_anon'
```

7. Login using password auth method

   - Run below command to login using password auth method.

```
boundary authenticate password \
  -auth-method-id <auth_method_id>
```

8. Create an Organization and Project scope

   - Set up an Organization and a Project scope to organize resources and manage access control within Boundary.
   - Please refer to create organization and project scope for detailed instructions.

9. Create roles with administrative privileges at the Organization and Project scope levels.

   - Run below command to create organization-admin role.

```
boundary roles create -name 'org_admin' \
  -scope-id 'global'

boundary roles set-grant-scopes \
  -id <org_admin_id> \
  -grant-scope-id <org_scope_id>

boundary roles add-grants -id <org_admin_id> \
  -grant 'ids=*;type=*;actions=*'
```

- Run below command to create project-admin role.

```
boundary roles create -name 'project_admin' \
  -scope-id <org_scope_id> \
  -grant-scope-id <project_scope_id>

boundary roles add-grants -id <project_admin_id> \
  -grant 'ids=*;type=*;actions=*'
```

- Assign users, groups, or managed groups as principals to the organization-admin and project-admin roles after setting up the LDAP or OIDC auth method. Please refer to the respective sections in this document for the instructions.
- Run below commands to assign a principal (user, group, or managed group) to the organization-admin or project-admin roles.

```
boundary roles add-principals -id <org_admin_id> \
  -principal <user_id|group_id|managed_group_id>

boundary roles add-principals -id <project_admin_id> \
  -principal <user_id|group_id|managed_group_id>
```

4 Integrating identity providers

4.1 Overview

Boundary allows you to secure user access to your infrastructure endpoints (e.g., SSH, RDP, web/HTTPs, databases, kubectl, and any other TCP service) based on the user's identity. Before you configure role-based access controls to these endpoints, you must set up a user authentication method in Boundary. We recommend that you integrate Boundary with your existing iden-

tity provider to ensure a consistent and secure user authentication process across your organization. Boundary supports LDAP and OIDC, allowing it to integrate with various identity services and providers, including Active Directory, Microsoft Entra ID, Auth0, and Okta.

4.2 Prerequisites

We recommend that you have completed the following steps before implementing the guidance in this document:

1. You have reviewed and implemented the HashiCorp Validated Design (HVD) for Boundary Solution Design
2. You have a running Boundary cluster.
3. You have valid administrator credentials for your Boundary cluster to configure the authentication method.
4. You have your identity provider configuration details to establish a connection to Boundary.
    - LDAP/Active Directory: Server address (URL and port), TLS certificates, binding DN, and password. Please refer to the LDAP auth method attributes documentation for more information.
    - OIDC providers: Client ID, client secret, claims scopes, callback and redirect URLs. Please refer to the OIDC auth method attributes documentation for more information.
5. You have configured the network firewall to allow Boundary servers to communicate with your identity provider over the required network ports and protocols, described in the HashiCorp Validated Design (HVD) for Boundary Solution Design.

4.3 Roles and responsibilities

Below is the list of roles and responsibilities related to integrating identity providers and configuring authentication methods in Boundary.

Role	Responsibility
Platform Team	Configures and manages resources in Boundary. In relation to this use case, the platform team manages Boundary organizations, configures the authentication method, and creates and assigns the organization admin role.

Role	Responsibility
Identity Management Team	**LDAP/Active Directory:** Manages LDAP user credentials and group memberships. Provides credentials to allow Boundary to query and authenticate users. **OIDC:** Manages the OIDC provider and user claims, registers Boundary as a new client with the provider, and provides necessary client credentials to the platform team.
Network Team	Configures network firewall rules to ensure Boundary servers can communicate with your identity providers over the required network ports and protocols. LDAP/Active Directory: Ensure port TCP-389 (LDAP) or TCP-636 (LDAPS) is open and accessible.OIDC: Ensure port TCP-443 (HTTPS) is open for secure communication with OIDC providers.
Security Team	Ensures compliance with organizational security policies and standards.
Boundary Organization Admin	Manages resources in a Boundary organization, configures managed groups leveraging LDAP groups or OIDC user claim attributes, and configures RBAC policies based on managed groups.

4.4 Configuring authentication method

Many organizations choose to implement Boundary as a service, with a central platform team managing the daily operations while development teams utilize Boundary's capabilities. Enabling this model requires a mechanism to isolate groups of resources within a single cluster. As mentioned above, the global scope is the outermost scope and serves as the entry point for initial administration, setup, and management of the organization scopes. Organizations hold IAM-related resources and project scopes. You create one or more project scopes within an org based on your business workflows. It is recommended that you create a project scope to organize Boundary resources such as host catalogs, targets, credential stores, and access controls by specific products or teams. A scope can be considered as a container for resources and permissions. Each scope has a permissions boundary that is isolated from other scopes on the same level, creating a defined blast radius.

For your users to securely connect to the targets, they need to be both authenticated and authorized. An authentication method is a resource that provides a mechanism for your users to authenticate to Boundary. It contains accounts that link an individual user to a set of credentials and groups. Principals are entities in Boundary that can be users or groups, and are assigned capabilities. We recommend that you assign users to groups and use these groups as principals in roles for simplified access control management. Roles in Boundary are a collection of zero or more grants that are assigned to principals and govern what actions they are authorized to perform. You can define roles within any scope. A role's lifecycle is dependent on the existence of its scope, meaning the role is deleted if the scope containing the role is deleted. The diagram below illustrates the relationship between the different IAM components in Boundary.

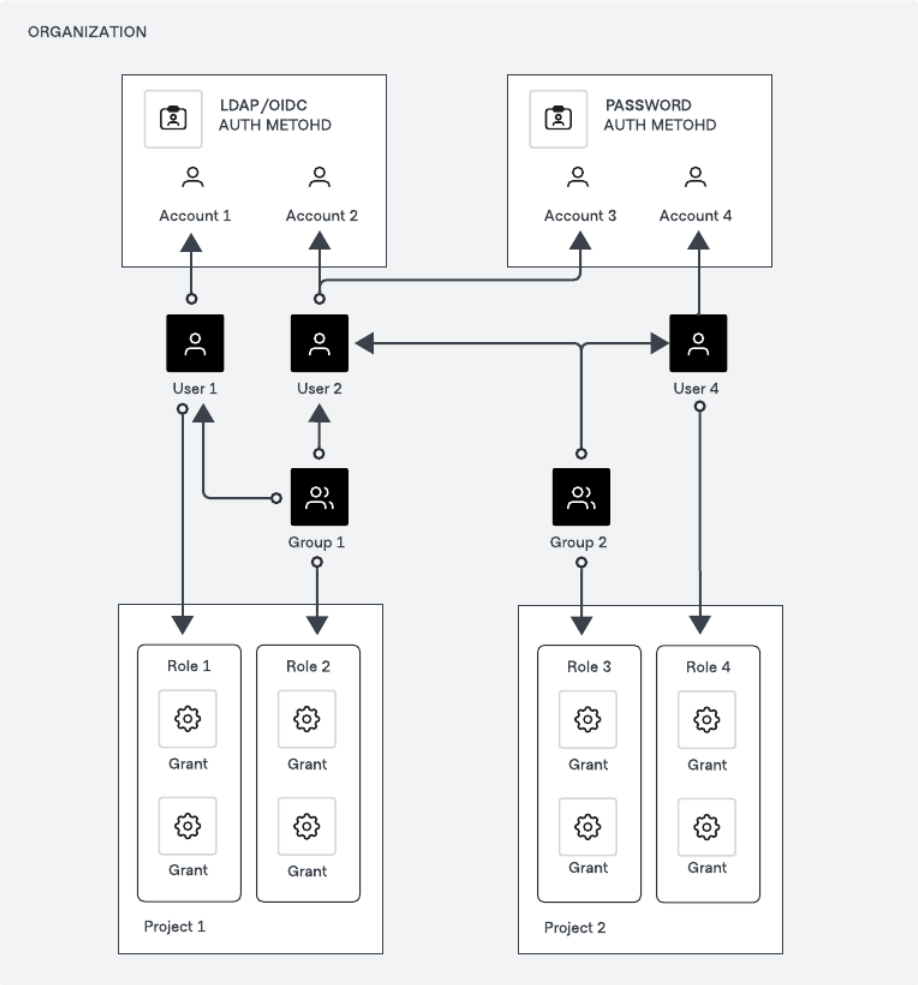


Figure 2: Boundary IAM Model

You can configure authentication methods at either a global or organization scope level. It is recommended to configure an authentication method at the global scope level as illustrated in below diagram to provide a consistent authentication experience for all users across different business units. This approach is ideal for enterprise environments where multiple departments or teams use a central authentication system.

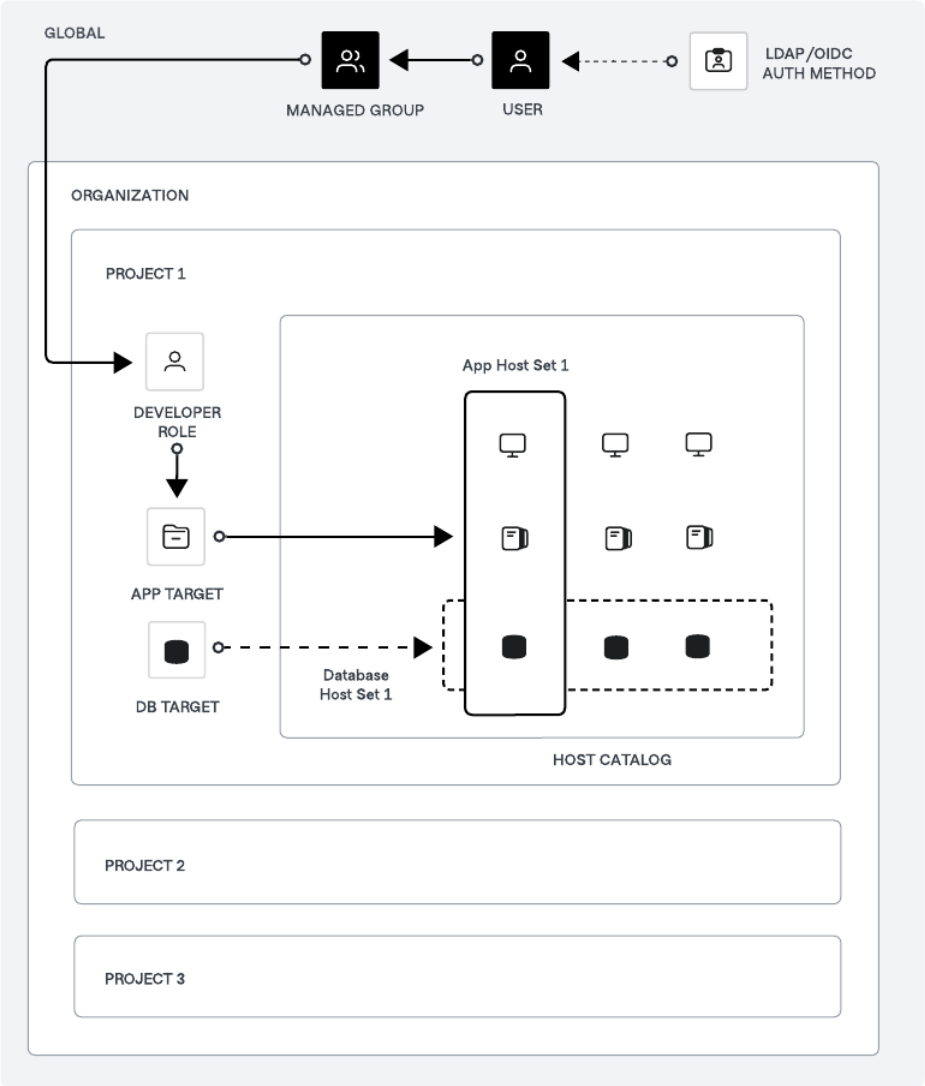


Figure 3: Boundary Auth Method Structure

You can also configure the authentication method at the organization scope level to meet specific requirements. For instance, you can configure a password authentication method at the org level with a few accounts for your contractors to provide them access to your resources without integrating these accounts into your primary identity provider, such as Active Directory, Microsoft Entra ID, or Okta.

A user or group assigned to a role at the project scope level can only perform actions on resources within that specific project. For instance, a user assigned the developer role in the illustration above can access hosts associated with the app target solely within that project. Consider using an organization scope level role if you need to grant the same permissions to users or groups across multiple projects. Using project scope roles ensures that access permissions are tightly controlled and limited to specific projects, enhancing security by isolating access. In contrast, org scope roles provide a broader permission set that can simplify management and ensure consistency across multiple projects.

Boundary supports several authentication methods, enabling you to leverage your existing identity providers for user authentication. Below are the available authentication methods:

- LDAP
- OIDC
- Password

4.4.1 LDAP/Active Directory auth method

The LDAP auth method allows your users to login using their existing LDAP username and password credentials. The LDAP auth method connects directly to your organizations' LDAP servers. Boundary must be able to communicate with the LDAP server over TCP/UDP-389, or TCP-636 for LDAPS. Please refer to LDAP Authentication for detailed instructions to configure the LDAP auth method.

We recommend you to set the authentication method status to private initially to validate the configuration without exposing it publicly. In this state, the authentication method is not visible to unauthenticated users but can still be used to authenticate users. Once testing is successful, set the authentication method status to public to allow all users to authenticate via LDAP.

When an authentication request is made through the auth method, Boundary verifies the provided credentials with the backend LDAP server through a service account. Authentication requests can be initiated using the Boundary UI, CLI, API, or the Boundary Desktop.

The first time a user successfully authenticates using an LDAP auth method, Boundary creates a new account using the user's account login name. If groups are enabled for an LDAP auth method, then each time a user authenticates, Boundary updates their account's group memberships. We recommend that you configure the managed group resource in Boundary to automatically associate the user account to the group based on the LDAP account's associated groups. Managed groups allow you to assign roles within Boundary based on an LDAP account's group memberships.

4.4.2 OIDC auth method

The OIDC auth methods allows Boundary users to authenticate via identity providers such as Okta, Auth0 or Microsoft Entra ID. The OIDC auth method allows a user's browser to be redirected to a configured identity provider to complete their login. Once authenticated, the user is directed back to Boundary's UI, CLI, API or Boundary Desktop. Please refer to the documents below for detailed instructions to configure the authentication method with your identity provider.

- OIDC authentication with Auth0
- OIDC authentication with Okta
- OIDC authentication with Azure

We recommend you to set the authentication method status to private initially to validate the configuration without exposing it publicly. In this state, the authentication method is not visible to unauthenticated users but can still be used to authenticate users. Once testing is successful, set the authentication method status to public to allow all users to authenticate via OIDC.

Authentication requests can be initiated using the Boundary UI, CLI, API, or Boundary Desktop. The first time a user successfully authenticates using OIDC provider auth method, Boundary creates a new account using the claims returned from the provider. We recommend that you configure the managed group resource in Boundary to automatically associate the user account to the group based on OIDC account attributes. You can configure managed groups with claims from the JSON Web Token (JWT), or the claims from the UserInfo endpoint.

4.4.3 Password auth method

The password auth method allows you to configure user accounts directly in Boundary.

You can consider password authentication method for the following use-cases:

1. Initial setup and testing
    - Allows you to quickly set up and test Boundary configurations.
    - Ensures Boundary is properly configured and functioning before integrating with other authentication methods like LDAP or OIDC.
2. Backup authentication method
    - Serves as a fallback authentication mechanism if the primary method (e.g., LDAP or OIDC) fails or is unavailable.
    - You can continue to access and manage Boundary even if external authentication systems experience downtime or connectivity issues.
3. Isolated environments
    - Allows you to authenticate users in isolated or air-gapped environments where external identity providers cannot be accessed.
4. Temporary access
    - You can grant temporary access to users, such as contractors or temporary staff, without integrating them into the primary identity provider.

4.5 Summary

In this section, you reviewed the recommended approach to securely authenticate users to Boundary. The preferred approach is to use your existing identity provider for centralized authentication management, enhanced security, and improved operational efficiency. You also reviewed the recommended approach to creating Boundary organizations and auth methods. Finally, you configured an auth method leveraging your existing identity provider. In the next section, we will cover managed groups to dynamically maintain groups of users based on specific criteria or rules such as LDAP groups or OIDC claims.

4.6 Useful resources

- Identity management
- Identity and access management (IAM) | Boundary | HashiCorp Developer
- Domain model index | Boundary | HashiCorp Developer

- LDAP auth method attributes
- OIDC auth method attributes

5 Managed Groups

5.1 Overview of managed groups

As organizations grow, the administrative tasks of manually managing users and group memberships becomes increasingly challenging. HashiCorp Boundary's managed groups feature provides significant benefits for enterprises to automatically manage OIDC and LDAP group membership at scale.

5.1.1 Centralized identity management

By integrating with enterprise identity providers (IdP) like Okta, Auth0 via OIDC, or Active Directory via LDAP, Boundary allows enterprises to leverage their existing user directories and group structures. We recommend using the Terraform Boundary Provider for onboarding and offboarding of Boundary resources such as users, groups and managed groups, etc.

5.1.2 Automated group synchronization

Managed groups automatically sync membership based on OIDC group claims from the identity provider (IdP). As your IdP administrator adds or removes users from groups in the central IdP, their access privileges in Boundary are updated accordingly without any manual intervention.

5.1.3 Scalable access control

With automated group management, enterprises can define role-based access controls (RBAC) in Boundary and map OIDC groups to these roles. By automatically revoking access when users leave the organization or change roles, managed groups help enforce the principle of least privilege and reduce the risk of unauthorized access. Please take note that the synchronization is only updated when users authenticate again to Boundary via OIDC/LDAP to reflect the changes. This allows for granular and consistent permissions management at scale across the entire organization. For

offboarding, when users leave the organization and are removed from the IdP group, Boundary will automatically remove them from the managed group, significantly reducing the risk of unauthorized access. This process ensures compliance with regulatory requirements and internal security policies by providing audit trails and consistent policy enforcement. Moreover, the reduced need for manual intervention minimizes administrative overhead and potential for human error.

5.1.4 Reduced administrative overhead

By eliminating manual user and group provisioning, managed groups significantly reduce the administrative burden on your teams, especially in large enterprises with thousands of users and dynamic team structures.

5.2 Managed groups implementation and operations

Boundary organization administrators should consider the following guidelines to implement and operate managed groups.

1. **Integrate with enterprise IdP**: The first step is to configure Boundary to authenticate users with the enterprise's OIDC-compliant identity provider (e.g., Okta, Auth0, Microsoft Entra ID). This can be done through the Boundary UI, CLI or Terraform.
2. **Defined managed group filters**: Boundary admins should create managed groups with filters that select users based on OIDC claims from the IdP or LDAP account's group memberships. For example, a filter like `engineering` in `/userinfo/groups` would automatically add all users from the "engineering" group in the OIDC IdP to the corresponding managed group in Boundary. For LDAP, we can use `group_filter` queries to return groups objects and user objects to resolve group membership according to the structure of your directory schema.
3. **Map managed groups to roles**: Managed groups should be assigned as principals to Boundary roles, which define the specific permissions and access controls for that group of users.
4. **Automate with Infrastructure-as-Code**: Enterprises operating at scale should use Terraform to define and manage Boundary resources, including OIDC auth methods, managed groups, and role mappings. This enables version control, automated deployments, and consistent configuration across multiple Boundary instances.

5.3 User authentication and authorization workflow with managed groups

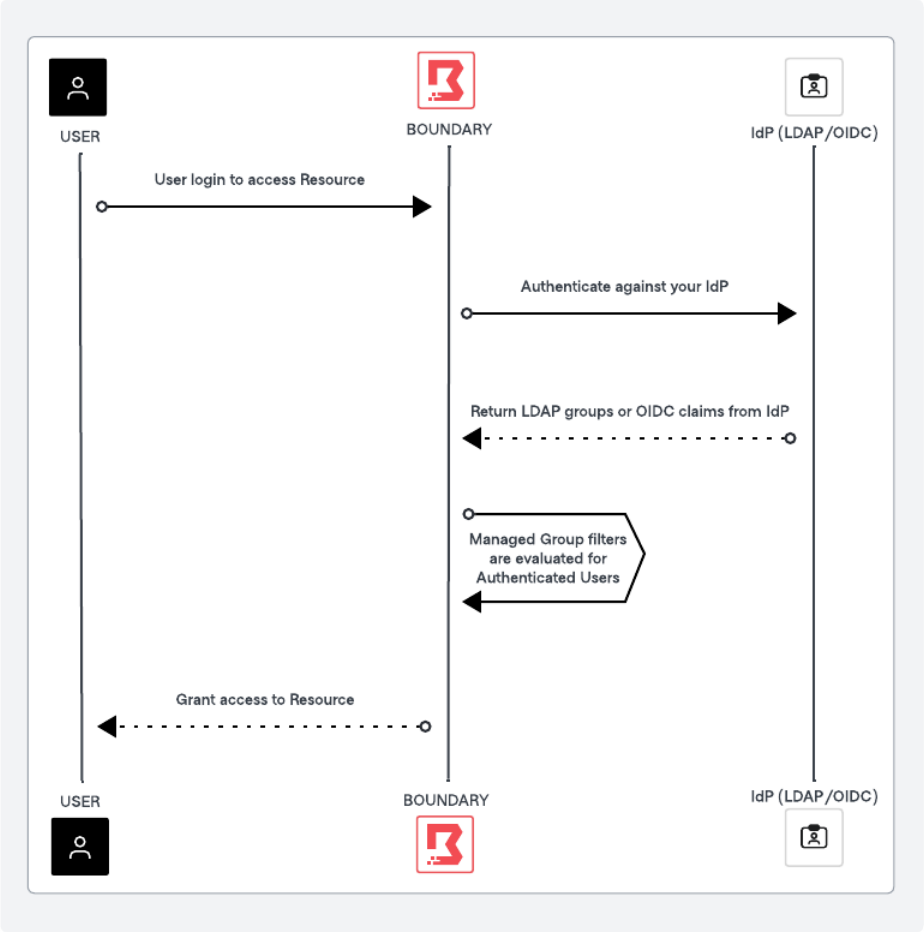


Figure 4: Boundary Managed Group Workflow

The diagram above outlines the high level process of user authentication and authorization using Boundary managed groups and IdP providers such as LDAP Active Directory or OIDC providers such as Auth0 or Okta. The key steps involve user authentication via LDAP/OIDC provider, token issuance, claim extraction, managed group evaluation, and role-based access control within Boundary.

1. **User initiates authentication**

- User logs onto Boundary using Boundary Desktop or CLI.
- Boundary redirects the user to the LDAP/OIDC provider login page.

2. User authenticates with LDAP/OIDC provider
   - The user enters their credentials on the LDAP/OIDC provider login page.
   - LDAP/OIDC provider verifies the credentials and authenticates the user.
3. LDAP/OIDC provider issues ID token
   - Upon successful authentication, the LDAP/OIDC provider issues an ID token and redirects the user back to Boundary with the token.
4. Boundary receives ID token
   - Boundary receives the ID Token from LDAP/OIDC provider.
   - Boundary extracts LDAP groups (or) OIDC claims from the ID Token and the UserInfo endpoint.
5. Evaluate managed group membership
   - Boundary evaluates the user's claims against the managed group filters.
   - If the user's claims match the filter criteria, the user is added to the corresponding managed group.
6. Assign roles and permissions
   - Managed groups are associated with specific roles in Boundary.
   - The user inherits the permissions associated with the roles of the managed groups they belong to.
7. Access granted
   - The user is granted access to the requested resource based on the permissions of the roles they have been assigned.

5.4 Configuring managed groups with LDAP (or) OIDC provider

As a prerequisite, Boundary administrators need to set up managed groups using either LDAP authentication or OIDC authentication. Boundary then uses information from these identity providers to automatically create accounts and users within the same scope as the authentication method.

5.4.1 Setup LDAP auth method

Please refer to LDAP authentication for detailed instructions to configure the LDAP auth method, and an exhaustive list of LDAP auth method attributes configuration parameters.

5.4.2 Setup OIDC auth method

Please refer to the documents below for detailed instructions to configure the authentication method with your identity provider, and an exhaustive list of configuration parameters.

1. OIDC authentication with Auth0
2. OIDC authentication with Okta
3. OIDC authentication with Azure

5.5 Useful resources

The following resources help you to implement, troubleshoot and resolve managed groups related issues.

- OIDC managed group information and attributes
- LDAP managed group information and attributes
- Filter managed groups
- Managed OIDC IdP groups
- Terraform Patterns for Boundary groups and RBAC

6 Assigning scopes, principals, roles and grants

6.1 Scopes

In Boundary, scopes help in structuring and managing principals. They provide a framework for organizing resources and permissions that enables effective identity and access management. Scopes are structured hierarchically, with three main levels:

- Global - the highest-level scope of Boundary where administrators configure and manage Boundary for the entire company.

- Organization - organization scopes are contained in the global scope and are typically used to represent business units, organizations, or departments. Organization scopes can also be used to contain multiple production-level auth methods in separate scopes.
- Project - project scopes are contained in Organization scopes and be used to separate different business workflows. For example, projects can be used to represent different teams, products, or environments.

6.2 Assigning principals

In Boundary, principals refer to entities that can be assigned roles and granted permissions. The main types of principals in Boundary are:

- Users: These are individual accounts representing human users or machines. Users can be associated with one or more accounts via authentication methods.
- Groups: These are collections of users. A group can include multiple users, allowing for easier management of permissions by assigning roles to the group rather than individual users.
- Managed Groups: Managed groups are typically used in conjunction with identity providers like LDAP or OIDC, where membership is determined based on authentication attributes.

These principals can be assigned to roles, which define the permissions they have within Boundary. Each principal is identified by a unique ID, which is used when assigning them to roles.

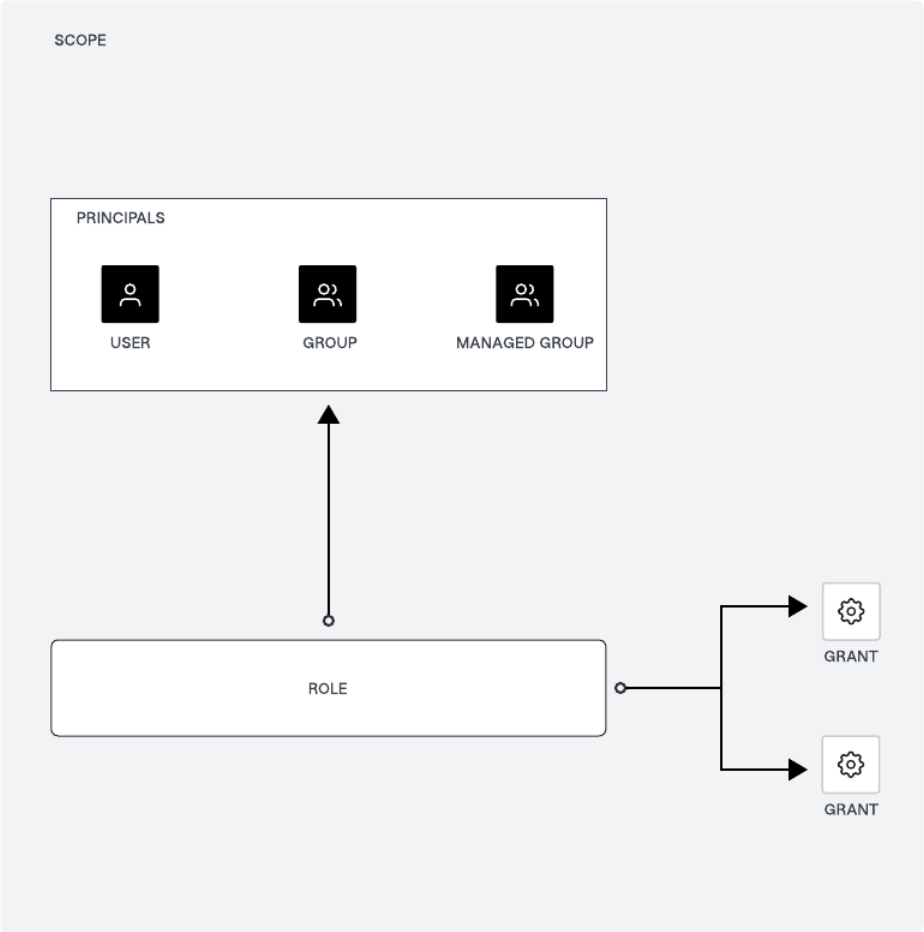


Figure 5: Boundary Principals and Roles Relationship

6.3 Assigning roles to managed groups

We recommend adding a user to a group, and then adding the group to the role(s), instead of adding the user directly to the role(s). This way, you can manage multiple users at the same time, and it is easier to change the permissions of the user by adding/removing them from groups. You can manage OIDC/LDAP users and managed groups the same way, directly in the identity provider.

Roles are collections of capability grants, which are permissions allow roles to take actions and

access resources.

1. Create a role
    - Firstly, you need to create a role within Boundary. Roles can be defined at the global, organization, or project scopes. A role is a collection of grants that can be assigned to principals, which include respective managed groups.
2. Assign principals to the role
    - Once the role is created, you can assign managed groups as principals to that role. This is done by adding the unique ID of the managed group to the role's principal IDs. This step effectively links the managed group to the role, allowing its members to inherit the permissions defined by the role.

6.4 Grant permissions to roles

With the managed group created and automatically managing group membership based on auth methods configured, the next step is to define grants for those principals which are your managed groups.

1. Define grants: Grants are defined using grant strings, which specify the actions that can be performed on resources. When you define grants permissions, please consider RBAC and Principle of Least Privilege (PoLP) in mind. These grant strings are added to the role. All grants take one of four forms: ID only, type only, pinned ID, or wildcards. The following is an example of a grant string:

```
ids=<id>;type=<type>;actions=<action list>;output_fields=<fields list>
```

    - `ids`: Specifies the resource IDs the grant applies to. Wildcards can be used to apply the grant to all resources of a type.
    - `type`: Specifies the type of resource, such as host-catalog, auth-method, group, etc.
    - `actions`: Specifies the actions allowed on the resources, such as read, list, create, etc.
    - `output_fields`: Specifies which fields of the resource should be visible.
2. Assign Grants to the Role: After defining the grants, they are assigned to the role. This process involves specifying the grant strings in the role configuration. Once the grants are assigned, the managed group (as a principal) will have permissions to perform the specified actions on the resources.

6.5 Example scenario

In our example scenario, we will start with creating the following roles at the Global scope level for boundary administrators and boundary viewers:

Global scope roles:

- `boundary_global_admin` role has the grant of:

```
ids=*;type=*;actions=*
```

- `boundary_global_viewer` role has the grant of:

```
ids=*;type=*;actions=read,list
```

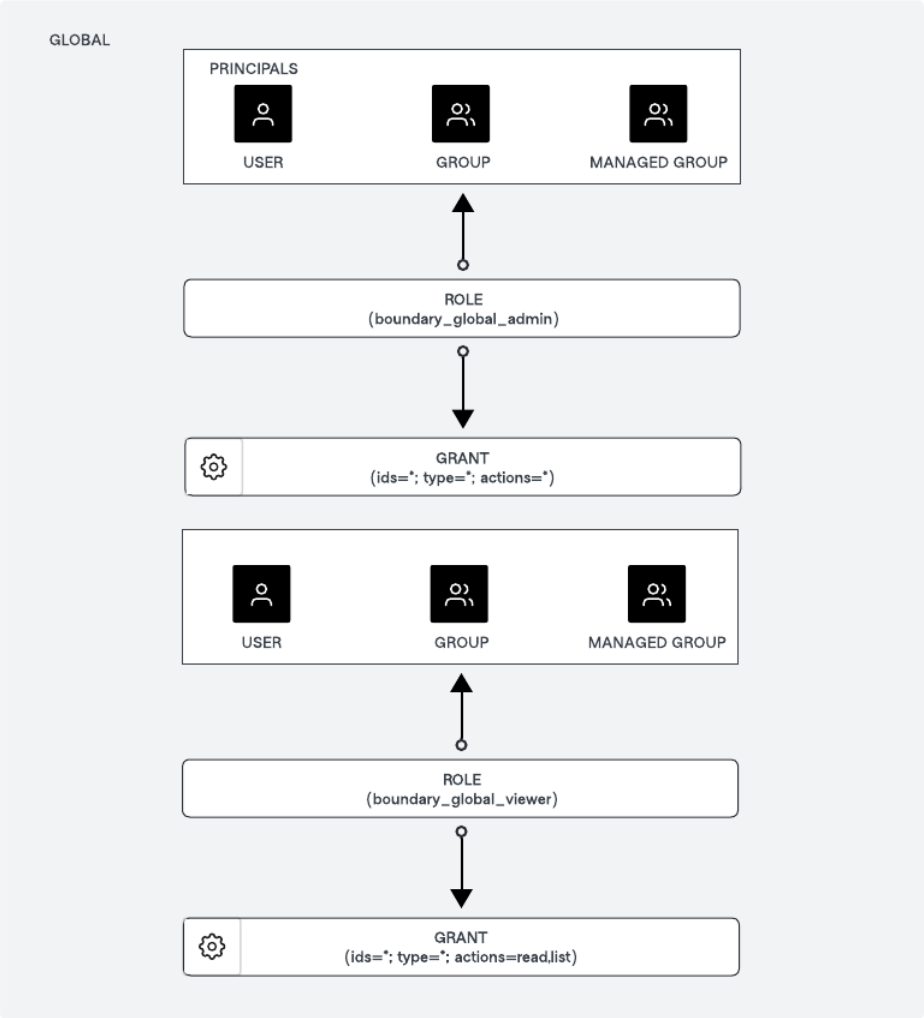


Figure 6: Boundary Principles Roles Grants Global

The next step is that we will create a role at the Organization level scope to manage the organization and roles at the Project level scope for multiple projects within an organization, and also the roles to access the targets. (Note: You can scale multiple organizations, and multiple projects based on your organizational structures.)

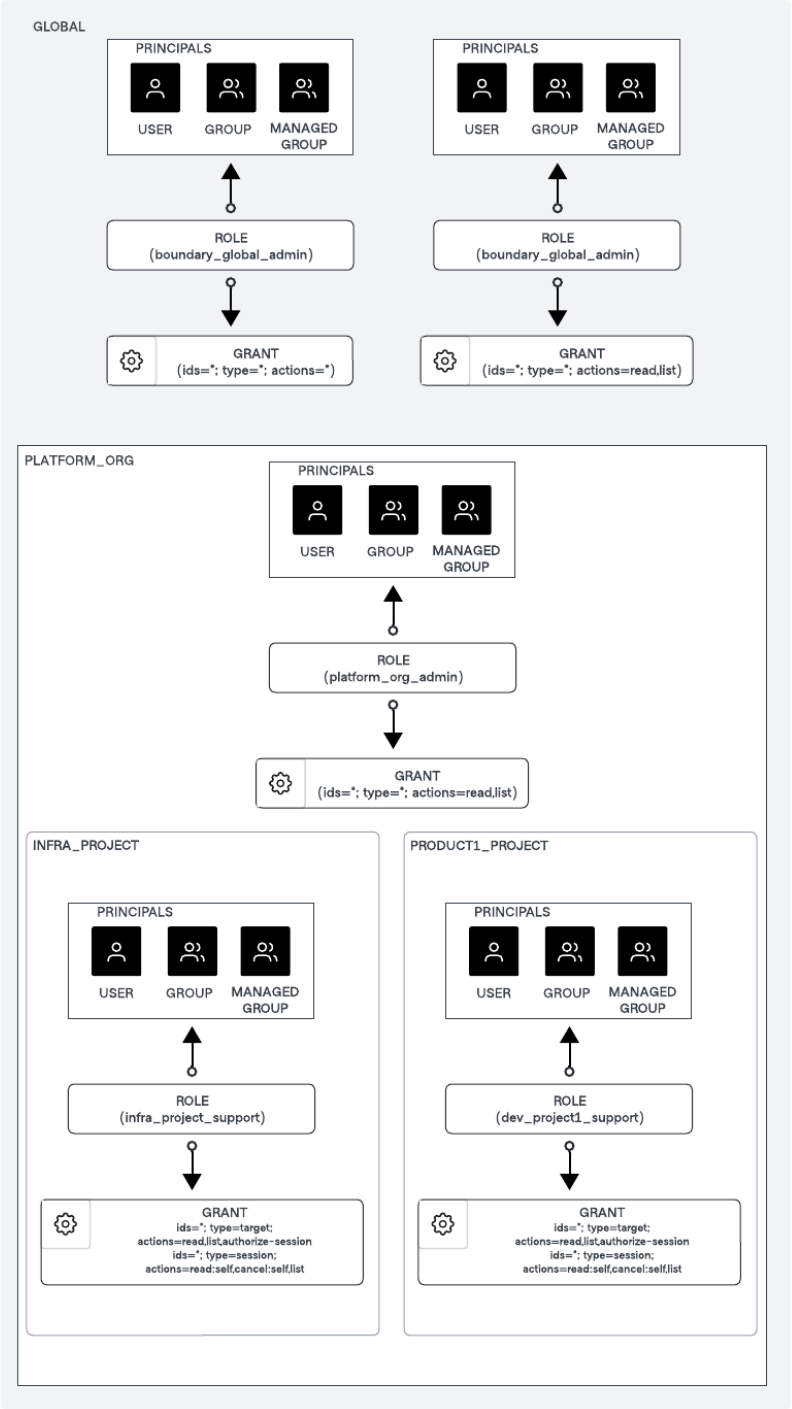


Figure 7: Boundary Principles Roles Grants Organization and Project Scope

Organization scope role for *PLATFORM_ORG*:

- `platform_org_admin` role has the grant of:

```
ids=*;type=*;actions=*
```

Project scope role for INFRA_PROJECT:

- `infra_project_support` role has the grant of:

```
ids=*;type=target;actions=list,read,authorize-session
ids=*;type=session;actions=read:self,cancel:self,list
```

Project scope role for PRODUCT1_PROJECT:

- `dev_project1_support` role has the grant of:

```
ids=*;type=target;actions=list,read,authorize-session
ids=*;type=session;actions=read:self,cancel:self,list
```

6.6 Useful resources

- Refer to Assignable permissions for more information about the permissions you can assign to Boundary principals.
- Refer to Permission grant formats for more information about grant strings and example formats.
- Refer to Manage roles and permissions for instructions to configure roles and grant scopes for principals.
- Refer to the Resource table for a cheat sheet to help you manage your permissions.

7 Administrative governance

7.1 Overview

Boundary provides you with detailed visibility into which systems are accessed by which identities and offers administrative controls to automatically or manually terminate sessions as needed.

Boundary establishes a system of record for your users' access and actions during remote sessions. This capability allows you to maintain security compliance and ensure robust access controls within your environment.

With Boundary, you can:

- **Monitor access**: Track which systems are accessed and by whom, ensuring comprehensive visibility into your users' activities.
- **Manage sessions**: Automatically terminate sessions based on predefined policies or manually end suspicious or unauthorized sessions, providing immediate response capabilities.
- **Maintain compliance**: Generate audit trails to meet regulatory requirements and security standards.
- **Enforce access controls**: Implement fine-grained access controls and policies to secure your environment against unauthorized access.

7.2 Managing sessions

A session represents a set of connections between a user and a target. A target allows you to define an endpoint with a protocol and default port to establish a session. A session may include a set of credentials which define the permissions granted to the user on the target for the duration of the session.

7.2.1 Session initiation

The session begins when an authorized user requests access to a target. Boundary sets the expiration time and connection limit for the session if you have configured these attributes on the target. The default session duration is set to 8 hours (28,800 seconds), after which all connections associated with the session are closed, and the session is terminated. If the target is associated with credential libraries, credentials are retrieved and returned from each credential library. Sessions are created in the project scope of the corresponding target. Deleting a project will terminate all of the active sessions in the project.

7.2.2 Monitoring sessions in real-time

You can view active sessions in real-time, including details like the user's identity, the targets they are accessing, the session start time and the current session status (i.e. active, pending, canceling or terminated).

You can use boundary CLI, desktop app or browser based admin UI to list all sessions.

For example, run the below command to list all sessions across all your projects using CLI.

```
boundary sessions list -scope-id global -recursive
```

To view details of a specific session, use the "boundary sessions read" command with the session ID.

```
boundary sessions read -id <session-id>
```

Similarly, you can use the browser-based admin UI and navigate to "Sessions" for a given Boundary organization and project to list all sessions.

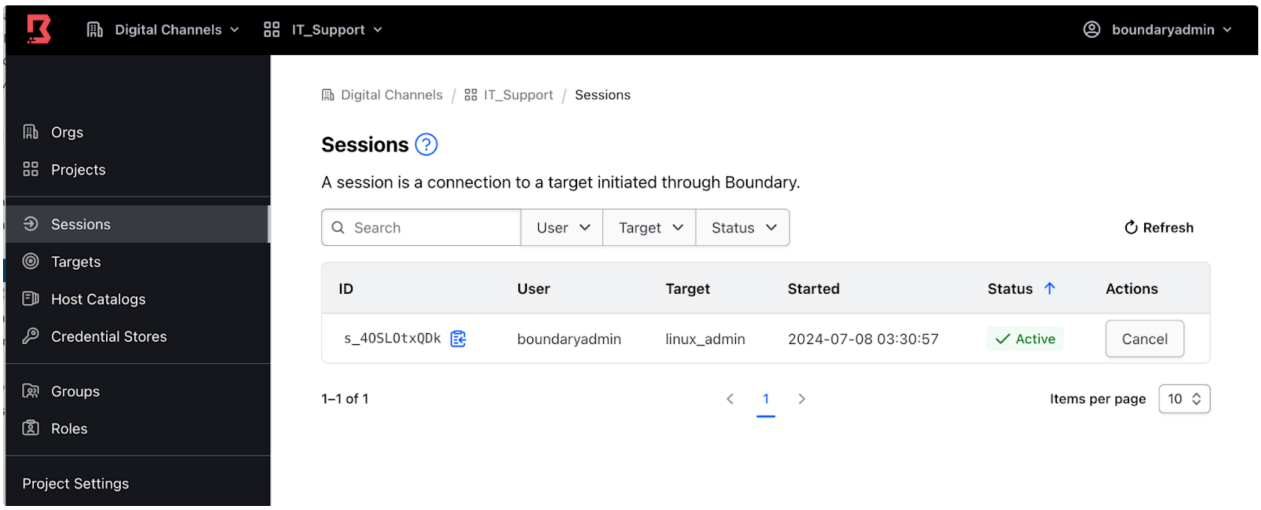


Figure 8: Boundary Sessions Management Active

7.2.3 Session logging

Boundary logs audit events related to user sessions, such as the creation or cancellation of a session. These logs capture critical details including user's identity, session start and end times, and resources accessed. Audit logs allow you to track user activity and enable security teams to ensure compliance in accordance with regulatory requirements.

HCP Boundary supports near real-time streaming of audit events to supported providers, currently including Datadog and AWS CloudWatch. This feature ensures that security teams have immediate visibility into user activities and potential security incidents. For self-managed Boundary Enterprise, we recommend streaming audit events to your existing centralized logging solution using log shippers. This approach integrates Boundary's audit logging with the organization's existing monitoring and alerting infrastructure. The example below demonstrates an audit event captured when a new session is initiated by a user to a remote host:


```
{
    "session_id:s_wYID78DBFL"
    "target_id:tssh_N5r14ExLV7"
    "scope:id:p_12BUBsbRog"
    "scope:type:project"
    "scope:name:IT_Support"
    "scope:description:IT Support"
    "scope:parent_scope_id:o_QljIK3QKUc"
    "created_time:seconds:1720352717"
    "created_time:nanos:940476000"
    "user_id:u_SpwJO5YyPh"
    "host_set_id:hsst_I25uYGYFOM"
    "host_id:hst_JCTxpHCzQ2"
    "type:ssh"
    "authorization_token:[REDACTED]"
    "endpoint:ssh://10.200.20.213:22"
    "endpoint_port:22"
    "expiration:seconds:1720381517"
    "expiration:nanos:931225000"
}
```


Please refer to the audit logging section for more details.

7.2.4 Session recording

Boundary also provides auditing capabilities via session recording which is useful for high-security environments where monitoring user actions is critical for regulatory and compliance. A session recording is associated with a target. The session recording captures all interactions that take place during the session, including metadata about the user, target and any hosts, host sets, host catalogs, or credentials used. A session recording represents a directory structure of files in an external object store that together are the recording of a single session between a user and a target.

Sessions are recorded by the Boundary workers. Workers are the proxy between an end user and a target. A session recording represents connections as separate entities within the recording. Each recorded connection may also contain a recorded channel. This represents a single channel in which the user interacts with the target in protocols that multiplex user interactions over a single connection. For example, the SSH protocol multiplexes user interactions in a single connection, so a user's interactions over SSH are recorded in a channel.

You can replay recorded sessions through the Boundary admin UI. This feature allows you to review user actions and investigate incidents, providing context on user actions during that session. Please refer to the find and view recorded sessions for more details. You can define how long session recordings are stored based on organizational policies and compliance requirements.

Please note that the session recording is currently supported only for SSH protocol.

7.2.5 Session termination

Boundary enables you to manage and terminate sessions both automatically and manually. As previously mentioned, you can view all active sessions in real-time using the CLI, Boundary Desktop or browser-based Boundary admin UI.

Manual termination

You can manually terminate sessions directly from the Boundary admin UI with a few clicks or using CLI commands. This capability allows you to immediately terminate sessions if you detect any suspicious or unauthorized activity.

For example, to terminate a session, navigate to "Sessions" for a given Boundary organization and project in the Boundary admin UI, and click the "Cancel" button.

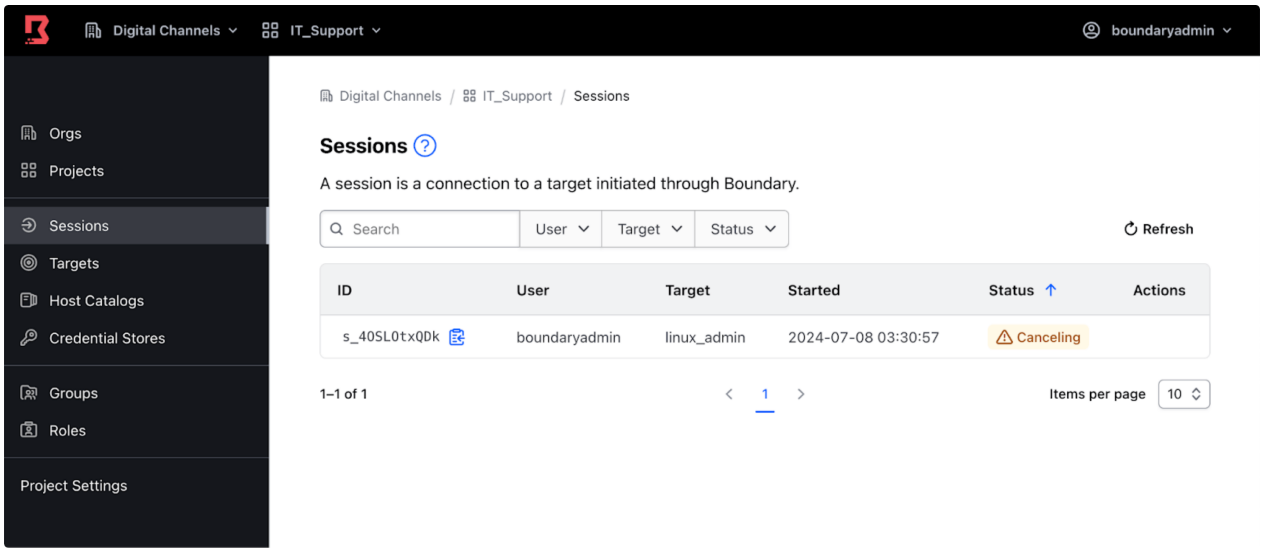


Figure 9: Boundary Sessions Management Cancel

The session status changes to "Canceling," followed by "Terminated." At this point, the user's connection to the target associated with this session is closed."

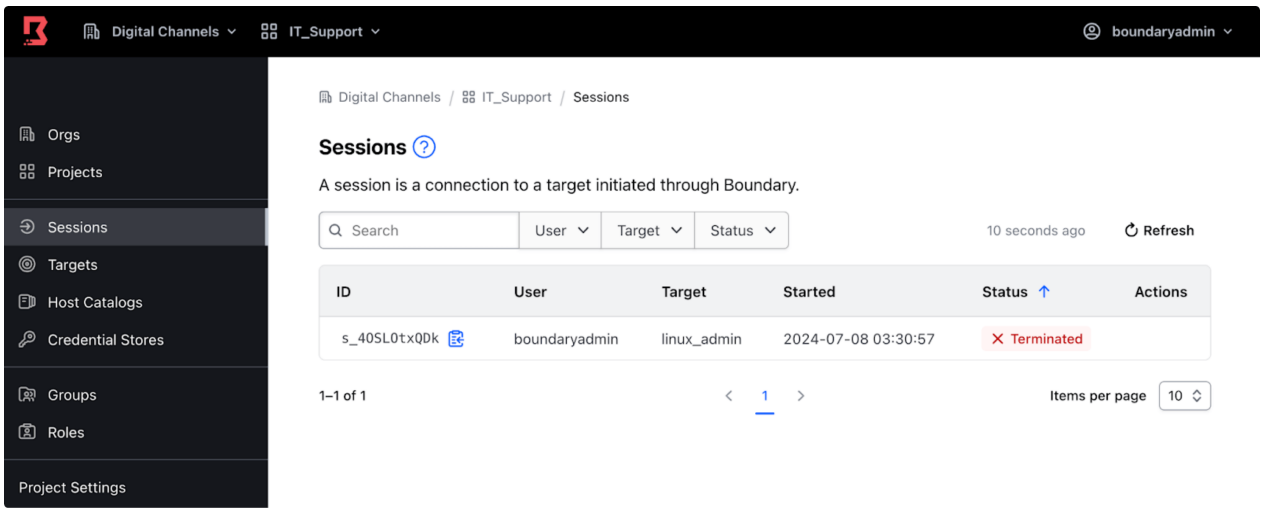


Figure 10: Boundary Sessions Management Terminate

Similarly, to terminate a specific session using CLI, use the `boundary sessions cancel` command with the session ID.

```
boundary sessions cancel -id <session-id>
```

Automatic termination

You can configure maximum session duration for a target after which the session is automatically terminated. If not configured, Boundary sets the default session duration to 8 hours. This ensures sessions do not remain active indefinitely, reducing the risk of unattended sessions being exploited. We highly recommend configuring a maximum session duration for a target to limit the time a potentially compromised session remains active, reducing the window for malicious activities. Setting a maximum duration also ensures that unattended sessions are automatically closed, requiring users to re-authenticate regularly to confirm that access is still valid and credentials have not been compromised.

7.3 Useful resources

- Session recording
- Session management

8 Secure proxy

Secure proxy is a Boundary Worker capability that is responsible for proxying sessions between clients and targets.

8.1 Operating modes

The following sections describe the various operating modes, topology, and network connectivity requirements.

- Proxying target sessions
- Proxying Vault connections
- Proxying multi-hop sessions

Depending on the mode of operation, workers are referred to as a

- Ingress worker
- Intermediary worker
- Egress worker

All workers use the same binary, and their mode of operation is determined by configuration. The access requirements determine which operating modes are required to satisfy the topology. For example, proxying multi-hop sessions will require a minimum of ingress and egress workers.

All operating modes and topologies apply to HCP Boundary and Boundary Enterprise.

8.1.1 HCP Boundary

Self-managed workers allow users to connect to private endpoints securely without routing traffic through HCP Boundary or HashiCorp-managed infrastructure.

For example, organizations with on-premise environments can deploy self-managed workers in their private networks to allow connectivity to those targets with HCP Boundary, often in a multi-hop topology with ingress and egress workers.

Self-managed workers use public key infrastructure (PKI) for authentication. They authenticate to Boundary using a certificate-based method that allows you to deploy workers without using a shared key management service (KMS).

8.1.2 Proxying target sessions

Proxying sessions between clients and targets can be achieved with a single-layered approach, which is the minimal requirement for establishing sessions. This approach is a simplified topology and should only be considered in non-production scenarios such as testing or proof-of-concepts.

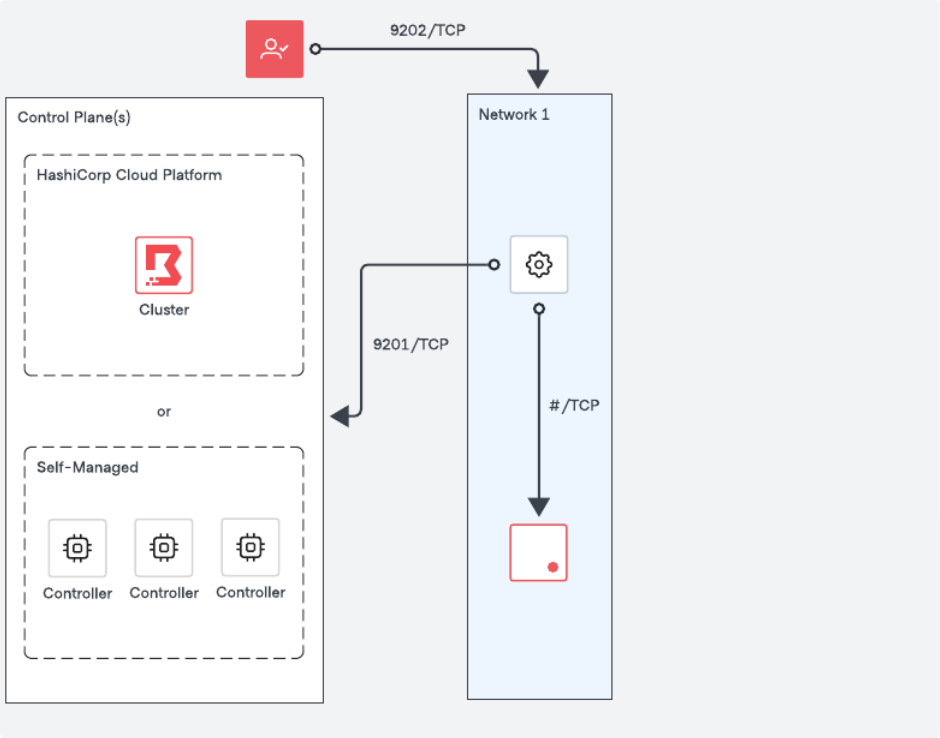


Figure 11: Boundary Proxy Target Sessions

Network requirements

- Outbound connectivity (default port TCP-9201) to an existing trusted Boundary control point, e.g., a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
- Outbound connectivity to the remote service port of the target.
- Inbound connectivity (default port TCP-9202) from clients establishing sessions to the targets

8.1.3 Proxying vault connections

Similar to Proxying target sessions, a worker can proxy connections to private Vault clusters to provide integration between Boundary and Vault.

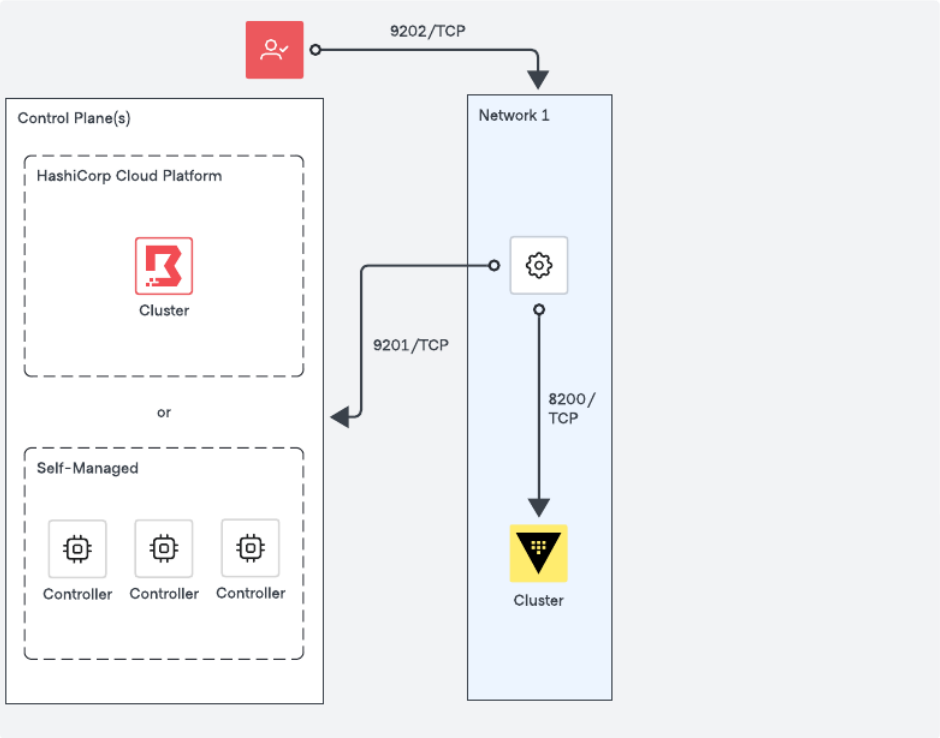


Figure 12: Boundary Proxy Vault Connections

Network requirements

- Outbound connectivity (default port TCP-9201) to an existing trusted Boundary control point, e.g., a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
- Outbound connectivity (default port TCP-8200) to the Vault cluster.

8.2 High availability and sizing

8.2.1 Recommendations for high availability

Each network enclave that Boundary accesses should have at least one worker to provide connectivity. Deploying workers into each network enclave enables organizations to minimize and simplify the firewall/security requirements typically required to provide connectivity to targets.

We recommend at least three workers per network enclave to ensure high availability for production

environments. Worker session assignment is intelligently dictated by the Boundary control plane based on the following:

- Which workers are candidates to proxy a session based on the worker's tags and the target's worker filter, and
- The health and connectivity of candidate workers, you **do not need** a load balancer to manage worker traffic.

The constraints of your access use case and the sensitivity of workloads in each network enclave dictate the redundancy and size you require for your workers.

8.2.2 Sizing guidance

Sizing recommendations have been divided into two cluster sizes, and they represent an initial starting point based on the use case(s).

- **Small** clusters are appropriate for most initial production deployments or non-production environments.
- **Large** clusters are production environments with a large number of Boundary clients.

We recommend that you continue to monitor your cloud providers' network throughput limitations for your machine types as you use Boundary and observe relevant metrics where possible, in addition to other host metrics, so that you can scale Boundary horizontally or vertically as needed.

Provider	Size	Instance/VM Type
AWS	Small	m5.large, m5.xlarge
	Large	m5n.2xlarge, m5n.4xlarge
Azure	Small	Standard_D2s_v3, Standard_D4s_v3
	Large	Standard_D8s_v3, Standard_D16s_v3
GCP	Small	n2-standard-2, n2-standard-4
	Large	n2-standard-8, n2-standard-16

Additional documentation

- AWS: EC2 Network Performance and Monitoring EC2 Network Performance
- Azure: Azure Virtual Machine Throughput and Accelerated Network for Azure VMs
- GCP: Network Bandwidth and About Machine Families

Worker performance

Performance is most affected by the number of concurrent sessions the worker is proxying and the data transfer rates within those sessions. The size of workers is dependent on how you use Boundary, for example, if you use Boundary for SSH connections and HTTP access, your instance selection and performance differ significantly than if you consistently do large data transfers.

Organizations should monitor worker performance using the appropriate telemetry platform. In addition to the typical OS metrics (e.g., CPU, Memory, Disk, Network), Boundary-specific worker metrics should be captured to provide better insight into performance, and enable Boundary administrators to make decisions about scaling.

9 Credential management

Boundary supports credential management using credential stores. There are two types of credential stores: Static and Vault credential stores. During the adoption phase you will focus on using Static credential store. Static credential stores are built into Boundary and only store static credentials like username and password, or keypairs.

You can find how to configure a static credential store following this guide.

9.1 Credential brokering

Credential brokering is a workflow, where Boundary retrieves credentials from a credentials store and presents them to the end user. The end user then enters the credentials into the session when prompted.

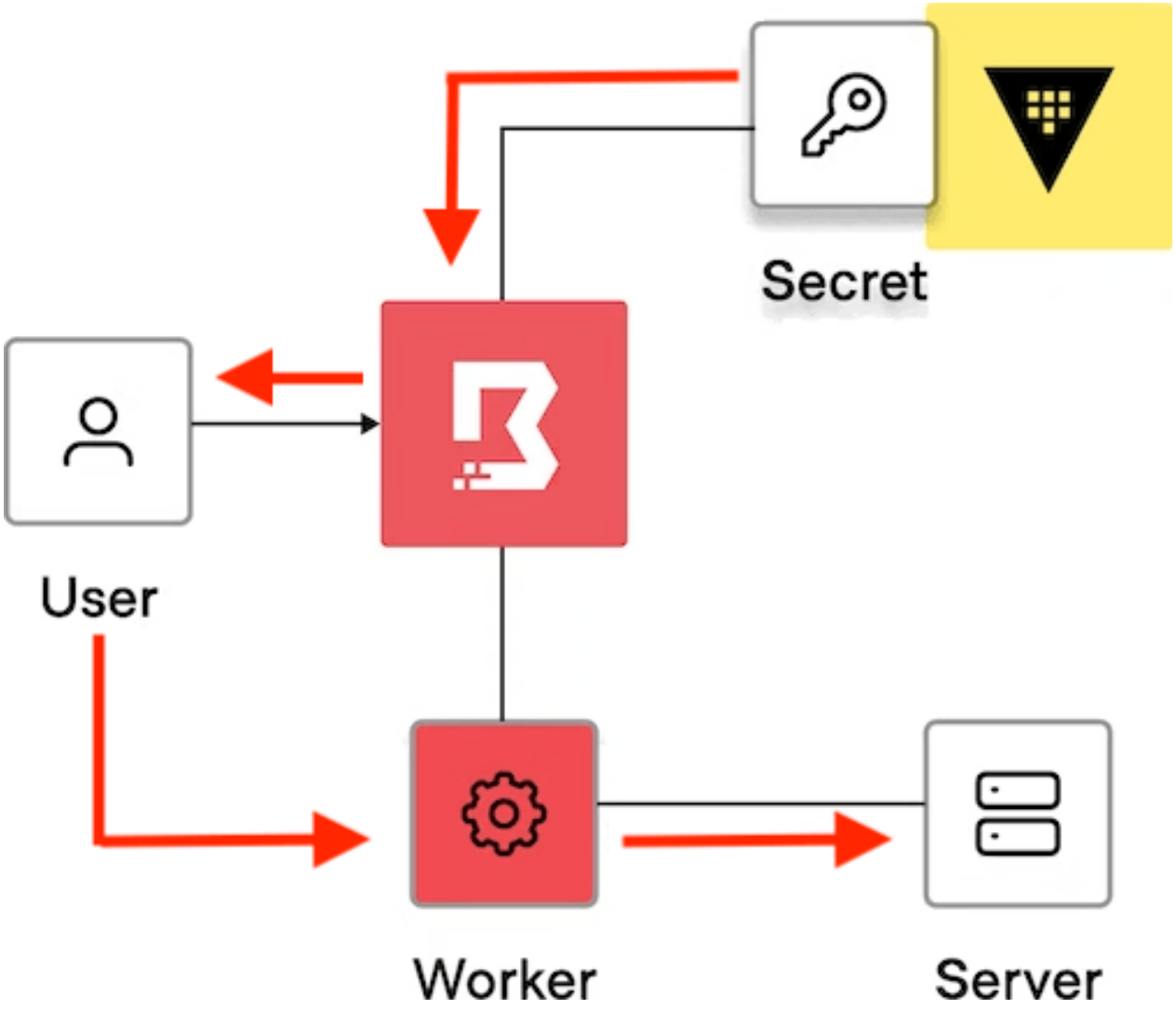


Figure 13: Boundary Credential Brokering

You can attach brokered credentials to either TCP or SSH targets. Brokered credentials can take the form of a token, username and password, SSH private key, certificate, JSON blob, or an unstructured secret stored in Vault, for example.

You can find how to configure targets with credential brokering following this guide.

9.2 Useful resources

The following resources help you to implement, troubleshoot and resolve Credential Brokering related issues.

- Credential Management Docs
- Credential Management Tutorials

10 Credits

List of key team members who contributed to the development of this guide. Hailing from various functional areas and representing diverse departments within HashiCorp, these experienced professionals have played pivotal roles in writing, editing, reviewing, and ensuring the completion of this material. Their collective efforts have been instrumental in shaping the final output.

- Ravi Panchal
- Sai Linn Thu
- Tony Phan
- Andrei Burd
- Nick Wong
- Shriram Rajaraman
- Van Phan
- Jim Lambert
- Jeff Mitchell
- Danny Knights

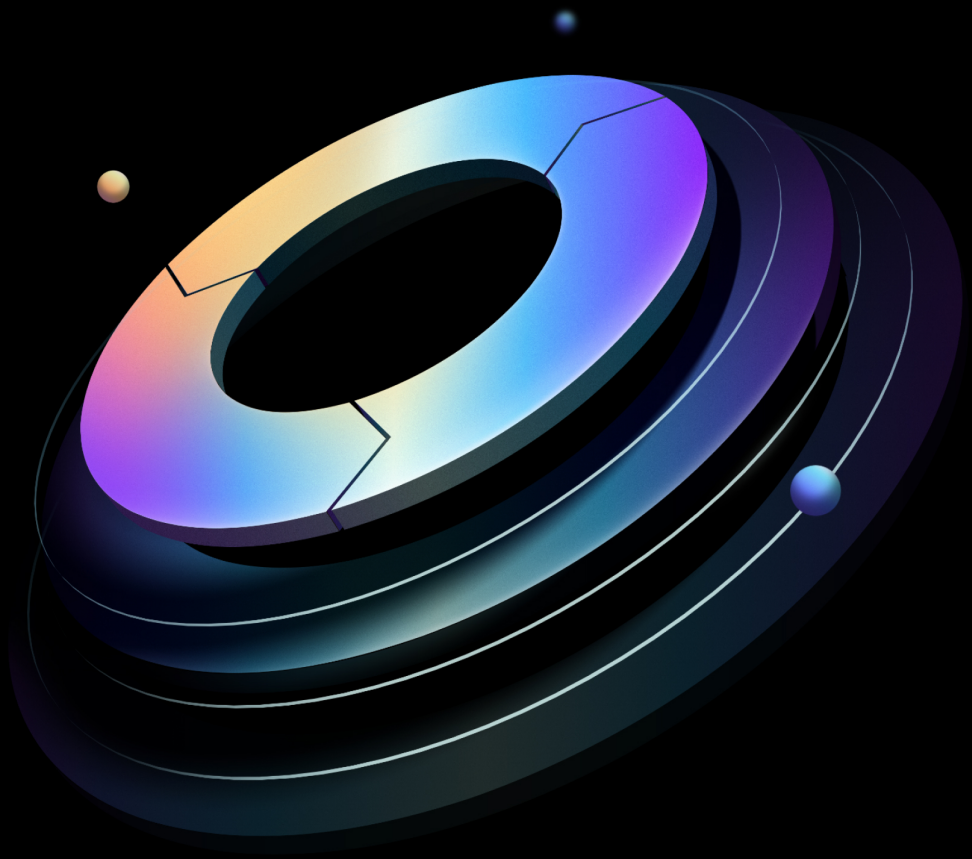
HashiCorp

Boundary: Operating Guide for Standardization

Built from commit 45cb599

HashiCorp | Validated Designs

10 July 2025



HashiCorp

Contents

1 Introduction

This document gives recommendations on how to Standardize on Boundary Enterprise as a shared service for your organization.

Hashicorp Validated Designs provide prescriptive guidance curated from our experience supporting numerous customer journeys with Boundary Enterprise.

1.1 Prerequisites

We recommend that you have completed the following steps before implementing the guidance in this document:

- Review: Boundary Operating Guide for Adoption
- Perform a maturity assessment/architecture review with our Solution Architecture team

1.2 Checklist

After completing the maturity assessment at the end of the adoption phase, you are ready to implement core portions of the standardization phase covered in this document.

While some parts of the standardizing maturity phase are optional and depend on the integrations your organization needs, we recommend adopting the following core capabilities at a minimum:

- Implement auditing to establish a system of record for user access and actions during remote sessions.
- Establish a workflow for periodic key rotation.
- Identify and prioritize use cases for:
    - Integrating with Vault for on-demand, ephemeral credentials to access target networked services.
    - Implementing session recording and storage policies according to your security and compliance requirements.

1.3 Language and definitions

While this guide intentionally uses technology-agnostic language, there are some terms that do not translate seamlessly between providers. This document uses the following terms:

Term	Definition
Scope	A permission boundary modeled as a container for resources.
Global scope	The top-level scope that encompasses all child scopes within the boundary system. It serves as the root of the hierarchical structure to organize resources. The resources such as storage buckets, storage policies, aliases, workers, users, groups, roles and auth methods can be configured at global scope level.
Organ	The intermediate scope level, also referred to as organizations, are a child scope of global. Identity and access management related resources such as users, groups, roles and auth methods can be configured at the organization and global scope levels.
Project	The lowest scope level and a child scope of organization. It allows logical grouping of resources within an organization, such as targets, host catalogs, credentials stores and sessions.
Host catalog	A collection of hosts that Boundary can connect to, organized into host sets.
Host set	A subset of hosts within a host catalog which are considered equivalent for the purposes of access control.
Host	A resource with a network address reachable from Boundary, such as a server or database.
Target	A resource that ties network address information (via a direct address or by referencing host sets), credential libraries for injection or brokering (if desired) and port information to represent a networked service available for connection through Boundary. Targets also contain parameters, such as lifetime and connection count, to configure on the sessions created via authorization against the target.. A target can also be configured with ingress/egress worker filters that determine which workers are used to access targets.
Worke	A secure network proxy, enabling users to access private targets by establishing a direct network tunnel between the Boundary client on the user's machine and the target systems.

Term	Definition
User	A resource that represents an individual person or entity for the purposes of access control. A user can be associated with zero or more accounts. A user authenticates to Boundary through an associated account and must be associated with at least one account before they can access Boundary.
Group	A resource that represents a collection of principals, allowing them to be treated equally for access control purposes. As a principal itself, a group can be assigned to roles, and any role assigned to a group is indirectly assigned to all users within the group. A group can be defined at the global, organization, or project scope levels.
Managed group	A resource that represents a collection of accounts. The collection is formed by evaluating account information (e.g. LDAP groups, OIDC claims) defined by the auth method's identity provider against the managed group's configuration. An account can be associated with zero or more managed groups within the same auth method. It can be used as a principal in roles.
Role	A role is a collection of permissions granted to any principal assigned to it. Users, groups, and managed groups can be configured as principals in a role.
Authentication method	The mechanism by which users authenticate to Boundary. Can also be integrated with external identity providers like LDAP, Active Directory, or OIDC providers.
Account	A representation of a user's identity within a specific authentication method. Accounts can be created manually or automatically generated and are associated with a user in the same scope as the account's auth method..
Credential	Authentication details such as passwords, tokens, or keys used to access resources.
Credential store	Secure storage for managing and accessing credentials. This may be built-in to Boundary or contain information for accessing an external store like HashiCorp Vault.
Credential library	A collection of credentials of the same type from a single credential store that can be brokered or injected into the network session when users are accessing the networked services via sessions.
Session	A session is a set of related connections between a user and a host. A session may include a set of credentials which define the permissions granted to the user on the host for the duration of the session. Limits can be placed on the session, such as a maximum lifetime and/or a maximum connection count.

Term	Definition
Session recordings	Recordings of user sessions for auditing and monitoring purposes.
Storage bucket	A container for storing session recordings and other data within Boundary.
Storage policy	Rules and configurations governing the management and retention of data within storage buckets.
Availability zone (AZ)	A distinct data center within a region that provides redundant and isolated infrastructure to ensure high availability and failover protection. Each region consists of multiple AZs to offer resilience against system failures.
Region	A geographically distinct area that hosts multiple data centers, providing redundancy and fault tolerance for cloud services. Each region consists of multiple, isolated locations known as Availability Zones.
Instance	A physical or virtual server or hardware unit used for computing purposes.
Load balancer	A hardware or software device used to distribute incoming network traffic across multiple servers.

1.4 Use cases covered

This document covers the “Standardizing” phase of operating Boundary on the maturity scale and includes the following:

Use Case	Summary
Credential management	A set of credentials is required when a user tries to access a remote machine. There are two types of credentials - static and dynamic. Credential management refers to the management of these static and dynamic credentials using the concepts of credential brokering or credential injection. Dynamic secrets/ credentials are generated on demand from HashiCorp Vault and are unique, instead of a static credential that is defined ahead of time and shared. Credential injection provides a passwordless experience for the user by fetching the credentials from a credential store and then passed on to a worker for authentication to a remote machine. Currently, only SSH credential injection is supported.
Audit logs	Audit events are considered an important principle of securing access to sensitive resources. Events are emitted for all requests and responses made to a Boundary Controller, every authentication attempt, and all upstream requests made from Workers to a Controller. This section describes the event types, support for sanitizing sensitive information and minimal configuration.

Use Case	Summary
Session recording	Session Recording is used for compliance and threat management and can be enabled for supported targets. Boundary will automatically record the users session to allow authorized users to playback the recording. This section describes the architecture considerations required to support Boundary's Session Recording capability.
Data encryption & key rotation	Improve security posture by regularly rotating data encryption keys
Just-in-time approvals	Just-in-time (JIT) approval workflows ensure that access is granted only when needed and for a limited duration. This reduces the risk of unauthorized access and potential security breaches.
Accessing private resources	Accessing resources within network segmentation or in complex network architectures with limited public accessibility
Automated target discovery	Methods for building out target host catalogs: Manual; Terraform, Dynamic.
Worker aware targets	Using worker tags and filters to direct which Boundary workers to use for proxied sessions.

2 Automating with Terraform

The first milestones of standardizing your Boundary installation are automation and repeatability. None of your daily tasks should stay forgotten and unaudited because of manual processes.

HashiCorp provides you with the official Boundary Terraform provider for you to be able to configure all the aspects of your Boundary using infrastructure-as-code (IaC) approach.

Most of Boundary Tutorials include Terraform code examples to demonstrate you how typical integrations are made.

Please remember that all the provided Terraform code is not intended to be used as is in your production environment and should only be used for reference.

For details on how to use Terraform to set up Boundary resources and the best practices to be followed, refer to Terraform patterns for Boundary.

3 Credential management

When users connect to a remote machine, they typically need a set of credentials for authentication. After they connect to the machine, they may also require another set of credentials to access services or other machines within the network.

In Boundary, there are three main concepts concerning credential management:

- Credential
- Credential Store
- Credential Library

3.1 Credential

A credential is a data structure containing one or more secrets that binds an identity to a set of permissions or capabilities.

A credential:

- Can be of static or dynamic type
- May be a Boundary resource
- Belongs to one and only one credential store
- Can be associated with zero or more targets directly if it is a resource
- Can be associated with zero or more libraries directly if it is a resource
- Is deleted when the credential store or credential library it belongs to is deleted

3.2 Credential store

A credential store is a resource that can retrieve, store, and potentially generate credentials of differing types and access levels. It belongs to a project and supports mechanisms that ensure credentials abide by the principle of least privilege. A credential store can also contain credential libraries.

A credential store:

- Is a Boundary resource
- Can belong to one and only one scope
- Owns zero or more credentials
- Owns zero or more credential libraries
- Is deleted when the scope it belongs to is deleted

In HashiCorp Boundary, credential stores are used to store and manage the credentials required to access various targets. Boundary supports different types of credential stores to accommodate various use cases and credential types.

The primary types of credential stores in Boundary are:

- Static credential stores
- Dynamic credential stores/ Vault credential store

Static credential stores are built into Boundary and only store static credentials (i.e.username and password, ssh-private key and json)

> **Note**
>
> Credentials stores require an Organization and Project scope to be in place to which they will be associated.

Static credential stores support the following credential types:

- Username password
- SSH private key
- SSH certificate
- JSON

Vault credential stores utilize a HashiCorp Vault instance, which provides additional capabilities such as generating ephemeral credentials.

Warning

While using Terraform to automate your credential stores and static credentials, please be aware that even though Boundary is storing static credentials in an encrypted manner, they are not encrypted by default in your Terraform code and state.

3.3 Credential library

A credential library provides credentials for sessions. All credentials returned by a library must be equivalent from an access control perspective, i.e. any user leveraging these credentials from the same library must have the same access to a target. A credential library manages the lifecycle of the credentials it returns. For dynamic secrets, this includes creation, renewal, and revocation. Rotating credentials include check-out, check-in, and rotation of secrets. The system retrieves credentials from a library for a session and notifies the library when the session has been terminated. A credential library belongs to a single credential store.

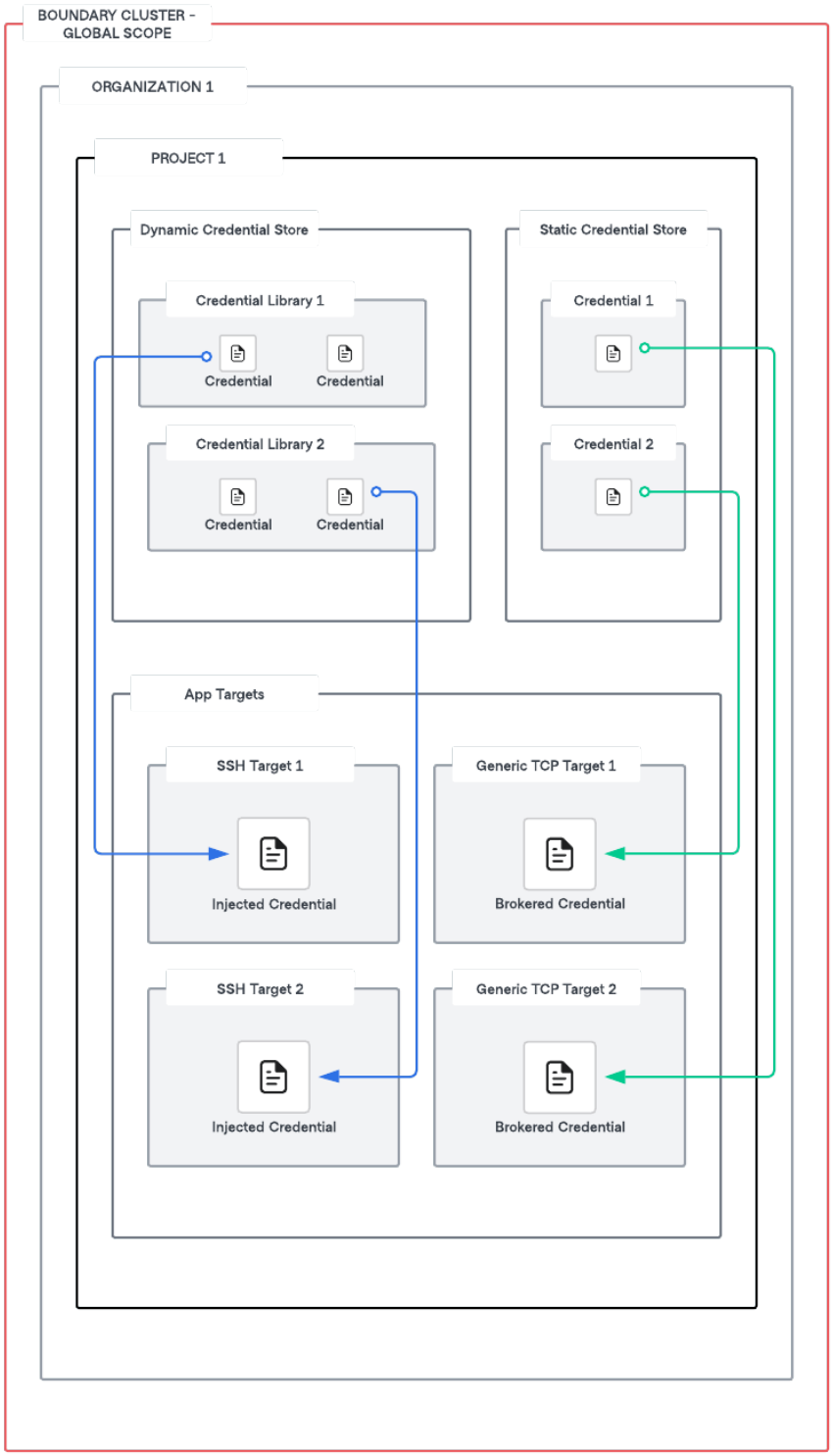
Vault credential libraries are the Boundary resource that maps to Vault secrets engines. A single credential store may have multiple types of credential libraries. For example, the Vault credential store might include separate credential libraries corresponding to each Vault secret engine backend.

A credential library:

- Is a Boundary resource
- Belongs to one and only one credential store
- Can be associated with zero or more targets
- Can contain zero or more credentials
- Is deleted when the credential store it belongs to is deleted

3.4 Architecture

App target types can be associated with static or dynamic credential stores. SSH target types can have either injected credentials or brokered credentials whereas generic TCP target types can only have brokered credentials. Both dynamic and static credentials can be brokered or injected. More information about injected credentials and brokered credentials will be discussed in the following section..


BOUNDARY CLUSTER - GLOBAL SCOPE
ORGANIZATION 1
PROJECT 1
Dynamic Credential Store
Credential Library 1
Credential
Credential
Credential Library 2
Credential
Credential
Static Credential Store
Credential 1
Credential 2
App Targets
SSH Target 1
Injected Credential
Generic TCP Target 1
Brokered Credential
SSH Target 2
Injected Credential
Generic TCP Target 2
Brokered Credential

Organizations may have valid reasons for using multiple credential stores within Boundary. You can only create credential stores at the project scope, and each project can contain multiple credential stores. Projects within an org scope may have different requirements from their credential stores. For example, within an org scope, you may have two projects: Database and Compute. These two projects may need to be isolated and can have a dedicated credential store per project.

4 Dynamic credentials

One of the key features of Boundary is its ability to broker or inject dynamic credentials through integration with an external credential management system - HashiCorp Vault. This capability is facilitated by dynamic credential stores, which provide on-demand, ephemeral credentials to users. Dynamic credential stores in Boundary enable the secure generation and management of temporary, time-bound credentials that are used to access various targets like databases, SSH servers, and other services. Unlike static credentials that are long-lived and manually managed, dynamic credentials are created just in time and automatically expire after a specified period, reducing the risk of credential leakage and minimizing the window of opportunity for unauthorized access.

The integration between Boundary and Vault improves two main areas of concern for organizations:

- Security posture for remote access
- Workflow efficiency

Integrating Boundary and Vault achieves these goals by enabling end-users to access targets without needing to manually distribute credentials.

Organizations should configure credentials to be dynamic and ephemeral by attaching a specific time to live (TTL) to the credential. This provides the highest level of security by applying a finite amount of time for those credentials to be used.

Timely access to resources creates improvements in workflow efficiency by removing manual approvals for access in favor of highly scoped, preconfigured access requests. Additional end-workflow improvements are added by removing credential management from the end user.

When integrated with Vault, Boundary has to be assigned a periodic, renewable, orphan token from Vault. Each credential store needs a separate Vault token.

The following have no impact on Vault's client count:

- The number of Boundary targets that source credentials from the stores
- The number of users connecting to the targets
- The number of sessions that get created
- The number of credential libraries the credential store contains

4.1 Leveraging dynamic credentials

End users have three workflows that can be operationalised for connecting to a target:

1. **Traditional Authentication** - when an end user connects to a target, Boundary initiates the session, but the end user must know the credentials to authenticate into the session. This workflow is available for testing purposes, but it is not recommended because it places the burden on the users to securely store and manage credentials.
2. **Credential Brokering** - credentials are retrieved from a credentials store and returned back to the end user. The end user then enters the credentials into the session when prompted by the target. This workflow is more secure than the first workflow since credentials are centrally managed through Boundary. For more information, see the credential brokering concepts page.
3. **Credential Injection (Recommended)** - credentials are retrieved from a credential store and injected directly into the session on behalf of the end user. This workflow is the most secure because credentials are not exposed to the end user, reducing the chances of a leaked credential. This workflow is also more streamlined as the user goes through a passwordless experience. For more information, see the credential injection concepts page.

4.2 Vault and Boundary for dynamic credentials.

HashiCorp recommends utilizing Terraform to provision Boundary clusters (HCP or self-managed), create organizations, and projects, and manage all Boundary resources. The Vault cluster and its resources should be provisioned in the same manner using the Vault Terraform provider).

4.2.1 Connecting from Boundary to private Vault

When connecting HCP Boundary to a private Vault cluster it requires connectivity via an HCP worker. When setting up an HCP worker, it's important to create a worker filter for the credential store. A

worker filter will identify workers that should be used as proxies for the new credential store, and ensure these credentials are brokered from the private Vault.

Example worker configuration

```
worker {
   auth_storage_path = "./hcp-worker1"
   tags {
      type = ["worker", "vault"]
   }
}
```

Create a new vault credential store with a worker filter.

Example command for creating a new vault credential store with a worker filter

```
 boundary credential-stores create vault -scope-id $PROJECT_ID -vault-address
    $VAULT_ADDR -vault-token $CRED_STORE_TOKEN -worker-filter='"vault" in "/tags /
    type"'
```

In the scenario of self-managed Boundary Enterprise, worker filters may not be required since workers will be provisioned in private networks accessible from Vault.

4.2.2 Boundary organizations, projects, and Vault namespaces.

Vault namespaces are an Vault Enterprise feature that enables isolated Vaults. It provides separate login paths and supports creating and managing data isolated to their namespace. This functionality enables you to provide Vault as a service to tenants.

Everything in Vault is path-based. Each path corresponds to an operation or secret in Vault, and the Vault API endpoints map to these paths; therefore, writing policies configures the permitted operations to specific secret paths. For example, to grant access to manage tokens in the root namespace, the policy path is `auth/token/\`. Managing tokens for a namespace named “education” would be at the following path, `education/auth/token/\`.

Namespace’s support secure multi-tenancy (SMT) within a single Vault Enterprise instance with tenant isolation and administration delegation so Vault administrators can empower delegates to manage their own tenant environment. When you create a namespace, you establish an isolated environment with separate login paths that function as a “mini-Vault” instance within your Vault

installation. Users can then create and manage their sensitive data within the confines of that namespace. Reference Vault Namespaces for additional information.

Boundary uses the concept of scopes in its domain model. There are three levels of scopes in Boundary where each scope can have multiple instances of its child scopes

- Global (Highest level scope)
    - Organization (Intermediate level scope)
        - Project (Lowest level scope)

While designing the integration between Boundary and Vault, there are some key factors to consider:

REQUIREMENTS	CONSIDERATIONS
Organizational Structure	What is your organizational structure? What is the level of granularity across lines of businesses (LOBs), divisions, teams, services, and apps that need to be reflected in your Boundary and Vault's end-state design from an organizational perspective?
Self Service Requirements	Given your organizational structure, what is the desired level of self-service required? How are Boundary Credential stores to be segregated and mapped to host catalogs? How are Vault policies to be managed? Will teams need to directly manage policies for their own scope of responsibility, i.e. credential stores?
Audit Requirements	What are the requirements around auditing usage of Vault credentials within your organization?

REQUIREMENTS	CONSIDERATIONS
Secrets engine requirements	What types of secret engines will you use in your credential stores (KV, database, SSH, PKI, etc.)? For large organizations, each of these might require different structuring patterns. For example, with the KV secrets engine, each team might have its own dedicated KV mount.

Considering the above factors, a recommended pattern for mapping Vault Organizations and Namespaces to a Boundary global scope is as follows:

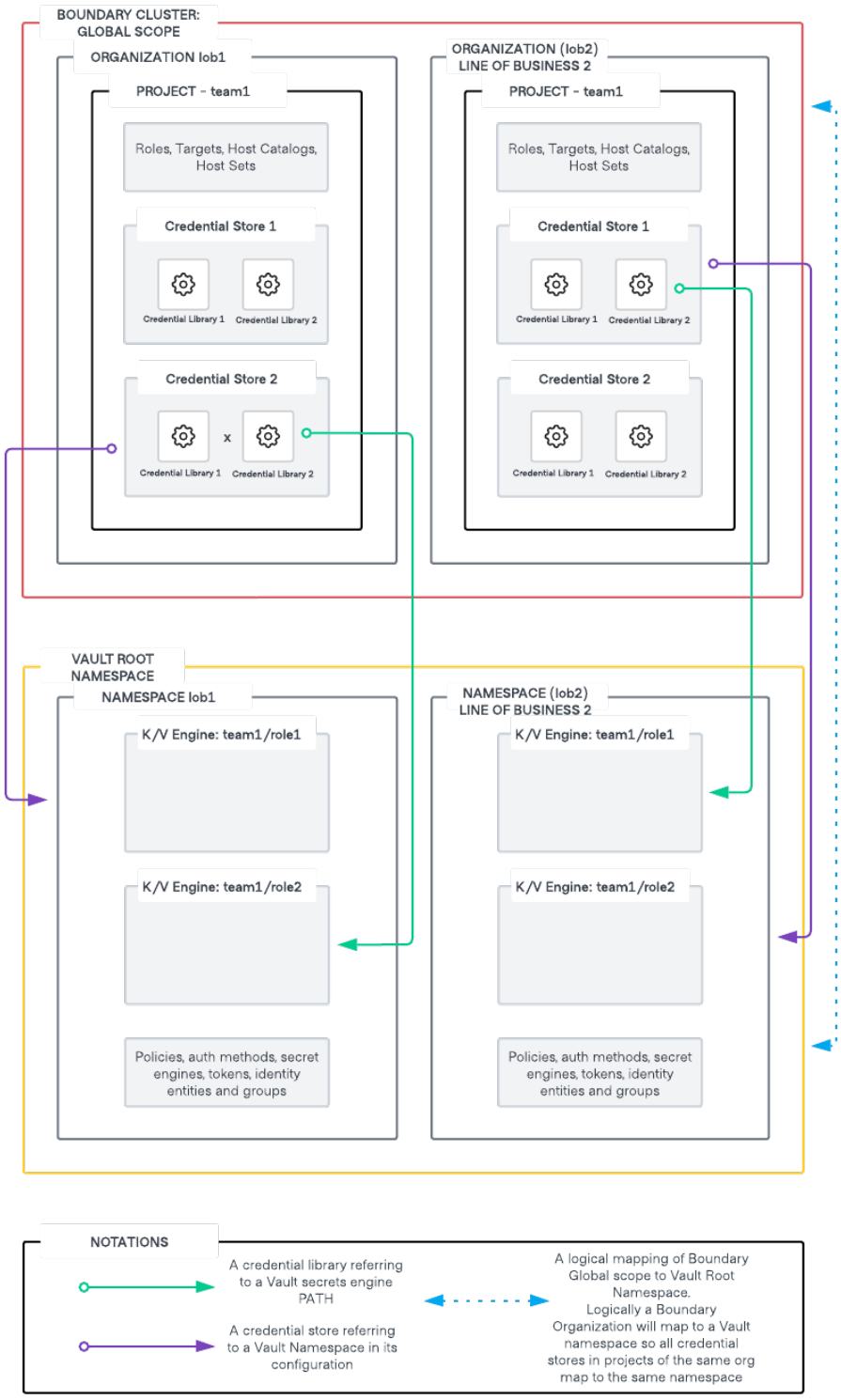


Figure 1: Mapping Boundary Orgs and Projects to Vault Namespaces

- Logically, a Boundary global Scope will map to the Vault root namespace.
- Each line of business [LOB] will have their dedicated organization in Boundary and a dedicated namespace in Vault. Logically an organization in Boundary will map to a namespace in Vault. As you cannot associate Vault's namespace at a Boundary organization scope, hence this mapping is termed as logical.
- All credential stores created in projects that are part of the same organization will refer to the same namespace in Vault for secrets.
  This can further be broken down into hierarchical namespaces within Vault and Boundary credential stores mapping to specific child namespaces within Vault. It is important to keep in mind the principle of least privilege while designing these constructs.
- Credential libraries refer to a particular path in the said Vault namespace. You can have hierarchical paths in secret engines such as Kvv2 (folder structure) to have more granular control over the mapping of specific credential libraries in a credential store to specific paths within the same mount in Vault.

Additional recommendations on Vault namespace design are listed here

4.2.3 Vault credential TTL vs session TTL

Dynamic credentials generated from Vault, such as database credentials, have a time to live (TTL) associated with them. Credentials brokered into a session will forcefully terminate when the TTL for the dynamic credentials expires before the session timeout. If the session timeout is greater than the TTL of the dynamic credentials, the credentials are revoked when the session is terminated.

TTL's are used for static credential sessions to control session termination whereas for dynamic credentials, which are brokered or injected, the credential TTL also dictates the session termination and must be considered while designing.

4.3 Boundary Credential Store Authentication to Vault

Boundary needs to lookup, renew, and revoke tokens and leases to broker credentials properly. This requires these minimum set of permissions for the token created for Boundary authentication to Vault.

- read `auth/token/lookup-self`
- update `auth/token/renew-self`

- update `auth/token/revoke-self`
- update `sys/leases/renew`
- update `sys/leases/revoke`
- update `sys/capabilities-self`

It is important to create policies that give access to the Vault secret engines based on the principle of least privilege. For example, consider a database secrets engine created for managing PostgreSQL credentials. Since Boundary will only be reading credentials from the DB role, read access needs to be provided to only to the particular roles. In this scenario, the following will be the set of permissions required for a database engine mounted at the path `database`

- read `database/creds/<role_name>`

Similarly for connecting to Kubernetes pods using Vault backed credentials, the following will be the set of permissions required for a Kubernetes secrets engine mounted at path `kubernetes`

- update `kubernetes/creds/<role_name>`

Since the Vault token that is generated for Boundary needs to be **periodic, orphan, and renewable**, the following is an example to generate a Vault token based on the above specifications

```
vault token create \
   -no-default-policy=true \
   -policy=<policy1> \
   -policy=<policy2> \
   -orphan=true \
   -period=<period> \
   -renewable=true
```

How to scale this setup? Consider the scenario of multiple secret engines and multiple roles.

- Should we generate an orphan token for each credential store?
- Should we generate an orphan token for each type of database, secret engine, and/or mount and provide the token with permissions for all roles that are part of a mount?
- How should Boundary tokens be scoped?
- Should you create single boundary token for a project or organization?

Considering the multi-tenancy design for Boundary and Vault discussed in the previous section, one of the key considerations to keep in mind is how to manage the scope of the Vault orphan token associated with a Boundary credential store.

It is recommended to create a Vault orphan token in a specific Vault namespace that will be used exclusively by a Boundary project and can have policies attached to read/update the relevant Vault secret engines.

For example, two credential stores in a project that includes a Postgres credential store and an AWS credential store will require two different orphan tokens with their appropriate scopes, respectively.

Breaking this approach to scope orphan tokens to each credential store in a project makes it very difficult to scale as you would be stuck with the overhead of creating and managing orphan tokens for each credential store and managing Vault policies for each one of them which makes it very difficult when you have hundreds of credential stores referencing secret engines in Vault across multiple namespaces.

5 Credential injection

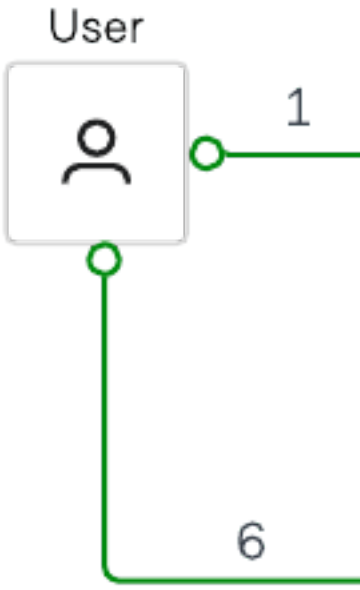
Credential injection is the process by which a credential is fetched from a credential store and then passed on to a worker for authentication to a remote machine. With credential injection, the user never sees the credential required to authenticate to the target. This provides a passwordless experience for the user, as the worker does both session establishment and authentication to the target on behalf of the user. This process differs from credential brokering, where credentials are returned to the user rather than injected into the session on worker nodes.

Consider a scenario where a user wants to access a target using SSH as shown in the diagram below. In this scenario, the user must authenticate to Boundary and must be authorized to access the remote host/ server. Once authorized, a credential is generated for the particular session and is injected directly into the session. This allows the user to establish a remote session with the target.

Credential injection works as follows:

1. A user initiates a session to connect to a remote target by authenticating to Boundary and choosing the target.
2. The Boundary controller requests a dynamic SSH credential to be generated by Vault for this particular session. Note that for private Vault, a self-managed HCP worker will help in the connectivity between the Boundary control plane and private Vault cluster.
3. The Boundary controller provides those credentials back to the Boundary worker

4. The Boundary worker then passes the credential to the target and authtenicates on behalf of the user
5. The Boundary worker authenticates on behalf of the user. In this workflow, the user/client never has access to the credential.



6. Once the credentials have been authenticated, a user session can be initiated.

Credential Injection supports both Static and Dynamic Credential stores. For further details on credential injection and associated security considerations, reference credential injection

6 Audit logs

An important principle of securing access to sensitive resources is creating a system of record for users' access and actions over remote sessions.

For many organizations, demonstrating compliance with their infrastructure's security posture to internal or external auditors is a critical requirement. In this context, records of remote access are often necessary.

Various laws and regulations impose record-keeping requirements. These stipulations outline the activities that need to be recorded and the duration for which the records must be retained. One of the primary reasons an organization maintains records of system access is to comply with these record-keeping requirements.

By default, Boundary does not emit audit events. Organizations should configure Boundary to emit audit events, which are ingested with the appropriate log analytics platform.

Boundary emits audit events for all requests and responses made to a Boundary controller, every authentication attempt, and all upstream requests made from workers to a controller.

6.1 Event types

There are three types of audit events emitted by Boundary: audit, observation, and error.

- **Audit** will be used for any user action that could contain sensitive material.
- **Observation** is any action that occurs within the execution of an event within the application. For example, any function that is called within the process of an event
- **Error** is utilized to handle any event that had an action that did not occur as expected.

6.2 Sensitive information

Boundary supports sanitizing sensitive information from audit events, and Boundary administrators can configure which sensitive information is encrypted or redacted.

Boundary audit events will support three levels of data redaction: none, hmac-sha256, or encrypted. The default is set to none. If hmac-sha256 or encrypted are specified, then a corresponding KMS for audit must be specified in Boundary's configuration.

Organizations should only sanitize if required by laws, regulations, organizational standards, or policies.

Boundary classifies event data into three categories, "public", "sensitive" or "secret".

- **Public** - Boundary events that capture request information contain fields such as "Id," "Method," and "Path."
- **Sensitive** - Boundary events that capture auth information contain fields such as "UserName" and "UserEmail". By default, sensitive data is encrypted unless audit_filter_overrides is configured. Overrides can be configured to "encrypt", "hmac-sha256" or "redact".
- **Secret** - By default, secret data is redacted unless audit_filter_overrides is configured. Overrides can be configured to "encrypt", "hmac-sha256" or "redact".

6.3 Retention

Audit events should be retained for a minimum period to comply with relevant laws, regulations, organizational standards, and/or policies.

6.4 Configuration

The events stanza configures Boundary events-specific parameters.

Default configuration

If no event stanza is specified then the following default is used. This is not recommended for production scenarios.

```
events {
   audit_enabled = false
   observations\_enabled = true
   sysevents_enabled = true
   telemetry_enabled = false
   sink "stderr" {
      name = "default"
      event_types = ["*"]
      format = "cloudevents-json"
      }
}
```

Minimum configuration

```
events {
   audit_enabled = true
   observations_enabled = true
   sysevents_enabled = true
   telemetry_enabled = false
   sink "stderr" {
      name = "all-events"
      description = "All events sent to stderr"
      event_types = ["*"]
      format = "hclog-text"
      }
}
```

Example of Audit events configurations can be found in the Tutorial

6.5 Audit event correlation

Boundary supports correlating audit events with other systems to meet traceability requirements. To track and correlate requests and responses across Boundary and other systems, Boundary uses a Correlation Identifier in the form of an X-Correlation-ID header.

Boundary will check for a Correlation Identifier, and if present, it will be included in the event stream as a public field. If not, a random UUIDv4 will be generated as the Correlation Identifier.

As the user can provide the Correlation Identifier, Boundary cannot guarantee its uniqueness.

6.6 Audit log streaming

Audit log streaming is HCP Boundary specific and supports near real-time streaming of audit events to existing customer-managed accounts of supported providers.

Supported providers

- AWS Cloudwatch
- Datadog

7 Session recording

For reasons similar to Audit Logs, an essential principle of securing access to sensitive resources is creating a record of users' access and actions over remote sessions. In this context, recordings of remote access are often necessary.

Many organizations require the ability to record and playback sessions as an auditing capability for compliance and threat management.

In Boundary, a session represents a set of connections between a user and a host from a target. A session begins when an authorized user requests access to a target and ends when the access is terminated.

When you enable session recording on a target, any user session that connects to the target is automatically recorded. An administrator can later view the recordings to investigate security issues, review system activity, or perform regular assessments of security policies and procedures.

7.1 Workers

Boundary workers perform the recording function. Recording data does not pass through the control plane.

The worker stores the session recording on the local disk during the recording phase and then moves it to the external object store when the session has terminated. Boundary stores recordings in the BSR (Boundary Session Recording) format. During session establishment, the control plane passes any credentials needed to access the external object store to the worker performing the recording. See storage for more information on considerations and provider options.

For targets configured with multi-hop workers, the worker configured to access the external object store records the session. If no worker can access the storage backend, the session is terminated, and an error is returned.

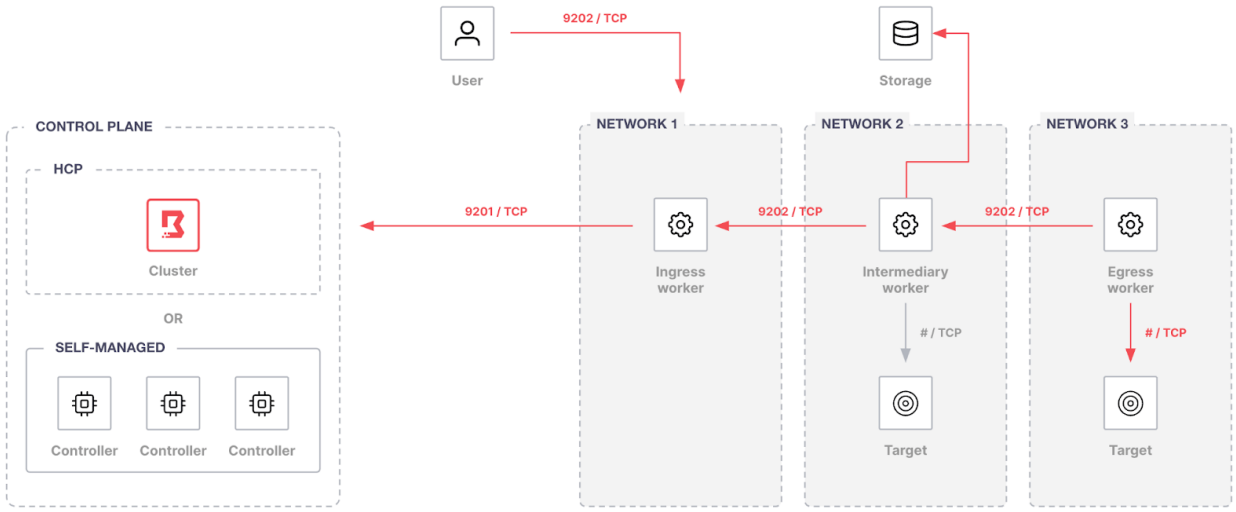


Figure 2: Session Recording

HCP Boundary

Self-managed workers are required to enable session recording with HCP Boundary.

Recorded sessions

Allowed users can view all recorded sessions through a Boundary's administrative web portal.

Please see the tutorial showcasing how to enable session recording with Terraform, AWS and Vault

7.2 Storage Considerations

Before enabling the session recording, one or more storage buckets in Boundary must be created and associated with the external object storage. Organizations must check the considerations for both local storage and external object storage to determine which will meet their requirements.

7.2.1 Local storage

- The number of concurrent sessions that will be recorded on that worker.

- Available disk space is defined by `recording_storage_minimum_available_capacity`
- If organizations don't configure the minimum available storage capacity, Boundary uses the default value of 500MiB.

Refer to the following example configuration to configure workers for session recording storage:

```
worker {
   auth_storage_path = "boundarydemo-worker-1"
   initial_upstreams = ["10.0.0.1"]
   recording_storage_path = "localstoragedirectory"
   recording_storage_minimum_available_capacity = "500MB"
}
```

If a worker is in an unhealthy local storage state, Boundary does not allow new session recordings or session recording playback until the worker is in an available local storage state.

Organizations can check the storage state determined by `recording_storage_minimum_available_capacity` :

- Available
- Low storage
- Critically low storage
- Out of storage
- Not configured
- Unknown

7.2.2 External object storage

The retention period is how long a BSR will be retained in the external storage and from the actual size of the recording data perspective, at a minimum, a session recording for a session with one connection requires 8KB of space for BSR keys, checksums, and metadata.

Estimating how much storage is required to allocate to workers and the external storage provider for recordings depends on user activity. You need to consider the number of concurrent sessions that will be recorded on that worker.

7.3 Storage providers

Organizations must associate the Boundary storage bucket with an Amazon S3 or MinIO storage.

7.3.1 Amazon S3

- Contains the bucket name, region, and optional prefix, as well as any credentials needed to access the bucket.
- Can use static or dynamic credentials. Organizations can configure static credentials using an access key and secret key or dynamic credentials using the AWS AssumeRole API. Here is an example IAM Role policy that can be referenced.

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Action": [
            "s3:PutObject",
            "s3:GetObject",
            "s3:GetObjectAttributes",
            "s3:DeleteObject",
            "s3:ListBucket"
         ],
         "Effect": "Allow",
         "Resource": "arn:aws:s3:::session_recording_storage\*",
         "Resource": "arn:aws:s3:::session_recording_storage/foo/bar/zoo/*"
         },
         {
            "Action": [
               "iam:DeleteAccessKey",
               "iam:GetUser",
               "iam:CreateAccessKey"
            ],
            "Effect": "Allow",
            "Resource": "arn:aws:iam::123456789012:user/JohnDoe"
         }
      ]
}
```

7.3.2 MinIO

- Organizations must provide service account access keys when configuring a Boundary storage bucket.
- It must be configured with R/W access. If you use a restricted IAM user policy, the following policy actions must be allowed at a minimum.

```
{
  "Version": "2012-10-17",
  "Statement": [
     {
        "Action": [
          "s3:PutObject",
          "s3:GetObject",
          "s3:GetObjectAttributes",
          "s3:DeleteObject"
        ],
        "Effect": "Allow",
        "Resource": "arn:aws:s3:::test-session-recording-bucket/*"
     },
     {
        "Action": "s3:ListBucket",
        "Effect": "Allow",
        "Resource": "arn:aws:s3:::test-session-recording-bucket"
     }
  ]
}
```

7.3.3 Storage buckets

A resource known as a storage bucket is used to store the session recordings. The storage bucket represents a location in an external object storage. A storage bucket's name is optional, but it must be unique if you define one. Storage buckets can be associated with zero to many targets.

Organizations must configure workers for local storage and a compatible S3 storage endpoint to create a storage bucket in Boundary. Follow the Create a storage bucket procedure for detailed configuration steps.

For Amazon S3, the required fields for creating a storage bucket depend on whether you configured the static or dynamic credentials. Although credentials are stored encrypted in Boundary, by default the AWS plugin attempts to rotate the credentials Organizations provide. The given credentials are

used to create a new credential, and the original credential is revoked. After rotation, only Boundary knows the client secret the plugin uses. Refer to the required field as below

- Common
  - Worker filter
  - Disable credential rotation
- Static
  - Access key ID
  - Secret access key
- Dynamic
  - Role ARN
  - Role external ID
  - Role session name
  - Role tags

7.4 Lifecycle

Boundary provides storage policies to manage the lifecycle of the stored recordings, allowing administrators to set retention and auto-deletion dates. This helps ensure that recordings are available and accessible for the desired retention period, ensuring organizations can meet regulatory requirements like HIPAA, SOC 2, etc. Auto-deletion helps to reduce management and storage costs by automatically deleting recordings at the designated time and date.

7.4.1 Storage policies

Boundary storage policies play a crucial role in enforcing compliance with specific laws or regulations. It is important to note that the system does not support an undo action. Therefore, updating the storage policy of a session recording can have immediate and possibly unexpected results, such as the immediate deletion of session recordings.

7.4.2 Retention

Organizations should configure Boundary's retention period in accordance with any regulatory requirement minimums, e.g. SOC 2 at 7 years. Generally, it is good practice to set retention periods in alignment to your organizational policies, with considerations of storage costs.

7.4.3 Scope

A storage policy exists in either the global scope or an org scope. Storage policies created in the global scope can be associated with any org scope. However, a storage policy created in an org scope can only be associated with that org scope. Any storage policies associated with an org scope are deleted when you delete the org.

Organizations should start with a global-scoped storage policy unless an org scope has specific requirements.

8 Data encryption and key rotation

8.1 Overview

Data encryption and key rotation are essential elements of HashiCorp Boundary's security framework, ensuring the protection and integrity of sensitive information. Boundary employs strong encryption methods to secure data both at rest and in transit. Implementing regular key rotation helps mitigate the risks associated with prolonged key usage, enhancing overall security. Boundary's encryption and key rotation capabilities enable you to comply with industry standards and regulatory requirements, such as GDPR, HIPAA, and PCI-DSS, which mandate stringent data protection measures.

Encryption at rest
Boundary employs strong encryption algorithms to secure data stored within its system. This includes encrypting sensitive information such as access credentials, session recordings, and audit logs. The encryption at rest ensures that even if the underlying storage is compromised, the data remains protected and inaccessible without the proper decryption keys.

Encryption in transit
Boundary ensures that data transmitted between clients, controllers, and workers is encrypted us-

ing TLS (Transport Layer Security). This prevents eavesdropping and man-in-the-middle attacks, ensuring that data remains confidential and unaltered during transmission.

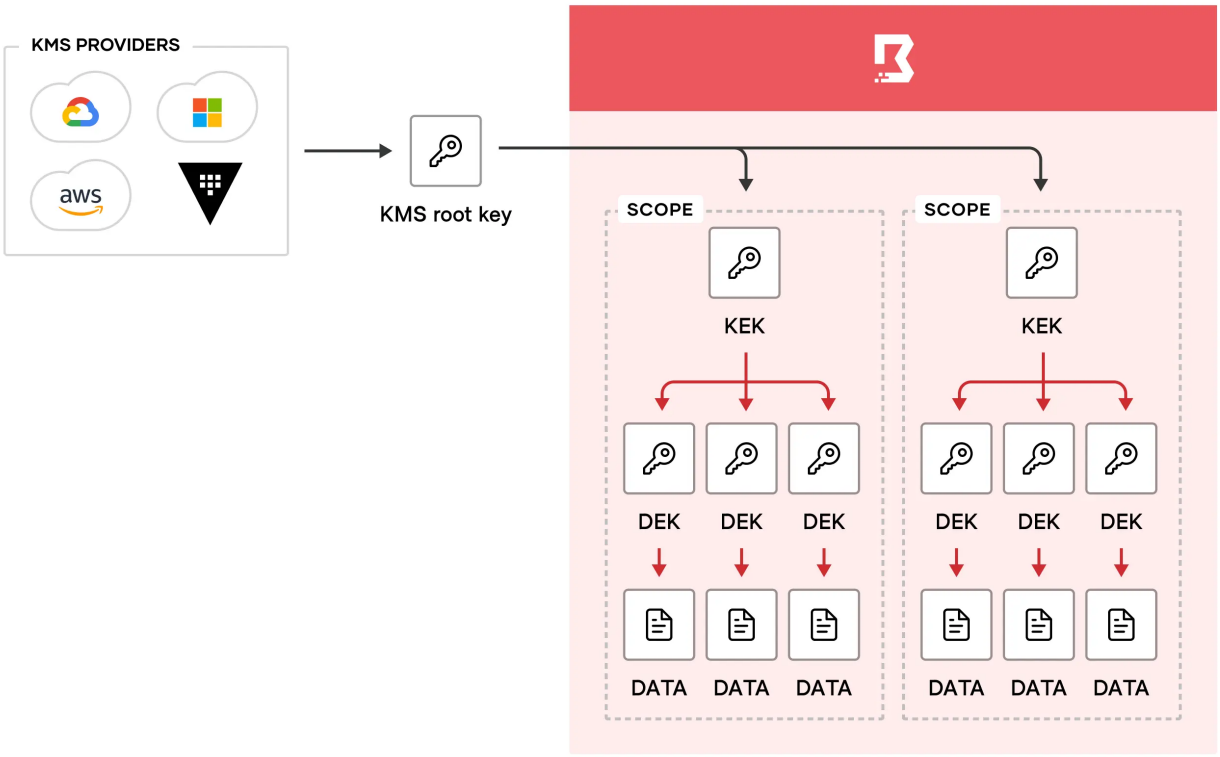
8.2 KMS providers

The KMS provider provides the root of trust for keys used in various encryption operations within Boundary, such as encrypting sensitive data stored in the Boundary database or encrypting the data used to authenticate a KMS worker to a controller.

Boundary supports various KMS providers, including HashiCorp Vault, AWS KMS, Azure Key Vault, GCP Cloud KMS, and more. You can configure KMS providers as part of the Boundary controller configuration. For detailed instructions on configuring KMS providers, please refer to this documentation.

8.3 Key types

The root key provides the root of trust for all scope-specific encrypted data in the Boundary database. This works by having several layers of encryption that all link back to the root key. Whenever you create a new scope, Boundary immediately creates one key-encryption-key (KEK), several data-encryption-keys (DEKs), and a new key version for each of the keys. The key version holds the keying material for the key. Using key versions allows you to rotate keying material, while retaining the same key resource.



Boundary creates following per-scope keys:

- A single key encrypting key (KEK), that is the "root" key for that scope. This key's responsibility is to encrypt the other keys in that scope.
- One or more data encrypting keys (DEKs), each defined for a specific purpose.

Every KEK is encrypted by an external KMS provider. The external KMS is defined as part of the configuration file (potentially with encrypted parameters via a config KMS).

8.3.1 External KMS key types

Boundary uses KMS keys for various purposes, such as protecting secrets, authenticating workers, recovering data, encrypting values in Boundary's configuration, and more. KMS configurations can specify one or more purposes per defined key. These are the currently defined purposes:

- "root": The root KMS key acts as a KEK for the scope-specific KEKs (also referred to as the scope's root key).

- "previous-root": The previous-root KMS key is utilized during the migration to a new root key. Including the previous-root KMS key in your configuration directs the boundary controller to use it for decrypting existing information in the database. This step is essential for rotating and rewrapping the KEKs, thereby completing the migration to the new root key.
- "worker-auth": The worker-auth KMS key is a key shared by the controller and worker in order to authenticate a worker to the controller. If a worker is registered using worker-led or controller-led methods, this is not needed.
- "recovery": The recovery KMS key is used for rescue/recovery operations that can be used by a client to authenticate almost any operation within Boundary.

> Note
>
> It is not required for this kms configuration block to exist in the controller's configuration file. We highly recommend to leave it out except when actually needed, and to use change control capabilities to ensure that the configuration file modification is authorized. After it's no longer needed, the block should be removed.

On the client side, a user can use the `-recovery-config` flag with any operation on the CLI to specify a configuration file containing a suitable kms block. This functionality is also accessible via the Go SDK

> Note
>
> Requests authorized via this mechanism show a user of `u_recovery`. This mechanism cannot be used to authorize a session, as there is no uniquely identifying user information available.

- "config": This key can be used to encrypt values within Boundary's configuration file. The config kms block allows you to encrypt sensitive or secret values, such as cloud API keys for KMS's, in boundary's configuration file. This enables you to safely pass the file to a change control system. Only another operator or system with access to that KMS can decrypt the values. Boundary checks for a config KMS block at startup, and if it exists, uses it to decrypt any encrypted values during startup.
- "bsr": The bsr KMS key is required for session recording. If you do not add a bsr key to your controller configuration, you will receive an error when you attempt to enable session recording. The key is used for encrypting data and checking the integrity of recordings.

8.3.2 DEK key types

DEKs are used to encrypt sensitive/secret application data in the database. Boundary encrypts each scope's DEKs with the corresponding scope's root KEK. This KEK is further encrypted using the KMS key designated for the root purpose. This way, only the root of trust (the root key) can be used to decrypt the data in the database, since you need to decrypt the KEK first, which can then be used to decrypt the DEKs, which in turn can be used to decrypt the application data.

The scoped DEKs and their purposes are detailed below:

- audit: This is used to encrypt secret values in the event log. For more information about the event log, refer to the events config.
- database: This is the general-purpose DEK used to encrypt values within the database. Values that are encrypted are those generally considered to be secret, such as API keys, third-party tokens, certificate private keys, and so on.
- oidc: This is used to encrypt OIDC information in cookies and authentication requests.
- oplog: This is used for encrypting oplog (operation log) values for the given scope.
- tokens: This is used for encrypting tokens generated by auth methods within the given scope.
- sessions: This is used as a base key against which to derive session-specific encryption keys.

8.4 Rotating keys

We recommend rotating keys regularly in alignment with security best practices and industry standards such as PCI DSS, which mandate periodic key rotation.

Rotating keys provides several advantages:

- Limiting the amount of data encrypted by the same key version reduces the risk of successful attacks.
- In the event a key is compromised, regular rotation minimizes the number of messages vulnerable to compromise. If there's any suspicion that a key has been compromised, it should be rotated and revoked immediately to mitigate potential damage.
- Regular rotation ensures that Boundary remains resilient and can handle manual rotation effectively during security incidents like key compromise.

When rotating keys, Boundary generates a new key version for the KEK and all DEKs within the specified scope. These new key versions are then used for future encryption operations, while older key versions remain available for decrypting existing data in the database.

Additionally, you have the option to use the rotate key command to rewrap existing key versions with the new KEK version. This process involves re-encrypting both new and existing DEK versions with the new KEK version, ensuring all DEK versions, both new and old, are encrypted with the latest KEK version. This is particularly useful when you want to discontinue using an old KEK version.

For detailed instructions on key rotation and rewrapping, refer to the key version lifecycle management documentation.

8.5 KMS root key migration

There are several reasons to consider migrating from one root key to another. For instance, you might want to transition from an old AWS KMS configuration that was set up with an account you no longer wish to use. Alternatively, you may prefer to migrate to an entirely new KMS provider, potentially opting for a cloud-agnostic solution like HashiCorp Vault.

8.5.1 Updating the "root" key configuration

The first step is to update the existing root purpose KMS stanza to previous-root. This informs Boundary to use this key provided by the KMS for decrypting the existing data in the database. For instance, if you were using an AEAD KMS provider, the configuration might look like this:

```
kms "aead" {
   purpose   = "root"
   purpose   = "previous-root"
   key_id        = "your-existing-key-id"
   ... // other configuration parameters
}
```

This configuration ensures that Boundary can decrypt the existing database entries using the specified previous-root key during the transition to a new root key.

8.5.2 Adding a new root purpose KMS stanza

Next, add a new root purpose KMS stanza with the new KMS provider configuration and restart the controller. After the restart, Boundary will immediately begin using the new KMS provider for any newly created scopes. However, the existing scopes will still contain KEKs encrypted with the previous root key.

For example, if you are transitioning to a new KMS provider, your configuration might look like this:

```
kms "aead" {
  purpose       = "root"
  key_id        = "new-key-id"
  ... // other configuration parameters
}
```

This configuration ensures that any new scopes created after the controller restarts will use the new KMS provider, while old scopes will continue to function with their KEKs encrypted by the previous root key.

To address this, you need to rotate the keys in all the old scopes, ensuring to specify the rewrap option. This process will re-encrypt all the KEKs with the new root key.

```
$ boundary scopes rotate-keys -scope-id global -rewrap
$ boundary scopes rotate-keys -scope-id <org-id> -rewrap
$ boundary scopes rotate-keys -scope-id <project-id> -rewrap
```

You can now remove the previous-root purpose KMS stanza from the configuration file and restart Boundary again. By doing this, you've successfully migrated from one KMS provider to another.

References:

- Data encryption in Boundary
- KMS configuration
- Boundary KMS (Key Management Service) Root Key Migration

9 Just-in-time approval workflow

9.1 Overview

Enterprises are increasingly looking to improve their enterprise digital workflows by integrating Just-in-Time (JIT) approval workflows with HashiCorp Boundary to enhance their security, compliance, and operational efficiency.

For example, a site reliability engineer might need higher permissions to fix an issue on sensitive systems, or an external contractor could need temporary access to an application. By integrating with approval workflow tools like PagerDuty, ServiceNow, and Slack, enterprises can enable just-in-time requests and approvals for time-limited access. This ensures that privileged roles are only active when a request is made and approved.

Here are key reasons why Enterprises integrate just-in-time approval workflow with Boundary:

9.1.1 1. Enhanced security and compliance

- **Minimized Access Windows:** JIT approval workflows ensure that access is granted only when needed and for a limited duration. This reduces the risk of unauthorized access and potential security breaches.
- **Audit and compliance:** Integrating JIT workflows allows for detailed logging and the context of who accessed what purpose and when, which is crucial for compliance with industry regulations and internal security audit policies.

9.1.2 2. Operational efficiency

- **On-Demand Access:** With JIT workflows, platform teams, partners and contractors can request access as needed, eliminating the need for standing permissions that might not be necessary at all times.
- **Streamlined Approvals:** Integrating with digital workflow platforms like PagerDuty, ServiceNow, and Slack enables seamless and rapid approval processes. Approvals can be managed directly within the tools that teams are already using, reducing friction and speeding up operations.

9.1.3 3. Integration with approval workflow platforms

- **PagerDuty:** Integrating with PagerDuty allows for automated incident response workflows. When an incident occurs, necessary personnel can request and receive access to critical systems immediately, ensuring a quick and efficient resolution.
- **ServiceNow:** Service Now is widely used for IT service management. Integrating JIT workflows with ServiceNow allows for access requests to be part of existing ITSM processes, enhancing visibility and control.
- **Slack:** Slack integration allows for real-time communication and approval. Teams can manage and approve access requests directly within their Slack channels, leveraging the collaboration platform for immediate action.

9.1.4 4. Improved user experience

- **Simplified Access Requests:** Users can request access through their existing approval workflow platforms, reducing the learning curve and making the process more intuitive.
- **Real-Time Notifications:** Integration with these platforms ensures that approvers are notified in real-time, enabling faster responses and reducing wait times for users.

9.2 How just-in-time approval works with Boundary

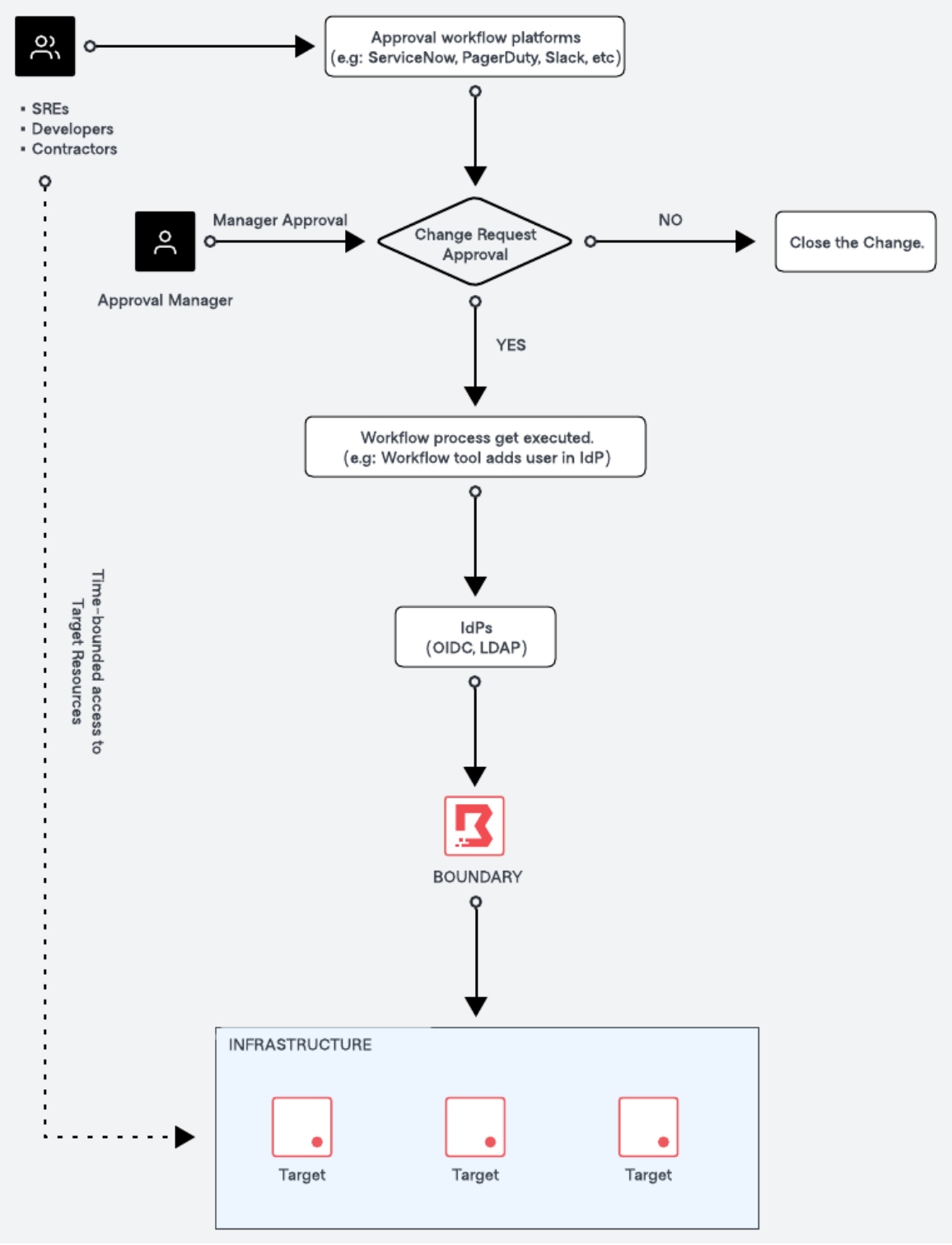


Figure 3: Just-In-Time Flow

The diagram above outlines the high level process of how just-in-time (JIT) approval works with Boundary.

1. **Request access submission**
    - The user submits a request to access target resources through approval workflow platforms such as ServiceNow, PagerDuty, Slack, etc
    - The request will include the context such as the purpose of the request, the required access time window, and the target resource.
2. **Manager approval**
    - The manager reviews the change request and makes the decision to either approve or deny the access request.
3. **Approval workflow execution (upon approval)**
    - Upon approval of the request, the workflow process is executed.
    - E.g: The workflow will update to IdP managed groups (which is our recommended approach) to grant access to the user.
4. **Access granted and time-bounded**
    - The user is added to the appropriate group within HashiCorp Boundary for a specific time period, and access is granted to the target resource in the specified project. The access granted to the user is time-bounded and will expire after a predefined number of hours.
5. **Approval workflow execution (upon expiry)**
    - Once the time period expires, the approval workflow is executed again. The flow updates the HashiCorp Boundary configuration to remove the user from the group.
6. **Access removal**
    - The user's access to the target resource is revoked. And the user is removed from the specific group in the HashiCorp Boundary.
7. **Close the change**
    - The change request is closed in the approval workflow platform, completing the workflow process.

9.3 Useful resources

- HashiConf 2024: Building a break glass solution with HashiCorp Boundary + Vault and ServiceNow integration
- HashiCorp Blog: Just-in-time approval workflow with Boundary and Azure

10 Accessing private resources

10.1 Multi-hop sessions

Organizations with complex network topologies often require inbound traffic to route through multiple network enclaves to reach the target system.

Even in complex networks with strict outbound-only policies, Multi-hop sessions allow you to chain two or more workers across numerous networks to form reverse proxy connections between the user and the target.

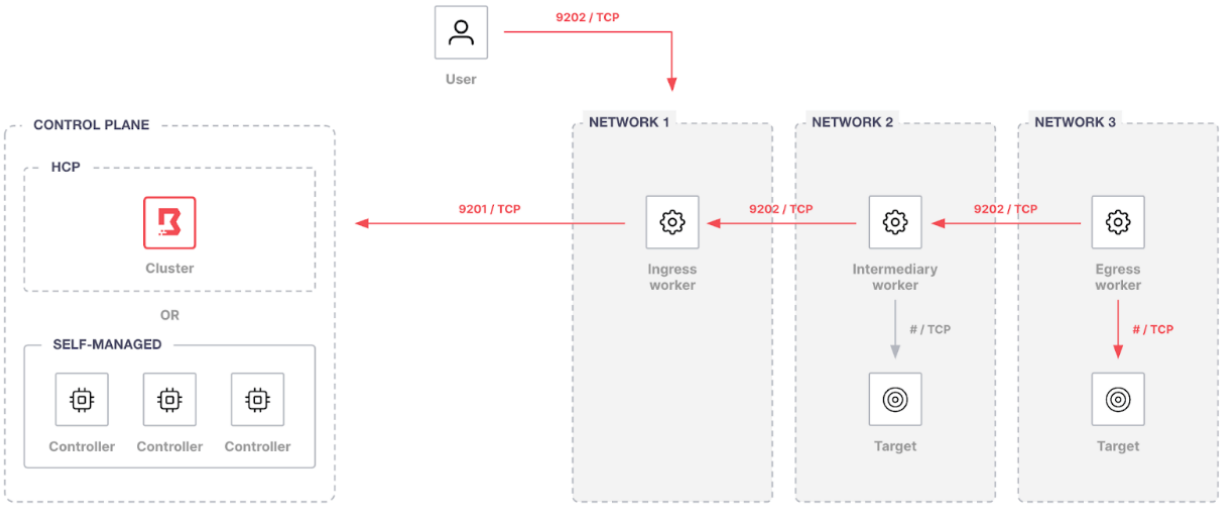


Figure 4: Multi-hop Sessions

When workers operate as part of a multi-hop chain, they have three distinct functions:

10.1.1 Ingress worker

An ingress worker is accessible by Boundary users and typically deployed on edge or public networks.

Network requirements

1. Outbound connectivity (TCP-9201) to an existing trusted Boundary control point, e.g., a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
2. Inbound connectivity (TCP-9202) from users establishing sessions

10.1.2 Intermediary worker

An intermediary worker is optional and is deployed between ingress and egress workers.

Network requirements

1. Outbound connectivity (TCP-9202) to an upstream worker. An upstream worker may be an ingress worker or another intermediary worker. Any upstream or intermediary worker must eventually connect to an ingress worker.
2. Inbound connectivity (TCP-9202) from a downstream worker. A downstream worker may be an egress worker or another downstream worker. Any downstream or intermediate worker must eventually connect to an egress worker.

10.1.3 Egress worker

An egress worker provides connectivity to the target. The worker initiates reverse proxy connections to the intermediary or ingress workers.

Network requirements

1. Outbound connectivity (TCP-9202) to an upstream worker
2. Outbound connectivity (client target port) to the target.

10.1.4 Redundancy

A recommendation is to create redundant or multiple ingress paths to ensure elasticity. Typically, a Boundary architecture *might* include multiple ingress paths to a particular target. By providing

multiple paths to a target, worker ingress is moved away from a single point of failure, as workers are not highly available at this time.

10.2 References

Multi-hop sessions with HCP Boundary tutorial shows an example how targets can be configured to be accessed through the chain of Boundary workers.

11 Automated target discovery

Boundary administrators are responsible for configuring and managing host discovery workflows. They use various methods to ensure hosts and targets are accurately discovered and configured within Boundary.

11.1 Overview of host discovery methods

Boundary supports three primary workflows for target/host discovery:

1. Manual configuration: Administrators manually configure static hosts and targets using the administrator GUI and CLI. This method requires knowledge of the IP address or endpoint used to connect to a host.
2. Configuration-as-code with Terraform: Boundary integrates with Terraform to automate the discovery and configuration of new infrastructure targets. This allows for dynamic configuration without prior knowledge of the target's connection information.
3. Dynamic host catalogs: Boundary automates the ingestion of computing instances and resources from infrastructure providers. Boundary currently supports a dynamic host catalog for AWS and Azure, and we will continue to grow this ecosystem to support additional providers. Hosts are automatically created, updated, and added to host sets, reflecting the connection information maintained by these providers.

11.2 Dynamic host catalogs

We recommend that boundary administrators use dynamic host/target catalogs to automate the discovery and configuration of hundreds of instances at a scale where infrastructure resources are

highly dynamic and ephemeral.

A recommendation is to have a proper tagging of your cloud resources based on organization, business units or product/services team, so that Boundary can discover automatically and manage dynamically.

11.2.1 Dynamic host catalogs workflow with AWS

Boundary administrators leverage dynamic host catalogs to discover and configure AWS resources associated with tags based on `tag:Name=Value`.

For example, instances within AWS could be deployed with tag names and values as follows:

- boundary-1-dev
  - service-type:database
  - environment:dev
- boundary-2-dev
  - service-type:database
  - environment:dev
- boundary-3-production
  - service-type:database
  - environment:production
- boundary-4-production
  - service-type:database
  - environment:production

The host set would then be defined using filters that select the discovered hosts for membership based on the tags defined.

```
boundary host-sets create plugin \
  -name database \
  -host-catalog-id $HOST_CATALOG_ID \
  -attr filters=tag:service-type=database
```

We recommend to begin using dynamic host catalogs for AWS by following the respective setup tutorial guides: Dynamic host catalogs on AWS

11.2.2 Dynamic host catalogs workflow with Azure

Boundary administrators leverage dynamic host catalogs to seamlessly discover and configure Azure resources available through Azure Resource Manager (ARM), adding them as Boundary hosts.

We recommend to begin using dynamic host catalogs for Azure by following the respective setup tutorial guides: Dynamic host catalogs on Azure

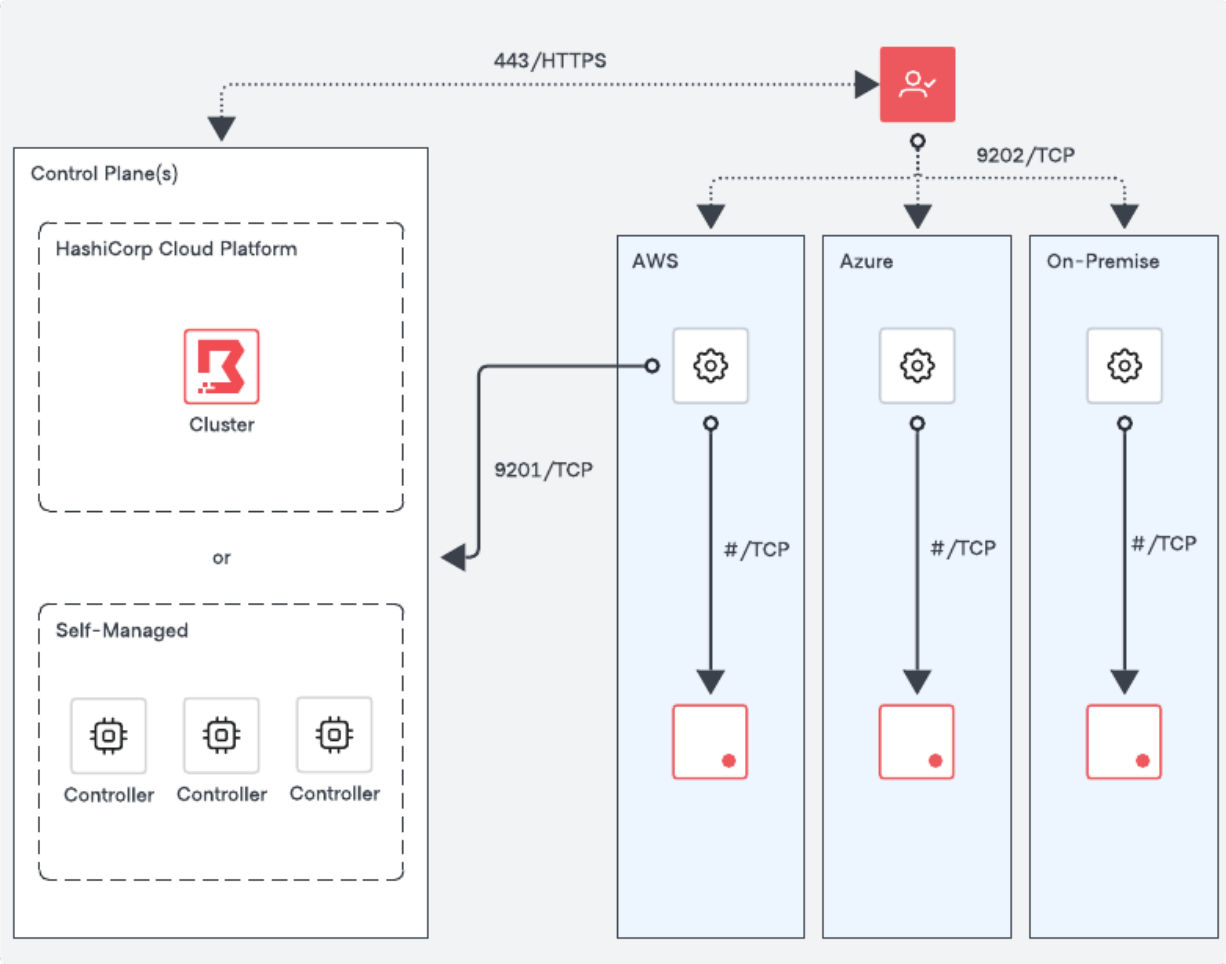
11.3 Useful resources

- Concepts: AWS dynamic host catalogs
- Concepts: Azure dynamic host catalogs
- Guided Tutorial: Dynamic host catalogs on AWS
- Guided Tutorial: Dynamic host catalogs on Azure

12 Worker aware targets

In traditional multi-datacenter and multi-cloud operating models, it's common to deploy a control plane for each environment, complete with controllers and workers, to minimize latency or meet security standards. However, managing multiple controllers can increase complexity, costs and operational overhead.

Boundary's control plane uses worker tags and filters to coordinate which workers can handle a target's session. A typical example is allowing a single set of controllers to live in one environment and placing workers in many other environments where their proxy targets live.



12.1 Multi-region deployments

This section provides an example of a multi-region deployment of HCP Boundary across multi-datacenter and multi-cloud environments. The same concept can be applied to self-managed Boundary Enterprise.

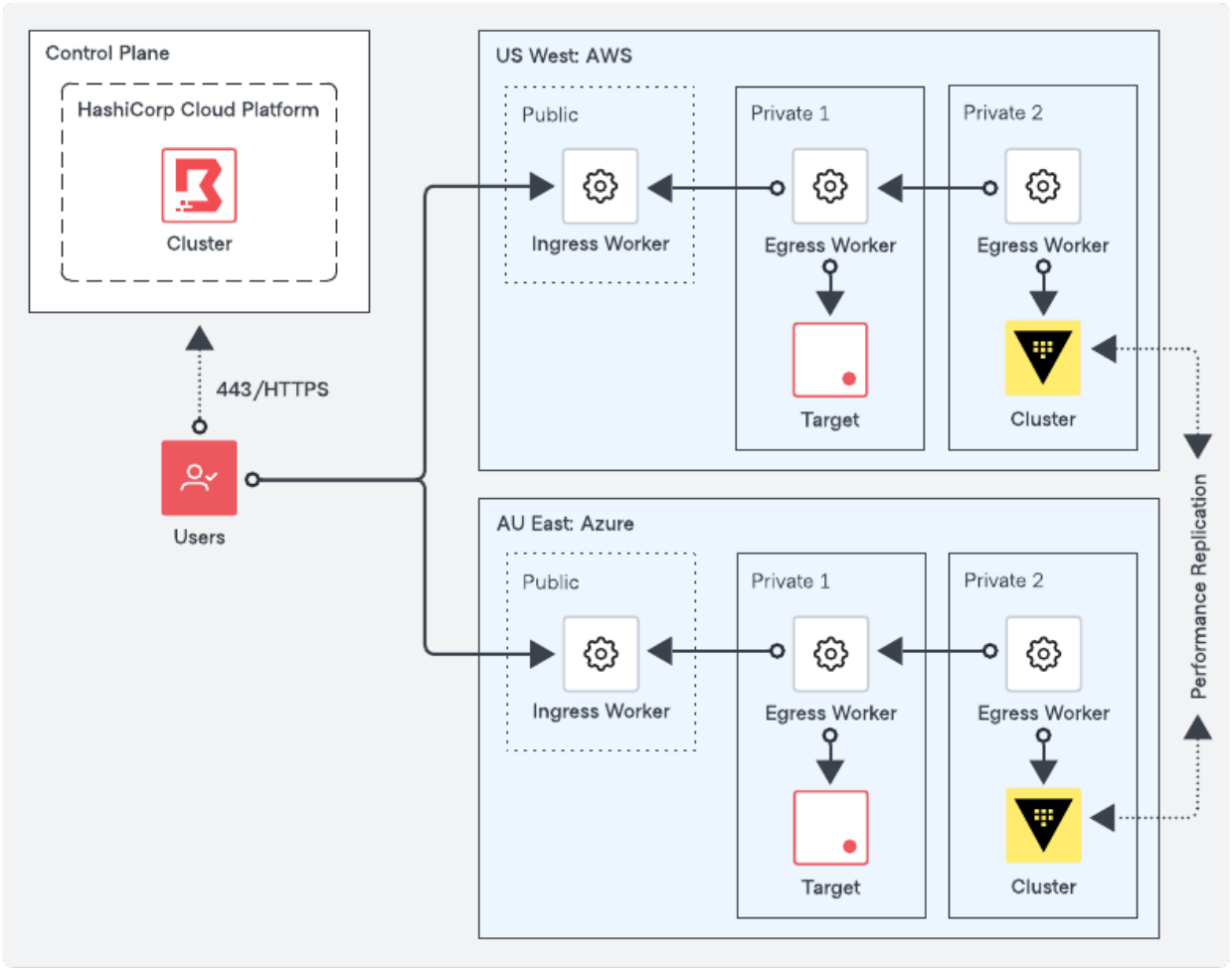


Figure 5: Multi-Region Deployment

- There is a publicly accessible ingress worker at each region's edge for user connectivity. The user session would transit through the appropriate ingress worker based on the target based on the target.
- Multi-hop sessions, where egress workers would proxy from one another to simplify connectivity requirements.
- Vault deployment in each region to comply with availability and regulatory requirements. The Vault cluster resides in a secure private network.
- Session recording data is stored locally within each region.

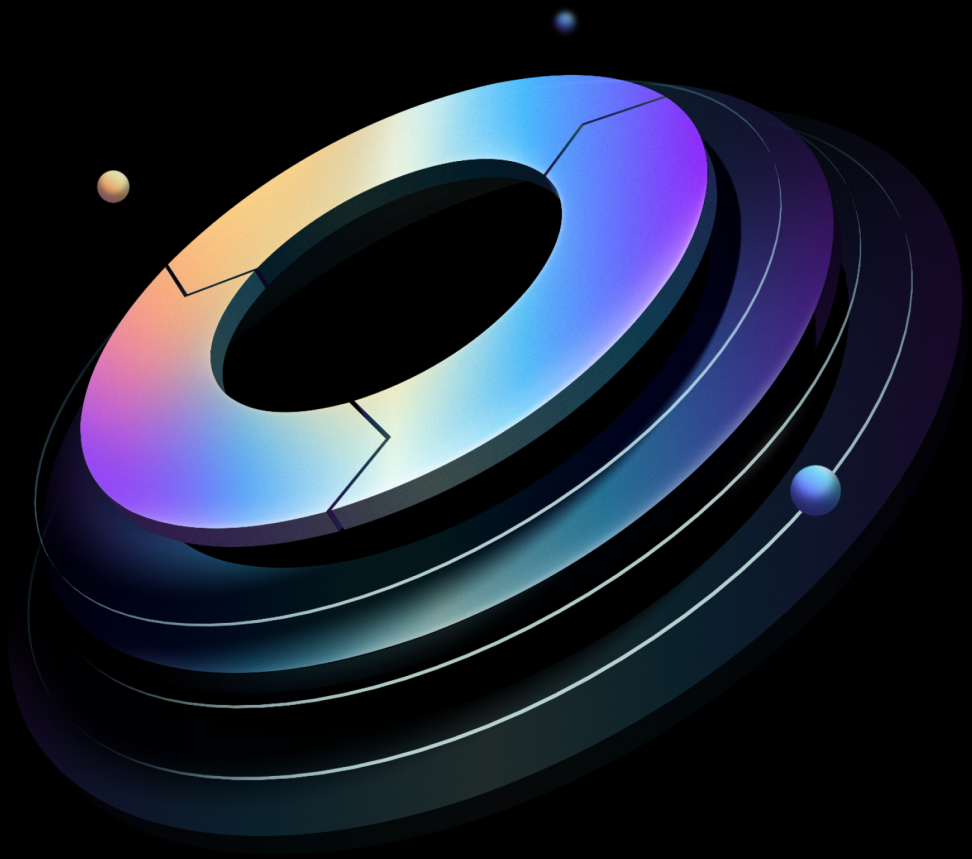
HashiCorp

Boundary: Solution Design Guide

Built from commit 45cb599

HashiCorp | Validated Designs

10 July 2025



HashiCorp

Contents

1 Introduction

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of Boundary Enterprise. These designs are purpose-built for delivering foundational Boundary Enterprise use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with Boundary Enterprise customers.

This Solution Design Guide provides customers with access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, the guide identifies modular design components that customers can adjust to align with organizational and/or regulatory requirements without compromising the overall integrity of the implementation. For customers deploying Boundary Enterprise to the cloud, the guide also includes Terraform modules to automate large portions of the infrastructure provisioning and software installation.

1.2 Audience

This document is primarily for practitioners (including platform, networking, identity, and information security teams) looking to deploy Boundary Enterprise clusters on-premises or in cloud infrastructure. After using the Solution Design Guide to deploy your Boundary Enterprise infrastructure, operators should refer to Boundary: Operating Guide for Adoption. Using this Operating Guide, Boundary Enterprise operators learn how to integrate with identity providers, configure secure user access, centralize credential management, and more.

1.3 Supported versions

This version of the guide relates to Boundary Enterprise 0.17.x+ent.

1.4 Language and definitions

This documentation intentionally uses technology agnostic terminology, however there are some terms which do not translate between the cloud providers. The following are the definitions of terms this document uses.

Term	Definition
Region	A physical location in a distinct geographic area containing one or more availability zones (AZ).
Availability zone (AZ)	An availability zone (AZ) is a single network failure domain that hosts part or all of a Boundary Enterprise cluster. Examples of availability zones include: an individual datacenter
Public subnet	A network accessible by users.
Private subnet	A network used by applications and services, but not directly accessible by users.

1.5 Organizational requirements

We expect a single team, often referred to as the Platform Team, to handle the tasks in this guide. However, a successful Boundary Enterprise deployment requires collaboration across multiple organizational functions. The Platform Team needs to work with other teams, such as networking or security for tasks such as IP allocation or certificate management. If hosting infrastructure on a public cloud, consolidate these responsibilities within a unified cloud team. The platform team should thoroughly review this Solution Design Guide to identify any teams they may rely on or need approval from for key deployment tasks. It is essential to designate a project lead to oversee the deployment process and ensure clear communication and coordination with all relevant teams and functions.

2 Architecture

2.1 Recommended deployment architecture

This section explores the recommended Boundary Enterprise architecture, which provides a highly available, scalable, and secure deployment suitable for production workloads.

The primary components that make up a Boundary Enterprise cluster are:

- Controller nodes
- Worker nodes
- Load balancer
- PostgreSQL database
- KMS (Key management service)

The following diagram shows the recommended architecture for deploying Boundary Enterprise within a single region.

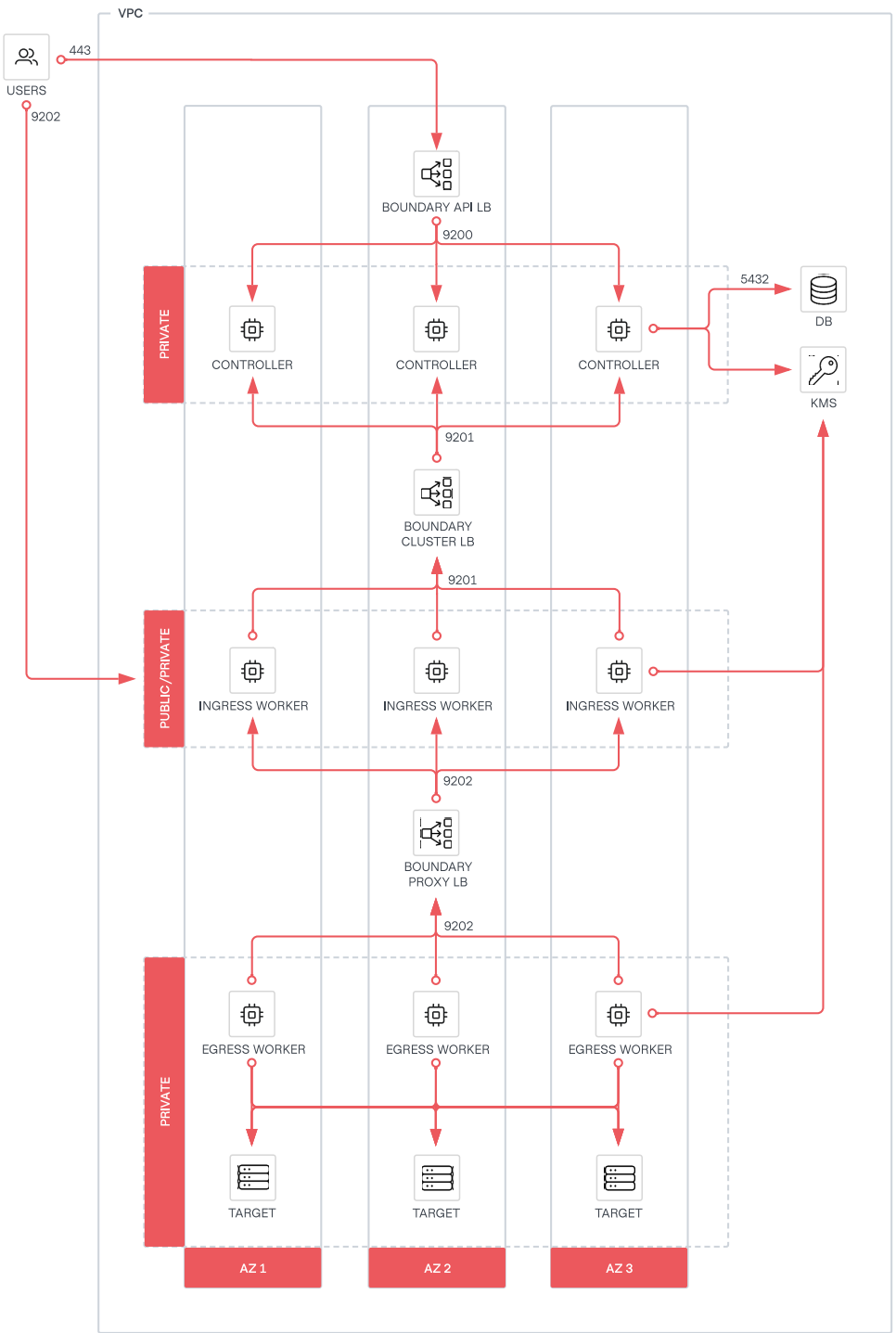


Figure 1: Boundary Single Region Deployment

2.2 Controllers

A minimum deployment consists of three controllers in three separate private subnets distributed across three availability zones to ensure high availability. Configure the controllers to run in a fault-tolerant setup, such as an auto-scaling group for a self-healing environment. Users authenticate with the controllers when using Boundary Enterprise. We recommend exposing the controller API and UI to your users through a layer 4 load balancer. See load balancing section for more information.

2.3 Database

Controllers are stateless, and manage all configuration through an external PostgreSQL database. We recommend configuring the PostgreSQL database for high availability. Please refer to the PostgreSQL high availability, load balancing, and replication documentation. If you use a managed service, refer to your provider's PostgreSQL high availability documentation.

2.4 Workers

A minimum deployment consists of at least three workers across different availability zones within each network boundary to ensure high availability. In environments which restrict inbound connections, deploy both ingress and egress workers to enable multi-hop session proxying, which only requires outbound connectivity. Please refer to the recommended architecture for more details.

2.5 Key management service

Boundary controllers use KMS keys to encrypt data at rest and in transit. Before the Boundary worker can proxy user sessions to targets, it must authenticate and register with the Boundary control plane. We recommend using a KMS-led authorization and authentication flow to auto-register the worker. This method requires the controller and worker to share a KMS key to authenticate the worker with the controller. We recommend using Vault's Transit secret engine for key management. If you use a managed service, refer to your provider's key management documentation for guidance.

2.6 Transport layer security

2.6.1 Client-to-controller TLS

We recommend configuring the TLS (with a public certificate, private key, and certificate authority bundle from a trusted certificate authority) on the Boundary controller nodes. Configure the load balancer to pass through TLS connections to controller nodes. Do not manage the TLS certificate on the load balancer. Terminating TLS connections at the controller nodes offers enhanced security by ensuring that a client request remains encrypted end-to-end. This reduces the attack surface for potential eavesdropping or data tampering.

2.6.2 Client-to-worker TLS

Workers do not require any configuration for their client-facing listeners. Instead, they create the TLS configuration dynamically via Server Name Indication during session authorization, and the session is then mutually authenticated.

2.6.3 Worker-to-upstream TLS

Workers establish TLS connections to upstreams (controllers or other workers) dynamically during registration. For more information, refer to this document.

2.7 Load balancing

The control plane components of Boundary's architecture benefit from using load balancers. Here are the three scenarios:

- Client-to-controller
- Worker-to-controller
- Worker-to-worker (multi-hop sessions)

The load balancers help secure Boundary's components and increase reliability and stability.

For load balancing from clients to workers, for example when clients initiate sessions to a Boundary Enterprise target, the Boundary control plane manages the distribution of sessions amongst available workers. As such, Boundary Enterprise workers do not require any load balancing.

The design of this architecture requires the use of layer 4 or 7 load balancers. It must be able to poll the `/health` API endpoint to detect the node's status and direct traffic accordingly.

3 Detailed design

This section provides more architectural detail on each Boundary Enterprise component. Review this section to identify all technical and personnel requirements before moving to implementation.

3.1 Sizing

Every hosting environment is different, and every customer's Boundary Enterprise usage profile is different. Refer to the tables below for sizing recommendations for controller nodes and worker nodes, as well as small and large use cases, based on expected usage.

Small deployments would be appropriate for most initial production deployments or for development and testing environments. **Large** deployments are production environments with a consistently high workload, such as a large number of sessions. **Controller nodes**

Size	CPU	Memory	Disk capacity	Network throughput
Small	2-4 core	8-16 GB RAM	50+ GB	Minimum 5 GB/s
Large	4-8 core	32-64 GB RAM	200+ GB	Minimum 10 GB/s

Worker nodes

Size	CPU	Memory	Disk capacity	Network throughput
Small	2-4 core	8-16 GB RAM	50+ GB	Minimum 10 GB/s
Large	4-8 core	32-64 GB RAM	200+ GB	Minimum 10 GB/s

Refer to Hardware sizing for Boundary servers for more details. These recommendations should only serve as a starting point for operations staff to observe and adjust to meet the unique needs of each deployment. Match your requirements and maximize the stability of your Boundary Enterprise

controller and worker instances, by performing load tests and continue monitoring resource usage and all reported metrics from Boundary's telemetry.

3.1.1 Hardware considerations

CPU, memory, and storage performance requirements depend on your exact usage profile for example types of requests, average request rate, and peak request rate. Boundary Enterprise controllers and worker nodes have distinct resource needs as they handle different tasks. Refer to the Hardware Considerations for more information.

Enable audit logging in Boundary Enterprise. It is best to write audit logs to a separate disk for optimal performance. We recommend monitoring both the file descriptor usage and the memory consumption for each Boundary Enterprise worker node. These resources can become constrained depending on the number of clients connecting to Boundary Enterprise targets at any given time. If you have enabled session recording on a target, the worker stores the session recordings locally during the recording phase. Refer to Storage Considerations to determine how much storage to allocate for recordings on the worker nodes.

3.2 Networking

Network bandwidth requirements for Boundary Enterprise controllers and workers depend on your specific usage patterns. It is also essential to consider bandwidth requirements for other external systems, such as monitoring and logging collectors. Refer to Network Considerations for more information. Monitor the networking metrics of Boundary Enterprise workers to prevent situations where they are unable to initiate session connections. Review your provider-specific virtual machine networking limitations. You should increase the VM size to achieve higher network throughput.

3.2.1 Network connectivity

Refer to Network Connectivity for the minimum requirements for Boundary Enterprise cluster nodes. You may also need to grant the Boundary Enterprise nodes outbound access to additional services that live elsewhere, either within your internal network or via the Internet. Examples may include:

- Authentication provider backends, such as Okta, Auth0, or Microsoft Entra ID
- Remote log handlers, such as a Splunk or ELK environment

- Metrics collection, such as Prometheus

3.3 Storage

Enable audit logging in Boundary Enterprise. For optimal performance, writing audit logs on a separate disk is advisable. The worker stores session recordings locally during the recording process, if enabled. When estimating worker storage needs, consider the number of concurrent sessions recorded on that worker. Refer to the storage guidelines to determine the appropriate amount of storage to allocate for recordings on the worker nodes.

3.4 KMS

Boundary Enterprise controllers and workers require different types of cryptographic keys. The KMS provider provides the root of trust for keys used for various purposes, such as protecting secrets, authenticating workers, recovering data, encrypting values in Boundary Enterprise's configuration. Refer to the KMS section for more information.

3.5 Traffic encryption

Boundary Enterprise is secure by default, and uses TLS for all network traffic communication. Boundary Enterprise has three types of connections, as described in the previous TLS section:

- Client-to-controller TLS
- Client-to-worker TLS
- Worker-to-upstream TLS

Refer to the TLS documentation for detailed information on each connection type.

From a load balancing requirement, always configure TLS passthrough. The load balancing section provides more information on this.

3.6 Load balancing

A layer 4 load balancer meets Boundary Enterprise's requirements. However, organizations may implement layer 7-capable load balancers for additional controls. Regardless of which, follow these requirements:

- HTTPS listener with valid TLS certificate for the domain it is serving or TLS passthrough
- Health checks should use TCP 9203

Each major cloud provider offers one or more managed load-balancing services suitable for Boundary Enterprise. Follow the guidance provided in the load balancer recommendations.

3.6.1 Client-to-controller

Place Boundary Enterprise controller nodes in a private network and not exposed directly to the public Internet. Expose services such as the API and administrative console via a load balancer. This design utilizes a layer 4 load balancer with additional network security controls, such as security groups or firewall access control lists, to restrict the network flow to the load balancer interface.

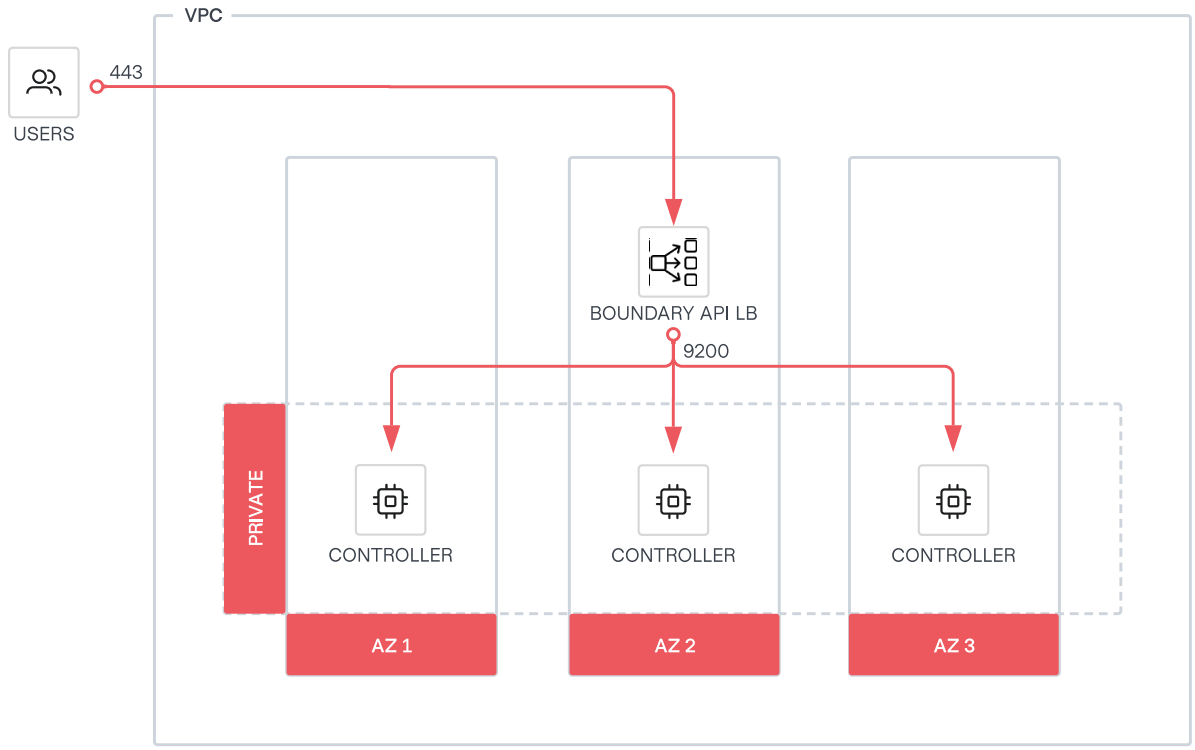


Figure 2: Boundary Enterprise client-to-controller

Health check

Use a load balancer to monitor the health of the Boundary Enterprise controller nodes. Do this by detecting the status of the `/health` endpoint.

This endpoint does not support any input. It returns an empty bodies to API responses.

Status	Description
200	GET `/health` returns HTTPS status `200` if the controller's API gRPC service is up.
5xx	GET `/health` returns HTTPS status `5xx` or request timeout if unhealthy.
503	GET `/health` returns HTTPS status `503` Service Unavailable status if the controller is shutting down.

Use the listener stanza to configure the controller's operational endpoints. By default, it listens on TCP 9203. The operational endpoint exposes both health and metrics endpoints.

Operational endpoint stanza configuration

```
# Ops listener for operations like health checks for load balancers
listener "tcp" {
  # Should be the address of the interface where your external systems' load balancer
       and/or metrics collectors etc. will connect on.
  address = "0.0.0.0:9203"
  # The purpose of this listener block
  purpose = "ops"

  tls_disable   = false
  tls_cert_file = "/etc/boundary.d/tls/boundary-cert.pem"
  tls_key_file  = "/etc/boundary.d/tls/boundary-key.pem"
}
```

3.6.2 Worker-to-controller

Similar to clients-to-controllers, ingress workers require access to Boundary Enterprise's controller nodes placed in a private network. For this design, where the deployment consists of a single cloud,

an internal load balancer would be sufficient to allow the ingress workers to establish connectivity via port TCP 9201 to the controllers.

For multi-cloud deployments operating a single control-plane, for example in AWS and targets with ingress workers in other clouds, or on-premise, it may be necessary to expose port TCP-9201 externally so that it is reachable. A consideration is to add another listener for port TCP-9201 to the load balancer used for client-to-controller communication.

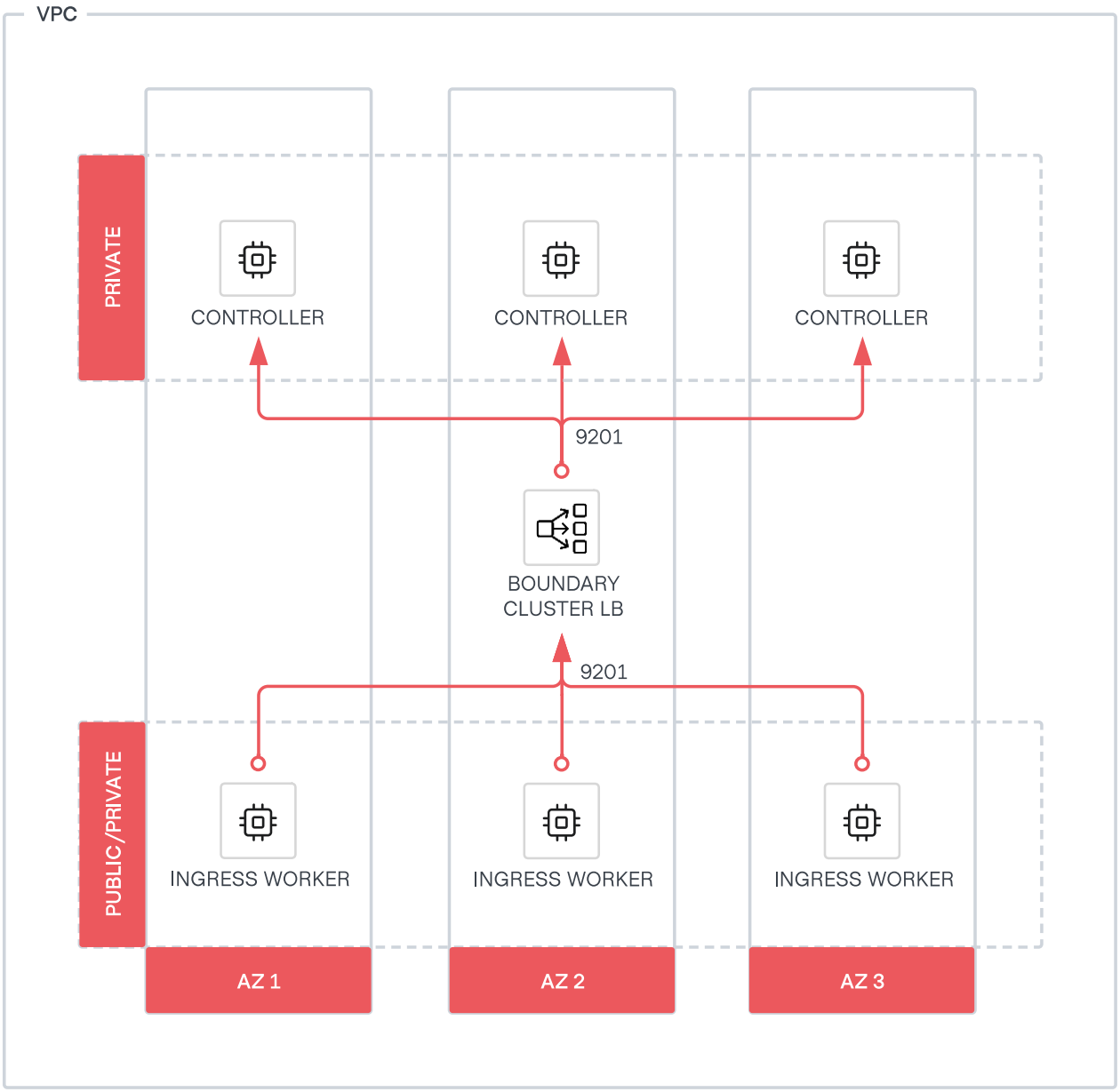


Figure 3: Boundary Enterprise Worker-to-controller

3.6.3 Worker-to-worker (multi-hop sessions)

With multi-hop sessions, workers operate as intermediaries or egress workers. If more than one provides identical capabilities (typically for increased availability, resilience, and scale), they should be part of a load-balanced set of workers. For example, configure the upstream configuration `initial_upstream` as the FQDN or virtual IP (VIP) address of the load-balanced pool of workers.

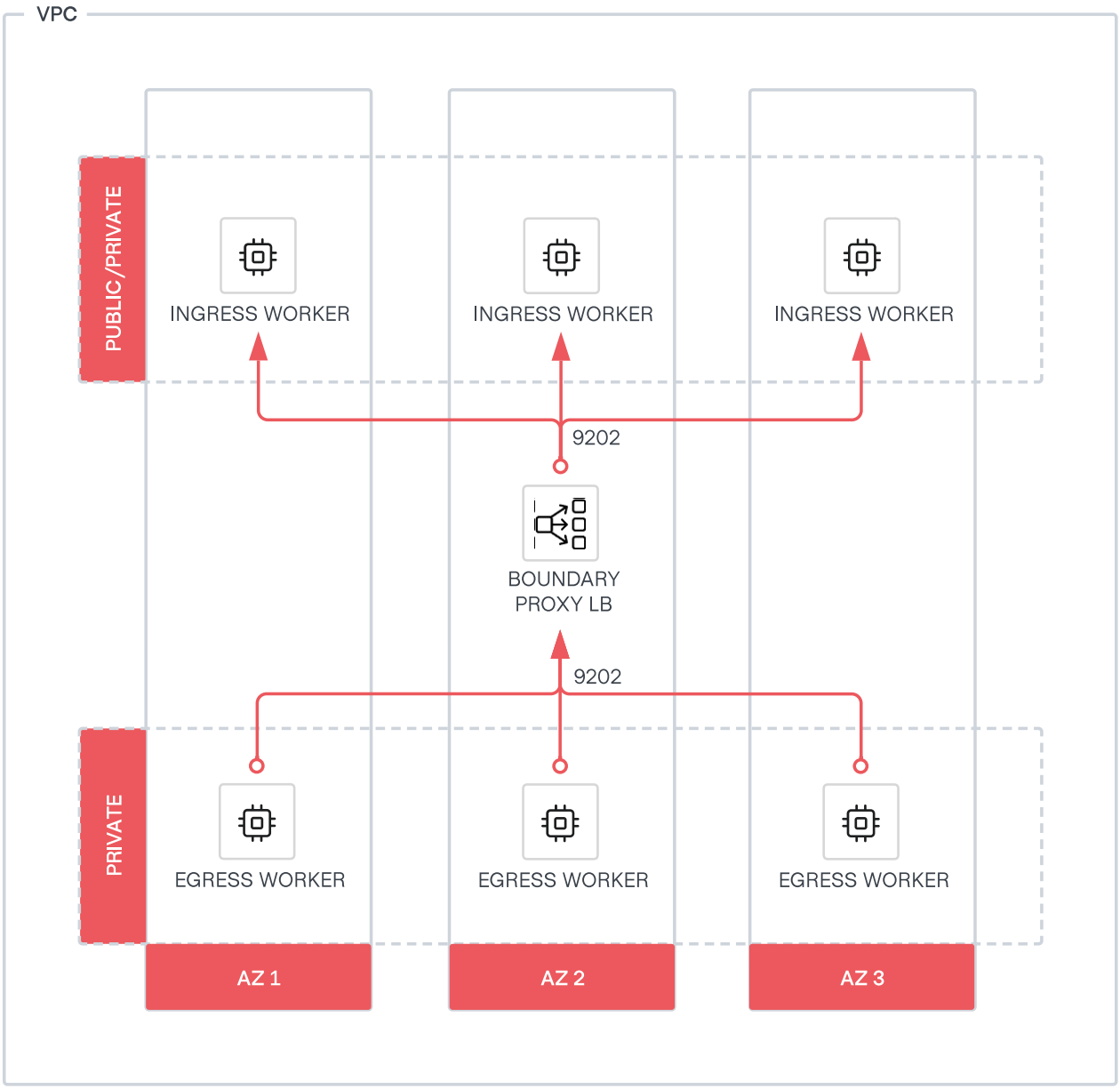


Figure 4: Boundary Enterprise Worker-to-worker

The upstream configuration `initial_upstream` allows a list of hosts/IPs. However, as workers can be dynamic, for example part of an auto scaling group, using a load balancer helps with future scale-out/in scenarios and ensures a robust architecture.

3.7 Monitoring

Gaining visibility into Boundary Enterprise's controllers and workers is essential for production environments. It enables operators to manage, scale, and troubleshoot Boundary Enterprise efficiently and assists in detecting and mitigating anomalies in a deployment.

3.7.1 Logs

If events are not configured, for example via the events stanza, Boundary Enterprise outputs logs to stdout/stderr by default. Linux distributions typically capture Boundary Enterprise's log output to the system journal.

In production environments, use the events stanza increase control over event logging. Event logging configured via the events stanza overrides the default behavior. For example, if configuring Boundary Enterprise to send events to a file, logs are no longer emitted to stdout or stderr.

Aggregate logs using a centralized platform for analysis, audit, and compliance, and aid in troubleshooting.

Minimum configuration

```
events {
  audit_enabled = true
  observations_enabled = true
  sysevents_enabled = true
  telemetry_enabled = false
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "hclog-text"
  }
}
```

3.7.2 Metrics

Metrics for controllers and workers are available for ingestion by third-party telemetry platforms, such as Prometheus and Grafana. The metrics use the OpenMetric exposition format. Refer to the Boundary Enterprise metrics documentation for a list of all available metrics.

Boundary Enterprise provides metrics through the /metrics path using a listener with the "ops" purpose.

Configure the controller's operational endpoints using the listener stanza. By default, it listens on TCP-9203. The operational `ops` listener exposes both health and metrics endpoints.

Operational endpoint stanza configuration

```
# Ops listener for operations like health checks for load balancers
listener "tcp" {
  # Should be the address of the interface where your external systems'
  # (eg: Load-Balancer and metrics collectors) will connect on.
  address = "0.0.0.0:9203"
  # The purpose of this listener block
  purpose = "ops"

  tls_disable   = false
  tls_cert_file = "/etc/boundary.d/tls/boundary-cert.pem"
  tls_key_file  = "/etc/boundary.d/tls/boundary-key.pem"
}
```

3.8 Failure considerations

Organizations should rely on the experience from their architecture, cloud, and platform teams to provide the appropriate levels of availability for Boundary Enterprise that meet their requirements. This architecture design leverages several principles and standard infrastructure services with major cloud providers to provide the highest availability and fault tolerance while balancing costs.

- **Auto scaling** to enhance fault tolerance. For example, if a Boundary Enterprise instance is unhealthy, the auto scaling service can replace it. You can also configure the service to use multiple availability zones and launch instances in another availability zone to compensate if one becomes unavailable.
- **Templating** images to decrease the time to deploy controllers and workers during the initial deployment, notably failure and scaling scenarios.
- **Infrastructure-as-code** to produce consistent, known deployments that reduce configuration errors.
- **Availability zones** to protect against data center failures. Spread Boundary Enterprise components across at least three availability zones in production environments. If deploying Bound-

ary Enterprise to three availability zones is not possible, you can use the same architecture across one or two availability zones at the expense of a reliability risk in case of an availability zone outage.

- **Load balancing** to provide traffic redirection and health checking.

3.8.1 Controllers

Boundary Enterprise controllers are stateless. They store all state and configuration within PostgreSQL and can withstand failure scenarios where only one node is accessible. When a controller node fails, users are still be able to interact with other Boundary Enterprise controllers, assuming the presence of additional nodes behind a load balancer. Boundary Enterprise controllers depend on a PostgreSQL database. Ensure the database is reachable to all Boundary Enterprise controller nodes. It should also inherit the same levels of availability as the controllers. Do not deploy PostgreSQL on the controller nodes.

3.8.2 Workers

Boundary Enterprise uses workers as either proxies or reverse proxies. Workers routinely communicate with the controllers to report their health. In the event of a worker node failure, it is best practice to have at least three workers per network boundary and per type (ingress and egress) for production environments. Therefore, the controller assigns a user's proxy session to an active Boundary Enterprise worker node.

3.8.3 Availability zone failures

The following section provides recommendations for controllers and workers to overcome availability zone outages.

Controllers

By deploying Boundary Enterprise controllers in the recommended architecture across three availability zones with load balancing in front of them, the Boundary Enterprise control plane can survive outages in up to 2 availability zones.

Workers

The best practice for deploying Boundary Enterprise workers is to have at least one worker deployed per availability zone. Provided correct security rules exist to allow for cross-subnet/AZ communication, should networking still be up in an otherwise-failed AZ, Boundary Enterprise proxies a user's session connection through a worker in another AZ.

Controllers

To continue to serve Boundary Enterprise controller requests during a regional outage, a deployment like the one outlined in this guide must be in a different region. Use a multi-regional database technology to allow the nodes in the secondary region to communicate with the PostgreSQL database. For example, promote secondary region AWS RDS read replicas to read-write in the event the primary region fails.

Use services like AWS Global Accelerator for AWS, Cross-region Load Balancer for Azure, and GCP Cloud Load Balancer for GCP to load balance healthy Boundary Enterprise controller requests across regions.

Workers

If a Boundary Enterprise worker cannot reach its upstream worker or a controller, the user cannot establish a proxied session to the target.

4 Deploying Boundary Enterprise using Terraform

HashiCorp provides a set of official HVD modules to make it easier to deploy a Boundary Enterprise environment that adheres to the requirements and standards laid out in this HashiCorp Validated Design.

4.1 Platform-specific guidance

AWS

4.1.1 Deployment on AWS EC2HashiCorp Provides official HVD Modules to deploy Boundary Enterprise controllers and workers on AWS EC2.Before deployment, deploy the prerequisite infrastructure in AWS as below.1) A functional VPC with the required subnets.

2) A Boundary Enterprise license stored in AWS Secrets Manager.
3) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to AWS Secrets Manager.
4) The ARN of the Boundary database password secret in AWS Secrets Manager.

Azure

4.1.2 Deployment on Azure VMsHashiCorp provides official HVD Modules to deploy Boundary Enterprise controllers and workers on Azure VMs.Before deployment, deploy the prerequisite infrastructure in Azure.1) An Azure resource group.

2) An Azure Key Vault to store the prerequisite secret material.
3) A Boundary Enterprise license stored in Azure Key Vault.
4) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to Azure Key Vault.
5) The name of the Boundary database password secret in Azure Key Vault.

GCP

4.1.3 Deployment on GCP GCEHashiCorp provides official HVD Modules to deploy Boundary Enterprise controllers and workers on GCP GCE.Before deployment, deploy the prerequisite infrastructure in GCP.1) A function VPC with a public and private subnet.

2) A Google Secret Manager to store the prerequisite secret material.
3) A Boundary Enterprise license stored in Google Secret Manager.
4) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to Google Secret Manager.
5) The version of the Boundary database password secret in Google Secret Manager.

Use these modules with Terraform to deploy a complete, end-to-end Boundary Enterprise deployment inside of your own cloud account.

While we have made efforts throughout this document to provide prescriptive best practices, we recognize that each organization has their own unique requirements and constraints when it comes to deploying infrastructure. Where possible, we have included considerations needed when deploying Boundary in your cloud environment within the context of this Terraform module. The module contains additional capabilities that you may wish to review if the variables from this module do not suit your specific needs.

4.2 Deployment sequence overview

1. Deploy the prerequisite resources.
2. Obtain the license file.
3. Download the Boundary command-line tool (or the Boundary desktop client).
4. Download the Terraform command-line tool.
5. Obtain the HashiCorp Validated Design Terraform module for deploying Boundary Enterprise controllers and workers using the links for your CSP in the preceding tab.
6. Configure your cloud credentials.
7. Initialize your Terraform workspace.
8. Input your variables, including the values from your prerequisite deployment, in the module for deployment.
9. Create a Terraform plan for the controller.
10. Apply the plan.
11. Bootstrap the Boundary controller.
12. Create a Terraform plan for the worker(s).
13. Apply the plan.
14. Begin creating targets and using Boundary Enterprise.

4.3 Preparation

4.3.1 Create the certificate files

Create a standard X.509 certificate that to install on the Boundary servers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users

to when accessing Boundary.

Ensure the following files are available.

- The TLS certificate (tls-cert-secret.pub).
- The TLS private key (tls-cert-private.key).
- The CA bundle file from the certificate authority used to vend the certificate (tls-ca-bundle.pub).

Keep these files to hand, as they you need them later in the installation process.

4.3.2 Obtain the Boundary enterprise license file

Obtain the Boundary Enterprise license file from your HashiCorp account team. This file contains a license key unique to your environment. Name the file something like *boundary.hclic*.

Keep this file to hand also, as you need it later in the installation process.

4.3.3 Download and install the Boundary command-line tool

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the *boundary* binary (boundary.exe for Windows) to a directory in your system's *PATH*.
- *Optional*: Install Boundary Desktop client

4.3.4 Download and install the Terraform command-line tool

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the Terraform binary (terraform.exe for Windows) to a directory in your system's *PATH*.

4.3.5 Download the Terraform module(s)

For the purpose of an automated deployment, Use these Terraform modules for your deployment, customizing where necessary. Once you have downloaded the module, navigate to the `examples/default/` directory. Use this as the base working directory during the installation process.

4.3.6 Configure cloud credentials

Use the tabs below to configure cloud credentials.

AWS

Ensure the correct AWS credentials are in place and accessible to Terraform. Terraform can read credentials from:* Credentials file: typically located at `$HOME/.aws/credentials` (`%UserProfile%\.aws\credentials` on Windows). * Environment variables as follows. - AWS_ACCESS_KEY_ID - AWS_SECRET_ACCESS_KEY - AWS_SESSION_TOKEN (if using an IAM role or other expiring credentials) - AWS_DEFAULT_REGIONFor complete details on how to configure AWS credentials for Terraform, see the HashiCorp Terraform AWS provider documentation.Ensure that the credentials have sufficient permissions to allow Terraform to perform the necessary actions.

Azure

Ensure the correct Azure credentials are in place and accessible to Terraform by running *az login* with your assumed credentials. For complete details on how to configure Azure credentials for Terraform, see the HashiCorp Terraform Azure provider documentation.Ensure the credentials have sufficient permissions to perform the necessary Terraform actions.

GCP

Ensure the correct GCP credentials are in place and accessible to Terraform. Terraform can read credentials from:* Credentials file: typically located at `$HOME/.config/gcloud/application_default_credentials.json` (`%APPDATA%\gcloud\application_default_credentials.json` on Windows). * Environment variables as follows. - `GOOGLE_CREDENTIALS`For complete details on how to configure GCP credentials for Terraform, see the HashiCorp Terraform GCP provider documentation.

Ensure the credentials have sufficient permissions to perform the necessary Terraform actions.

4.4 Installation

4.4.1 Initialize Terraform

Run `terraform init` to initialize your Terraform workspace and ensure there are no outstanding errors before continuing.

4.4.2 Configure variables for deployment

> Warning
>
> You can only configure variables for the installation module's `terraform.tfvars` file after all the prerequisite resources are available. You need to supply values from the prerequisites to the Vault module.

Review the `terraform.tfvars.example` file HashiCorp maintains in the `examples/default/` directory for explanations of each relevant variable. There is a `terraform.tfvars.example` file in the respective module for each public cloud provider. Copy this file to a file called `terraform.tfvars`, and then fill in the values for each declared variable with the applicable values for your environment.

4.4.3 Create and apply Terraform plan

From the `examples/default/` directory, generate a Terraform plan with the following command:

```
terraform plan -out=tfplan
```

Review the plan output, then apply the changes with this command:

```
terraform apply tfplan
```

Confirm the changes by typing `yes` when prompted.

4.4.4 Validate installation

After your `terraform apply` finishes, you can monitor the installation progress by connecting to your Boundary controller VM instance shell via SSH, AWS SSM, or Google IAP and observing the cloud-init (`user_data`) logs using the commands below in separate terminal windows.

```
tail -f /var/log/boundary-cloud-init.log
journalctl -xu cloud-final -f
```

> Note
>
> The `-f` argument is to follow the logs as they append in real time, and is optional. You may remove the `-f` for a static view.

The log files should display the following message after the cloud-init (`user_data`) script finishes.

```
[INFO] boundary_custom_data script finished successfully!
```

Once the cloud-init script finishes, while still connected to the VM via SSH or equivalent, you can check the status of the Boundary service has the line below.

```
[INFO] boundary_custom_data script finished successfully!
```

From the terminal where you performed the `terraform apply` run the following command.

```
terraform output
```

Using the Terraform output command that references the load balancer name or IP address, create a new DNS entry that matches your TLS certificate and points to the load balancer for the **Vault** cluster. Set the following environment variable.

```
export BOUNDARY_ADDR="https://boundary.example.com"
```

4.4.5 Bootstrapping Boundary Enterprise

After deploying a Boundary Enterprise controller the system is in a partially initialized state. To complete initialization and configuration for initial authentication utilize the bootstrapping module.

Repeat the steps starting from the preceding *Initialize Terraform* section.

After bootstrapping is complete you should now be able to authenticate to the Boundary cluster via command-line tool or administrator UI.

4.4.6 Install Boundary workers

> Note
>
> Boundary workers in all instances require access to either the controller or the upstream workers. See the Network Connectivity page for more information.

Utilize the Boundary Enterprise worker HVD module for AWS, Azure, or GCP and repeat the steps starting from *Initialize Terraform* for this new module.

After your `terraform apply` finishes, monitor the installation progress by connecting to your Boundary worker VM instance shell via SSH, AWS SSM, or Google IAP and observing the cloud-init (user_data) logs using the commands below in different terminal windows.

```
tail -f /var/log/boundary-cloud-init.log
journalctl -xu cloud-final -f
```

> Note
>
> The `-f` argument is to follow the logs as they append in real time, and is optional. You may remove the `-f` for a static view.

The log files should display the following message after the cloud-init (user_data) script finishes.

```
[INFO] boundary_custom_data script finished successfully!
```

Once the cloud-init script finishes, while still connected to the VM via SSH you can check the status of the boundary service using the command below.

```
sudo systemctl status boundary
```

After the Boundary worker has deployed, it is visible in the Boundary clusters' workers list.

5 Deploying Boundary Enterprise

5.1 Overview

This section details the steps to create a Boundary Enterprise cluster manually in a private datacenter. This guide assumes that you have already read the Recommended Deployment Architecture and Detailed Design section of this guide and have a basic understanding of the Boundary Enterprise architecture and the steps required to deploy it in a private datacenter.

This guide uses the following names and IP addresses for the Boundary Enterprise controllers and workers.

DNS Name	IP Address	Node Type	Location
controller-api-lb.boundary.domain	controller_api_lb_addr	Internet-facing controller API load balancer (TCP 443)	(all zones)
controller-cluster-lb.boundary.domain	controller_cluster_lb_address	Internal-facing controller cluster load balancer (TCP 9201)	(all zones)
controller1.boundary.do	10.0.253.11	Controller VM	zone1
controller2.boundary.domain	10.0.254.12	Controller VM	zone2
controller3.boundary.do	10.0.255.13	Controller VM	zone3
ingressworker-lb.boundary.domain	ingress_lb_address	Internal-facing ingress worker load balancer (TCP 9202)	(all zones)
ingressworker1.boundar	10.0.253.101	Ingress worker VM	zone1
ingressworker2.boundary.domain	10.0.254.102	Ingress worker VM	zone2

DNS Name	IP Address	Node Type	Location
ingressworker3.boundar	10.0.255.103	Ingress worker VM	zone3
egressworker1.boundary.domain	10.0.253.201	Egress worker VM	zone1
egressworker2.boundary	10.0.254.202	Egress worker VM	zone2
egressworker3.boundary.domain	10.0.255.203	Egress worker VM	zone3

5.2 Prepare

5.2.1 License

Obtain your active Boundary Enterprise license file. If you do not have this file, please contact your HashiCorp account team.

5.2.2 Servers

1. Refer to the Detailed Design section of the guide for more information on what to consider the sizing based on your environment.
2. Identify the availability zones within your datacenter where you plan to host your Boundary controllers and ingress/egress. For the rest of this document, we refer to these as zone1, zone2, zone3, and so on.
3. Build the servers. Ensure there is 1 Boundary Enterprise controller, ingress/egress worker per each of the 3 availability zones, for a total of 3 servers. For the rest of this document, we refer to these servers as controller1, controller2, controller3, ingressworker1, ingressworker2, ingressworker3, egressworker1, egressworker2, egressworker2, and so on.
4. Ensure you can log in to each server as a user with sudo or root privileges via SSH or equivalent.

5.2.3 Load balancer

1. A layer 4 load balancer exposes controller API and administrator UI via HTTPS (port 443) to Boundary Enterprise clients. The load balancer distributes Boundary Enterprise client requests to the controllers's API port (default TCP 9200).

2. Another layer 4 load balancer exposes the controller's cluster port (default TCP 9201) for workers session authorization, credentials, and so on.
3. Refer to the Detailed Design section of the guide for more information on how to configure the load balancer.

5.2.4 PostgreSQL

Controllers are stateless and manage all configurations through an external PostgreSQL database. We recommend configuring the PostgreSQL database for high availability. Please refer to PostgreSQL High Availability, Load Balancing, and Replication. If you use a managed service, refer to your provider's PostgreSQL high availability documentation.

5.2.5 Storage for session recording

We recommend using S3-compliant Object Storage for audit logging and session recording. Refer to the Detailed Design section of the guide for more information on how to configure storage for session recording.

5.3 Boundary Enterprise controller configuration

5.3.1 TLS

Create an X.509 certificate. Install this onto each of the Boundary Enterprise controllers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users to when accessing Boundary Enterprise. In this case, the DNS record pointing at the load balancer: `boundary.domain`. Please replace `boundary.domain` with your actual domain name.

You need these files:

1. The certificate (`cert.pem`).
2. The certificate's private key (`key.pem`).
3. The certificate authority bundle from a trusted certificate authority (`bundle.pem`).

You may have to create a new directory to store the certificate material at `/etc/boundary.d/tls`.

```
ls -l /etc/boundary.d/tls
-rw-r----- 1 boundary boundary 1801 Oct 17 03:47 bundle.pem
-rw-r----- 1 boundary boundary 1842 Oct 17 03:47 cert.pem
-rw-r----- 1 boundary boundary 1679 Oct 17 03:47 key.pem
```

Specify the following configuration in the `/etc/boundary.d/controller.hcl` file to contain the TLS certificates configuration.

```
# API listener configuration block
listener "tcp" {
  address              = "0.0.0.0:9200"
  purpose              = "api"
  tls_disable          = false
  tls_cert_file        = "/etc/boundary.d/tls/cert.pem"
  tls_key_file         = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file   = "/etc/boundary.d/tls/bundle.pem"
  cors_enabled         = true
  cors_allowed_origins = ["*"]
}

# Ops listener for operations like health checks for load balancers
listener "tcp" {
  address            = "0.0.0.0:9203"
  purpose            = "ops"
  tls_disable        = false
  tls_cert_file      = "/etc/boundary.d/tls/cert.pem"
  tls_key_file       = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file = "/etc/boundary.d/tls/bundle.pem"
}
```

> **Note**
>
> We recommend storing the TLS material and license key for Boundary Enterprise controllers in HashiCorp Vault, HCP Vault Secrets or cloud provider equivalents.

5.3.2 KMS for controllers

You need four KMS keys:

1. root

- The root key is the primary encryption key used by Boundary Enterprise.

2. worker-auth
   - The controller and worker share this to authenticate a worker to the controller.
3. recovery
   - Use this key for rescue/recovery operations in case of system issues or when normal authentication methods are unavailable.
4. bsr
   - Use this key for session recording capabilities. Recording of session data uses this key and ensures the integrity of those recordings.

Specify the following configuration in the `/etc/boundary.d/controller.hcl` file to contain the KMS keys configuration.

```
# Root KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "root"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey1"
}

# Recovery KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "recovery"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey2"
}

# Worker-Auth KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}

# BSR KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "bsr"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey4"
}
```

Refer to the Vault Transit page to configure Boundary Enterprise to use Vault's Transit secret engine for key management. If you use a managed service, refer to our KMS documentation for guidance.

5.3.3 Prepare controller configuration to initialize a PostgreSQL database

Use the following in the `/etc/boundary.d/controller.hcl` file to configure the database URL.

```
# Controller configuration block
controller {
  name        = "<controller1>"           # update here for other controllers
  description = "<Boundary Controller 1>" # update here for other controllers
  database {
    url = "postgresql://POSTGRESQL_CONNECTION_STRING"
  }

  license = "file:////opt/boundary/license/license.hclic"
}
```

5.3.4 Prepare Boundary Enterprise controller configuration

Populate the `/etc/boundary.d/controller.hcl` file with the configuration information below.

Use the following controller configuration on each controller node, replacing the name, description, database URL, license path, listener IP address and KMS configuration in each case.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# Controller configuration block
controller {
  name        = "<controller1>"                      # update here for other
      controllers
  description = "<Boundary Enterprise Controller 1>" # update here for other
      controllers
  database {
    url = "postgresql://POSTGRESQL_CONNECTION_STRING"
  }

  license = "file:////opt/boundary/license/license.hclic"
}

# API listener configuration block
listener "tcp" {
  address              = "0.0.0.0:9200"
  purpose              = "api"
  tls_disable          = false
  tls_cert_file        = "/etc/boundary.d/tls/cert.pem"
  tls_key_file         = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file   = "/etc/boundary.d/tls/bundle.pem"
  cors_enabled         = true
  cors_allowed_origins = ["*"]
}

# Data-plane listener configuration block (used for worker coordination)
listener "tcp" {
  address = "<10.0.253.11>:9201" # update here for other controllers IP address
  purpose = "cluster"
}

# Ops listener for operations like health checks for load balancers
listener "tcp" {
  address            = "0.0.0.0:9203"
  purpose            = "ops"
  tls_disable        = false
  tls_cert_file      = "/etc/boundary.d/tls/cert.pem"
  tls_key_file       = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file = "/etc/boundary.d/tls/bundle.pem"
}

# Events (logging) configuration. This configured logging for ALL events to both
# stderr and a file at /var/log/boundary/controller.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
```

5.3.5 Initialize a PostgreSQL database

Before you can start Boundary Enterprise, you must initialize the database from one controller.

The following command initializes a Boundary Enterprise's database with the configuration specified in the `/etc/boundary.d/controller.hcl` file:

```
boundary database init -config=/etc/boundary/controller.hcl
```

5.3.6 Starting Boundary Enterprise controller service

When the configuration files are in place on each Boundary Enterprise controller, you can proceed to enable and start the binary via `systemd` on each of the Boundary Enterprise controller nodes.

Perform these steps on all Boundary Enterprise controllers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
$ adduser --system --group boundary || true
$ mkdir -p /etc/boundary.d /etc/boundary.d/tls /opt/boundary/license /var/log/
    boundary
$ chown -R boundary:boundary /etc/boundary.d /var/log/boundary
```

1. Download the Boundary Enterprise package from HashiCorp. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user.

```
$ curl -O https://releases.hashicorp.com/boundary/0.17.1+ent/boundary_0.17.1+
    ent_linux_amd64.zip
$ unzip boundary_0.17.1+ent_linux_amd64.zip
$ mv boundary /usr/local/bin/
$ chown boundary:boundary /usr/local/bin/boundary
```

1. Add the license file, certificate files, and relevant config file to `/etc/boundary.d`. In the end, the directory should look like the below.

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary 1652 Oct 17 03:47 controller.hcl
drwxr-x---  2 boundary boundary 4096 Oct 17 03:47 tls

$ ls -l /etc/boundary.d/tls
-rw-r----- 1 boundary boundary 1801 Oct 17 03:47 bundle.pem
-rw-r----- 1 boundary boundary 1842 Oct 17 03:47 cert.pem
-rw-r----- 1 boundary boundary 1679 Oct 17 03:47 key.pem

$ ls -l /opt/boundary/license
-rw-rw-r-- 1 root     root     3514 Aug 15 18:21 EULA.txt
-rw-rw-r-- 1 root     root     4922 Aug 15 18:21 LICENSE.txt
-rw-rw-r-- 1 root     root     9518 Aug 15 18:21 TermsOfEvaluation.txt
-rw-r--r-- 1 root     root     1163 Oct 17 03:47 license.hclic

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

1. Create a `systemd` unit file for the Boundary Enterprise service, then load it into `systemd`. Note that the `ExecStart` line runs the `boundary` binary pointing to your `controller.hcl` file.

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://developer.hashicorp.com/boundary/docs
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/controller.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/controller.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
$ systemctl daemon-reload
$ systemctl enable boundary
```

1. Start the Boundary Enterprise controller service.

```
$ systemctl start boundary
```

5.4 Boundary Enterprise ingress workers configuration

5.4.1 KMS for ingress workers

Get the `worker-auth` KMS key from the KMS for controllers section. This key enables secure communication between workers and controllers, ensuring that only authorized workers can connect to each other.

Specify the following configuration in the `/etc/boundary.d/worker.hcl` file to contain the KMS keys configuration.

```
# Worker-auth KMS key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}
```

5.4.2 Prepare ingress workers configuration

Populate the `/etc/boundary.d/worker.hcl` file with the configuration information below replacing content prompted in angled brackets with the correct characters for your deployment.

All three ingress workers' configurations are the same except the `worker configuration stanza`
.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# worker block for configuring the specifics of the worker service
worker {
  public_addr = "<10.0.253.101>"      # update here for other ingress workers ip
      address
  name = "<ingressworker1>"           # update here for other ingress workers name
  initial_upstreams = ["<controller_cluster_lb_address>:9201"]
  recording_storage_path="/opt/boundary/bsr"
  recording_storage_minimum_available_capacity="500MB"
  tags {"app"="worker","env"="uat","bsr"="enabled","worker-type"="ingress"}
}

# listener denoting this is a worker proxy
listener "tcp" {
  address            = "0.0.0.0:9202"
  purpose            = "proxy"
}

# Ops listener for operations like health checks for ingress workers
listener "tcp" {
  address     = "0.0.0.0:9203"
  purpose     = "ops"
  tls_disable = true
}

# Events (logging) configuration. This
# configured logging for ALL events to both
# stderr and a file at /var/log/boundary/ingress-worker.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "cloudevents-json"
  }
  sink {
    name = "file-sink"
    description = "All events sent to a file"
    event_types = ["*"]
    format = "cloudevents-json"
    file {
      path = "/var/log/boundary"
      file_name = "ingress-worker.log"
    }
    audit_config {
      audit_filter_overrides {
        sensitive = "redact"
```

5.4.3 Starting Boundary Enterprise ingress worker service

When the configuration files are in place on each Boundary Enterprise ingress worker, you can proceed to enable and start the binary via `systemd` on each of the Boundary Enterprise ingress worker nodes.

Perform these steps on all Boundary Enterprise ingress workers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
$ adduser --system --group boundary || true
$ mkdir -p /etc/boundary.d /opt/boundary/bsr /var/log/boundary
$ chown -R boundary:boundary /etc/boundary.d /opt/boundary/bsr /var/log/boundary
```

1. Use the official well-architected method to download the Boundary Enterprise binary and signature files from HashiCorp and confirm the integrity using the `gpg` binary. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user as per the below.

```
$ unzip boundary_0.17.1+ent_linux_amd64.zip
$ mv boundary /usr/local/bin/
$ chown boundary:boundary /usr/local/bin/boundary
```

1. Add relevant config file to `/etc/boundary.d`. In the end, the directory should look like this:

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary  704 Oct 17 06:23 worker.hcl

$ ls -l /opt/boundary/
drwxr-x--- 3 boundary boundary 4096 Oct 17 06:23 bsr
drwxr-x--- 2 boundary boundary 4096 Oct 17 06:23 data

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

1. Create a systemd unit file for the Boundary Enterprise service, then load it into `systemd`. Note

that the `ExecStart` line runs the `boundary` binary pointing to your `worker.hcl` file:

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://www.boundaryproject.io/docs/
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/worker.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/worker.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
$ systemctl daemon-reload
$ systemctl enable boundary
```

1. Start the Boundary Enterprise ingress worker service.

```
$ systemctl start boundary
```

5.5 Boundary Enterprise egress workers configuration

5.5.1 KMS for egress workers

Get the `worker-auth` KMS key from the KMS for controllers section. This key enables secure communication between workers and controllers, ensuring that only authorized workers can connect to each other.

Use the following KMS configuration block in the `/etc/boundary.d/worker.hcl` file.

```
# Worker-Auth KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}
```

5.5.2 Prepare egress workers configuration

Populate the `/etc/boundary.d/worker.hcl` file with the configuration information below.

The configuration of all three egress workers should be identical except the `worker` and `kms` blocks. Amend these as necessary.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# worker block for configuring the specifics of the worker service
worker {
  public_addr = "<10.0.253.201>" # update here for other egress workers IP address
  name = "<egressworker1>"       # update here for other egress workers name
  initial_upstreams = ["<ingress_lb_address>:9202"]
  recording_storage_path="/opt/boundary/bsr"
  recording_storage_minimum_available_capacity="500MB"
  tags {"app"="worker","env"="uat","bsr"="enabled","worker-type"="egress"}
}

# listener denoting this is a worker proxy
listener "tcp" {
  address            = "0.0.0.0:9202"
  purpose            = "proxy"
}

# Ops listener for operations like health checks for ingress workers
listener "tcp" {
  address     = "0.0.0.0:9203"
  purpose     = "ops"
  tls_disable = true
}

# Events (logging) configuration. This
# configured logging for ALL events to both
# stderr and a file at /var/log/boundary/egress-worker.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "cloudevents-json"
  }
  sink {
    name = "file-sink"
    description = "All events sent to a file"
    event_types = ["*"]
    format = "cloudevents-json"
    file {
      path = "/var/log/boundary"
      file_name = "egress-worker.log"
    }
    audit_config {
      audit_filter_overrides {
        sensitive = "redact"
        secret    = "redact"
```

5.5.3 Starting Boundary Enterprise egress worker service

When the configuration files are in place on each Boundary Enterprise egress worker, enable the binary via `systemd` on each of the Boundary Enterprise egress worker nodes.

Perform these steps on all Boundary Enterprise egress workers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
$ adduser --system --group boundary || true
$ mkdir -p /etc/boundary.d /opt/boundary/bsr /var/log/boundary
$ chown -R boundary:boundary /etc/boundary.d /opt/boundary/bsr /var/log/boundary
```

1. Use the official well-architected method to download the Boundary Enterprise binary and signature files from HashiCorp and confirm the integrity using the `gpg` binary. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user as per the below.

```
$ unzip boundary_0.17.1+ent_linux_amd64.zip
$ mv boundary /usr/local/bin/
$ chown boundary:boundary /usr/local/bin/boundary
```

1. Add relevant config file to `/etc/boundary.d`. In the end, the directory should look like this:

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary  704 Oct 17 06:23 worker.hcl

$ ls -l /opt/boundary/
drwxr-x--- 3 boundary boundary 4096 Oct 17 06:23 bsr
drwxr-x--- 2 boundary boundary 4096 Oct 17 06:23 data

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

4. Create a `systemd` unit file for the Boundary Enterprise service, then load it into `systemd`. Note that the `ExecStart` line runs the `boundary` binary pointing to your `worker.hcl` file:

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://www.boundaryproject.io/docs/
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/worker.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/worker.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
$ systemctl daemon-reload
$ systemctl enable boundary
```

5. Start the Boundary Enterprise egress worker service.

```
$ systemctl start boundary
```

5.6 Next steps

After setting up a Boundary Enterprise cluster, it is essential to perform initial configuration steps to ensure the environment is secure, functional, and ready for use. Refer to Initial Configuration in the Boundary Enterprise: Operating Guide for Adoption.